TCL scripting in VLSI EXPERT

The **tclsh** command in Linux starts the Tcl shell, which is an interpreter for running TCL (Tool Command Language) scripts. It reads and evaluates Tcl commands from standard input or from a file.

The tcksh command invokes the Tcl shell.

- Check the path, where installed the tool/or not-

[b23n_sarif_727@prac3 Documents]$ cd tck_lab/

[b23n_sarif_727@prac3 tck_lab]$ ls

[b23n_sarif_727@prac3 tck_lab]$ which tclsh

/usr/bin/tclsh ----- it is showing the path installed the tool—binary file path

[b23n_sarif_727@prac3 tck_lab]$ which tclsh

/usr/bin/tclsh

[b23n_sarif_727@prac3 tck_lab]$ tclsh

% puts "Hello world"

Hello world

% set a 12

12

% set b 45

45

% set c [expr $a + $b]

57

% puts "The sum of $a & $b is $c"

The sum of 12 & 45 is 57

%

%

% exit ---- for coming outside of tclsh inside loop

[b23n_sarif_727@prac3 tck_lab]$ ls

[b23n_sarif_727@prac3 tck_lab]$

[b23n_sarif_727@prac3 tck_lab]$ ls

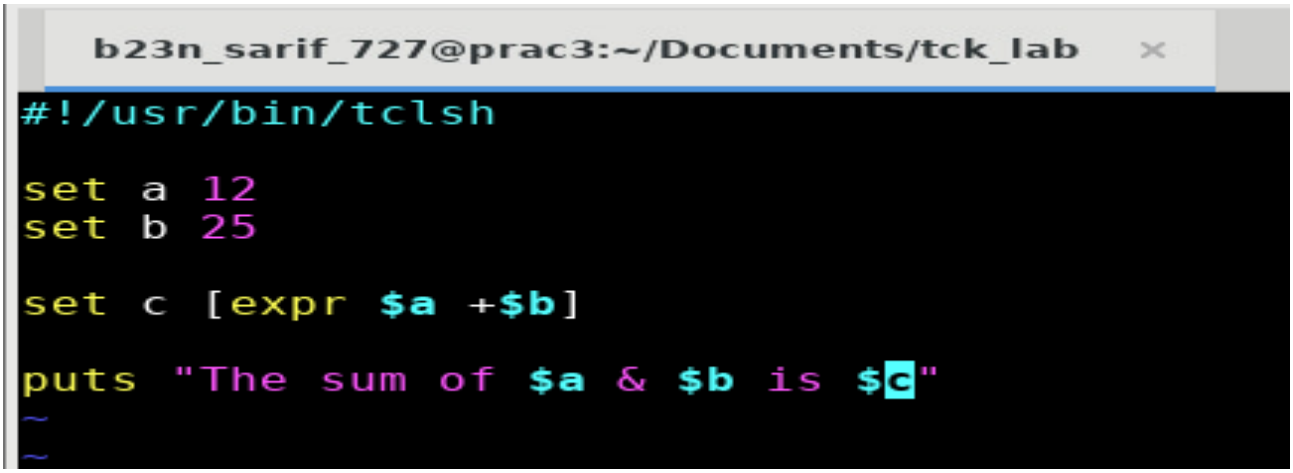
[b23n_sarif_727@prac3 tck_lab]$ vi sum1.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum1.tcl

bash: ./sum1.tcl: Permission denied

[b23n_sarif_727@prac3 tck_lab]$ chmod 777 sum1.tcl

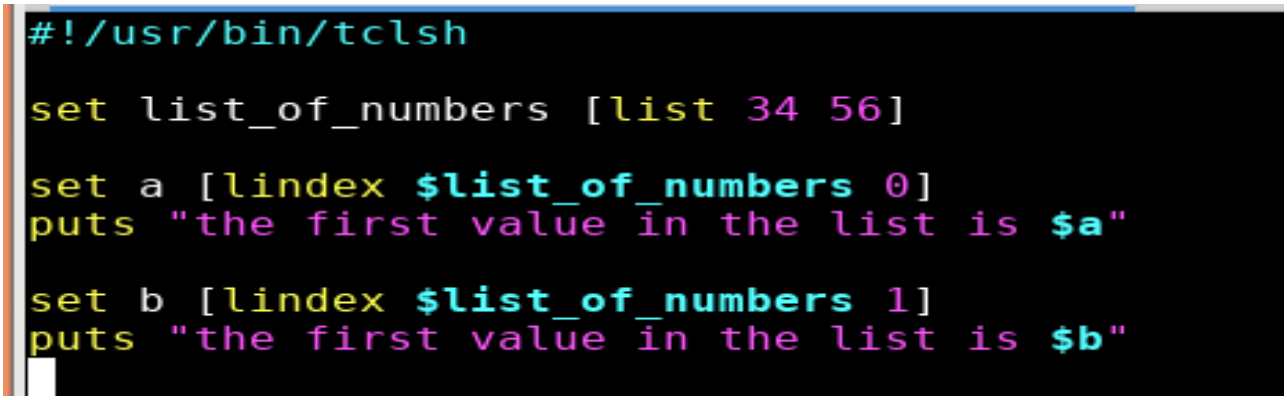
[b23n_sarif_727@prac3 tck_lab]$ ./sum1.tcl



The sum of 12 & 25 is 37

[b23n_sarif_727@prac3 tck_lab]$ vi sum1.tcl

[b23n_sarif_727@prac3 tck_lab]$ vi sum2.tcl



[b23n_sarif_727@prac3 tck_lab]$ chmod 777 sum2.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum2.tcl

the first value in the list is 34

the first value in the list is 56

#!/usr/bin/tclsh

set list_of_numbers [list 34 56]

set a [lindex $list_of_numbers 0]

puts "the first value in the list is $a"

set b [lindex $list_of_numbers 1]

puts "the first value in the list is $b"

set c [expr $a + $b]

puts "the sum of $a & $b is $c"

```
#!/usr/bin/tclsh

set list_of_numbers [list 34 56]

set a [lindex $list_of_numbers 0]
puts "the first value in the list is $a"

set b [lindex $list_of_numbers 1]
puts "the first value in the list is $b"

set c [expr $a + $b]
puts "the sumo of $a & $b is $c"
~
```

[b23n_sarif_727@prac3 tck_lab]$ vi sum2.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum2.tcl

the first value in the list is 34

the first value in the list is 56

the sum of 34 & 56 is 90

```
#!/usr/bin/tclsh

set list_of_numbers [list 12 34 45 56 78  25]
set sum 0
foreach var1 $list_of_numbers {
        set sum [expr $sum + $var1]
}

puts "The sum of $list_of_numbers number is $sum"
~
```

[b23n_sarif_727@prac3 tck_lab]$ vi sum3.tcl

[b23n_sarif_727@prac3 tck_lab]$ chmod 777 sum3.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum3.tcl

The sum of 12 34 45 56 78 25 number is 250

[b23n_sarif_727@prac3 tck_lab]$

#!/usr/bin/tclsh

set list_of_numbers [list 1 2 3 4 12 34 45 56 78 25]

set sum 0

foreach var1 $list_of_numbers {

    set sum [expr $sum + $var1]

}

puts "The sum of $list_of_numbers number is $sum"

[b23n_sarif_727@prac3 tck_lab]$ vi sum3.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum3.tcl

The sum of 1 2 3 4 12 34 45 56 78 25 number is 260

[b23n_sarif_727@prac3 tck_lab]$

```
#!/usr/bin/tclsh
proc sum_of_nos { n1 n2 } {
        set sum [expr $n1 +$n2]
        puts "The sum of $n1 + $n2 is $sum"
}
sum_of_nos 34 78
sum_of_nos 40 60
[b23n_sarif_727@prac3 tck_lab]$ vi sum4.tcl
[b23n_sarif_727@prac3 tck_lab]$ chmod 777 sum4.tcl
[b23n_sarif_727@prac3 tck_lab]$ ./sum4.tcl
The sum of 34 + 78 is 112
The sum of 40 + 60 is 100
```

Part-2

By the user at the run time- more easy, relevant and user friendly

```
#!/usr/bin/tclsh

#set a 12
#set b 25

puts "Enter the first value of variable a"
set a [gets stdin]
puts "Enter the second value of variable b"
set b [gets stdin]

set c [expr $a +$b]
```

```
puts "The sum of $a & $b is $c"

[b23n_sarif_727@prac3 tck_lab]$ ls
sum1.tcl  sum2.tcl  sum3.tcl  sum4.tcl
[b23n_sarif_727@prac3 tck_lab]$ vi sum1.tcl
[b23n_sarif_727@prac3 tck_lab]$ ./sum1.tcl
Enter the first value of variable a
56
Enter the second value of variable b
74
The sum of 56 & 74 is 130
[b23n_sarif_727@prac3 tck_lab]$ ./sum1.tcl          -- Again the code then provide input
Enter the first value of variable a
1500
Enter the second value of variable b
3500
The sum of 1500 & 3500 is 5000

#!/usr/bin/tclsh

#set list_of_numbers [list 34 56]

puts "Enter the two numbers seperated by space---"
set list_of_numbers [gets stdin]

set a [lindex $list_of_numbers 0]
```

```
puts "the first value in the list is $a"

set b [lindex $list_of_numbers 1]
puts "the second value in the list is $b"

set c [expr $a + $b]
puts "the sum of $a & $b is $c"

[b23n_sarif_727@prac3 tck_lab]$ vi sum2.tcl
[b23n_sarif_727@prac3 tck_lab]$ ./sum2.tcl
Enter the two numbers seperated by space---
45 55
the first value in the list is 45
the second value in the list is 55
the sum of 45 & 55 is 100

#!/usr/bin/tclsh

#set list_of_numbers [list 1 2 3 4 12 34 45 56 78  25]
puts "Enter the numbers in the list sepearted by space ---"
set list_of_numbers [gets stdin]

set sum 0
foreach var1 $list_of_numbers {
        set sum [expr $sum + $var1]
}
```

puts "The sum of $list_of_numbers number is $sum"

[b23n_sarif_727@prac3 tck_lab]$ vi sum3.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum3.tcl

Enter the numbers in the list sepearted by space ---

1 2 3 4 5 6 7 8 9 10

The sum of 1 2 3 4 5 6 7 8 9 10 number is 55

[b23n_sarif_727@prac3 tck_lab]$ ./sum3.tcl

Enter the numbers in the list sepearted by space ---

10 24 26 40 60 50 100

The sum of 10 24 26 40 60 50 100 number is 310

```
#!/usr/bin/tclsh

proc sum_of_nos { n1 n2 } {
        set sum [expr $n1 +$n2]
        puts "The sum of $n1 + $n2 is $sum"
}

puts "Enter the 4 values in sequence seperated by enter key"
set a [gets stdin]
set b [gets stdin]
set c [gets stdin]
set d [gets stdin]

sum_of_nos $a $b
sum_of_nos $c $d

#sum_of_nos 34 78
#sum_of_nos 40 60
~
~
~
~
~
"sum4.tcl" 18L, 320C
```

[b23n_sarif_727@prac3 tck_lab]$ ./sum4.tcl

Enter the 4 values in sequence seperated by enter key

1

2

3

4

The sum of 1 + 2 is 3

The sum of 3 + 4 is 7

Part-3

Argument- Passing the Argument at run time

argv - argument value --- $argv ---also acting like as list and this is pre-defined function

argc - argument count --- $argc

```
#!/usr/bin/tclsh

# argv - argument value
# argc - argument count

puts "The aggument has a value as : $argv"
puts "The argument has count as : $argc"
~
```

[b23n_sarif_727@prac3 tck_lab]$ vi sum5.tcl

[b23n_sarif_727@prac3 tck_lab]$ chmod 777 sum5.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum5.tcl

The aggument has a value as :

The argument has count as : 0

[b23n_sarif_727@prac3 tck_lab]$ ./sum5.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum5.tcl ABCD abcd 1234 5678

The aggument has a value as : ABCD abcd 1234 5678

The argument has count as : 4

```
#!/usr/bin/tclsh

# argv - argument value
# argc - argument count

puts "The aggument has a value as : $argv"
puts "The argument has count as : $argc"

set sum 0
foreach var1 $argv {
        set sum [expr $sum +$var1]
}

puts "The sum of numbers entered in the argument is $sum"
```

[b23n_sarif_727@prac3 tck_lab]$ vi sum5.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum5.tcl 1234 34 78 50

The aggument has a value as : 1234 34 78 50

The argument has count as : 4

The sum of numbers entered in the argument is 1396

[b23n_sarif_727@prac3 tck_lab]$ ./sum5.tcl 1234 34 78 50 100 4

The aggument has a value as : 1234 34 78 50 100 4

The argument has count as : 6

The sum of numbers entered in the argument is 1500

Vi number.txt

score_1 12

score_2 34

score_3 52

score_4 60

[b23n_sarif_727@prac3 tck_lab]$ vi number.txt

[b23n_sarif_727@prac3 tck_lab]$ cat number.txt

score_1 12

score_2 34

score_3 52

score_4 60

```
#!/usr/bin/tclsh

set file_handler [open number.txt r]

while {[gets $file_handler data] >=0 } {
        puts "#####"
        puts "$data"
        puts "####"
}
```

[b23n_sarif_727@prac3 tck_lab]$ vi number.txt

[b23n_sarif_727@prac3 tck_lab]$ vi sum6.tcl

[b23n_sarif_727@prac3 tck_lab]$ chmod 777 sum6.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum6.tcl

#####

####

#####

score_1 12

####

#####

score_2 34

####

#####

score_3 52

####

#####

score_4 60

####

```
#!/usr/bin/tclsh

set file_handler [open number.txt r]

while {[gets $file_handler data] >=0 } {
#        puts "$data"
         set a [lindex $data 1]
         puts $a
}
```

[b23n_sarif_727@prac3 tck_lab]$ vi sum6.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./sum6.tcl

12

34

52

60

Vi number.txt

score_1 12

score_2 34

score_3 52

score_4 60

```
#!/usr/bin/tclsh

set file_handler [open number.txt r]

set sum 0
while {[gets $file_handler data] >=0 } {
#        puts "$data"
         set a [lindex $data 1]
#        puts $a
         set sum [expr $sum + $a]
}

puts "the sum of numbers in file is $sum"
```

[b23n_sarif_727@prac3 tck_lab]$ vi number.txt

[b23n_sarif_727@prac3 tck_lab]$ ./sum6.tcl

the sum of numbers in file is 158

Print this line as a file itself----------

```
#!/usr/bin/tclsh

set file_handler [open number.txt r]

set sum 0
while {[gets $file_handler data] >=0 } {
#       puts "$data"
        set a [lindex $data 1]
#       puts $a
        set sum [expr $sum + $a]
}
close $file_handler
puts "the sum of numbers in file is $sum"

set file_handler_2 [open number.txt a]

puts $file_handler_2 "the sum of number in file is $sum"

close $file_handler_2

[b23n_sarif_727@prac3 tck_lab]$ vi sum6.tcl
[b23n_sarif_727@prac3 tck_lab]$ cat number.txt
score_1 12
score_2 34
score_3 52
```

score_4 60

[b23n_sarif_727@prac3 tck_lab]$ ./sum6.tcl

the sum of numbers in file is 158

……

set a 5

set b 10

set sum1 [expr {$a + $b}]

puts " Sum1 = $sum1"

set c 10

set sum2 [expr {$sum1 + $c}]

puts " Sum2 = $sum2"

[b23n_sarif_727@prac3 tck_lab]$ ./proc.tcl

Sum1 = 15

Sum2 = 25

```
#!/usr/bin/tclsh

# proc - Procedure (Function in C)

proc add {x y} {
    return [expr {$x + $y}]
}

set a 5
set b 10
set c 10

set sum1 [add $a $b]
set sum2 [add $sum1 $c]

puts "Print the value of Sum1 = $sum1"
puts "Print the value of Sum2 = $sum2"
```

[b23n_sarif_727@prac3 tck_lab]$ ./proc.tcl

Print the value of Sum1 = 15

Print the value of Sum2 = 25

#!/usr/bin/tclsh

proc - Procedure (Function in C)

```
proc sum {} {
        set a 4
        set b 5
        set add [expr $a + $b]
}
puts "The sum of a & b is [sum]"
```

[b23n_sarif_727@prac3 tck_lab]$ ./proc.tcl

The sum of a & b is 9

proc - Procedure (Function in C)

```
proc sum {} {
        set a 4
        set b 5
        set add [expr $a + $b]
}

puts "The sum of a & b is [sum]"
```

proc - Procedure (Function in C) with an argument

```
proc sum_arg {A B} {
        set ADD [expr $A + $B]
```

```
}

puts "The addition of 4 & 5 is [sum_arg 4 5]"
puts "The addition of 1 & 3 is [sum_arg 1 3]"
puts "The addition of 7 & 4 is [sum_arg 7 4]"
puts "The addition of 9 & 0 is [sum_arg 9 0]"
puts "The addition of 2 & 5 is [sum_arg 2 5]"
puts "The addition of 6 & 8 is [sum_arg 6 8]"
[b23n_sarif_727@prac3 tck_lab]$ ./proc.tcl

The sum of a & b is 9
The addition of 4 & 5 is 9
The addition of 1 & 3 is 4
The addition of 7 & 4 is 11
The addition of 9 & 0 is 9
The addition of 2 & 5 is 7
The addition of 6 & 8 is 14



#!/usr/bin/tclsh

# proc - Procedure (Function in C)

proc sum {} {
        set a 4
        set b 5
```

```
        set add [expr $a + $b]
}

puts "The sum of a & b is [sum]"

proc sum_arg {A B} {
        set ADD [expr $A + $B]
}

puts "The addition of 4 & 5 is [sum_arg 4 5]"
puts "The addition of 1 & 3 is [sum_arg 1 3]"
puts "The addition of 7 & 4 is [sum_arg 7 4]"
puts "The addition of 9 & 0 is [sum_arg 9 0]"
puts "The addition of 2 & 5 is [sum_arg 2 5]"
puts "The addition of 6 & 8 is [sum_arg 6 8]"
#puts "The addition of 4 & 4 is [sum_arg [gets stdin] [gets stdin]]"

puts "enter the value of a:"
set a [gets stdin]
puts "enter the value of b:"
set b [gets stdin]

set sum1 [sum_arg $a $b]
puts "The sum of $a & $b is $sum1"

[b23n_sarif_727@prac3 tck_lab]$ ./proc.tcl
The sum of a & b is 9
```

The addition of 4 & 5 is 9

The addition of 1 & 3 is 4

The addition of 7 & 4 is 11

The addition of 9 & 0 is 9

The addition of 2 & 5 is 7

The addition of 6 & 8 is 14

enter the value of a:

15

enter the value of b:

35

The sum of 15 & 35 is 50

TUTORIALS POINT

```
#!/usr/bin/tclsh

puts "Hello World"
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Hello World

Tcl - Special Variables

Sr. No.	argument	Special Variable & Description
1.0	argc	Refers to a number of command-line arguments.
2.0	argv	Refers to the list containing the command-line arguments.

```
#!/usr/bin/tclsh
```

puts "$tcl_version"

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

8.6

#!/usr/bin/tclsh

puts "$tcl_library"

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

/usr/share/tcl8.6

Tcl - Basic Syntax

First Tcl Program

All Tcl files will have an extension, i.e **.tcl.**

Comments

Comments can be written using a hash_(#) sign in the beginning.

Identifiers

A Tcl identifier is a name used to identify a variable, function, or any other user-defined item. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores, dollars ($) , and digits (0 to 9).

Tcl does not allow punctuation characters such as @, and % within identifiers. Tcl is a **case-sensitive** language.

Example: name, sarif, a_123, _123, name2, abc

Whitespace in Tcl

A line containing only whitespace, possibly with a comment, is known as a **blank line**, and a Tcl interpreter totally ignores it.

#!/usr/bin/tclsh

```
# my first program in Tcl
puts "Hello, World!"

puts "Hello World!" ; # my first print in Tcl program

puts [expr 3 + 2] ; # print sum of the 3 and 2
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Hello, World!

Hello World!

5

```
set a
puts $a
can't read "a": no such variable
    while executing
"set a "
    (file "./practice.tcl" line 10)
```

Tcl Commands

Tcl command is actually a list of words, with the first word representing the command to be executed. The next words represent the arguments. In order to group the words into a single argument, we enclose multiple words with "" or {}.

The syntax of Tcl command is as follows −

```
commandName argument1 argument2 ... argumentN
```

```
#!/usr/bin/tclsh

puts "Hello, World!"
```

puts stdout "Hello, world!"

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Hello, World!

Hello, world!

In the above code, puts is the Tcl command, stdout is argument1, and "Hello World" is argument2. Here, stdout makes the program to print in the standard output device.

Command Substitution

In command substitutions, square brackets are used to evaluate the scripts inside the square brackets. A simple example to add two numbers is shown below −

Variable Substitution

In variable substitutions, $ is used before the variable name and this returns the contents of the variable. A simple example to set a value to a variable and print it is shown below.

Backslash Substitution

These are commonly called **escape sequences**; with each backslash, followed by a letter having its own meaning. A simple example for newline substitution is shown below −

#!/usr/bin/tclsh

puts "Hello, World!"

puts [expr 1 + 6 + 9]

set a 3

puts $a

puts "Hello\nWorld"

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Hello, World!

3

Hello

World

Tcl - Data Types

Simple Tcl Objects

In Tcl, whether it is an integer number, boolean, floating point number, or a string. When you want to use a variable, you can directly assign a value to it.

#!/usr/bin/tclsh

set myVariable 18

puts $myVariable

String Representations

in Tcl, you need not include double quotes when it's only a single word. An example can be −

When we want to represent multiple strings, we can use either double quotes or curly braces. It is shown below –

List

List is nothing but a group of elements. A group of words either using double quotes or curly braces can be used to represent a simple list.

Associative Array

Associative arrays have an index (key) that is not necessarily an integer. It is generally a string that acts like key value pairs.

Handles

Tcl handles are commonly used to represent files.

set myfile [open "filename" r]

#!/usr/bin/tclsh

```
set myVariable 18
puts $myVariable
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
18

set myVariable 18
puts [expr $myVariable + 6 + 9]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
33

set myVariable "hello world"
puts $myVariable
set myVariable {hello world}
puts $myVariable
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
hello world
hello world

set myVariable {red green blue}
puts [lindex $myVariable 2]
set myVariable "red green blue"
puts [lindex $myVariable 1]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
blue
green
```

```
set  marks(english) 80
puts $marks(english)
set  marks(mathematics) 90
puts $marks(mathematics)
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
80
90
```

Tcl - Variables

In Tcl, there is no concept of variable declaration.

Variable Naming

The name of variables can contain any characters and length. You can even have white spaces by enclosing the variable in curly braces.

The set command is used for assigning value to a variable. The syntax for set command is,

set variableName value

Dynamic Typing

Tcl is a dynamically typed language. The value of the variable can be dynamically converted to the required type when required.

Mathematical Expressions

expr is used for representing mathematical expression. The default precision of Tcl is 12 digits. In order to get floating point results, we should add at least a single decimal digit.

```
#!/usr/bin/tclsh

set variableA 10
set {variable B} test
puts $variableA
```

```
puts ${variable B}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
10
test

set variableA "10"
puts $variableA
set sum [expr $variableA +20];
puts $sum
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
10
30

set variableA "10"
set result [expr $variableA / 9];
puts $result
set result [expr $variableA / 9.0];
puts $result
set variableA "10.0"
set result [expr $variableA / 9];
puts $result
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
1
1.1111111111111112
1.1111111111111112

set variableA "10"
```

```
set tcl_precision 5

set result [expr $variableA / 9.0];

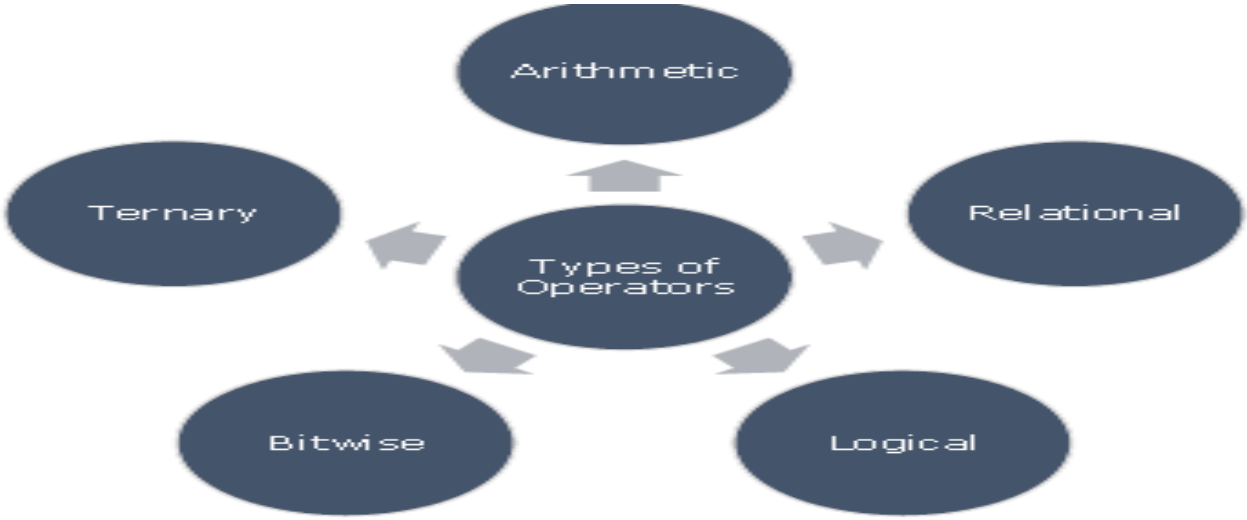
puts $result

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

1.1111
```

Tcl - Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations.



Arithmetic Operators

Following table shows all the arithmetic operators supported by Tcl language. Assume variable A holds 10 and variable B holds 20, then −

Show Examples

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0

```
#!/usr/bin/tclsh

set a 21
set b 10
set c [expr $a + $b]
puts "Line 1 - Value of c is $c\n"
set c [expr $a - $b]
puts "Line 2 - Value of c is $c\n"
set c [expr $a * $b]
puts "Line 3 - Value of c is $c\n"
set c [expr $a / $b]
puts "Line 4 - Value of c is $c\n"
set c [expr $a % $b]
puts "Line 5 - Value of c is $c\n"
```

Simulation Output

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Line 1 - Value of c is 31


Line 2 - Value of c is 11


Line 3 - Value of c is 210


Line 4 - Value of c is 2


Line 5 - Value of c is 1
```

Relational Operators

```
set a 21
set b 10

if { $a == $b } {
   puts "Line 1 - a is equal to b\n"
} else {
   puts "Line 1 - a is not equal to b\n"
}
if { $a < $b } {
   puts "Line 2 - a is less than b\n"
} else {
   puts "Line 2 - a is not less than b\n"
}
if { $a > $b } {
   puts "Line 3 - a is greater than b\n"
} else {
   puts "Line 3 - a is not greater than b\n"
}
# Lets change value of a and b
set a 5
set b 20
if { $a <= $b } {
   puts "Line 4 - a is either less than or equal to  b\n"
}
if { $b >= $a } {
   puts "Line 5 - b is either greater than  or equal to a\n"
}
```

Simulation Output

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Line 1 - a is not equal to b

Line 2 - a is not less than b

Line 3 - a is greater than b

Line 4 - a is either less than or equal to b

Line 5 - b is either greater than or equal to a

Following table shows all the relational operators supported by Tcl language. Assume variable A holds 10 and variable B holds 20, then −

Show Examples

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Logical Operators

Following table shows all the logical operators supported by Tcl language. Assume variable A holds 1 and variable B holds 0, then –

```
set a  5
set b 20

if { $a && $b } {
   puts "Line 1 - Condition is true\n"
}
if { $a || $b } {
   puts "Line 2 - Condition is true\n"
}
# lets change the value of  a and b
set a  0
set b 10
if { $a && $b } {
   puts "Line 3 - Condition is true\n"
} else {
   puts "Line 3 - Condition is not true\n"
}
if { !($a && $b) } {
   puts "Line 4 - Condition is true\n"
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Line 1 - Condition is true


Line 2 - Condition is true
```

Line 3 - Condition is not true

Line 4 - Condition is true

Show Examples

Operator Description Example

&& Called Logical AND operator. If both the operands are non-zero, then condition becomes true. (A && B) is false.

|| Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true. (A || B) is true.

! Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false. !(A && B) is true.

Bitwise Operators

Bitwise operator works on bits and perform bit-by-bit operation. The truth tables for &, |, and ^ are as follows −

p	q	p & q	p | q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume if A = 60; and B = 13; now in binary format they will be as follows −

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

The Bitwise operators supported by Tcl language are listed in the following table. Assume variable A holds 60 and variable B holds 13, then –

```
set a 60  ;# 60 = 0011 1100

set b 13  ;# 13 = 0000 1101


set c [expr $a & $b] ;# 12 = 0000 1100

puts "Line 1 - Value of c is $c\n"


set c [expr $a | $b;] ;# 61 = 0011 1101

puts "Line 2 - Value of c is $c\n"


set c [expr $a ^ $b;] ;# 49 = 0011 0001

puts "Line 3 - Value of c is $c\n"


set c [expr $a << 2] ;# 240 = 1111 0000

puts "Line 4 - Value of c is $c\n"


set c [expr $a >> 2] ;# 15 = 0000 1111

puts "Line 5 - Value of c is $c\n"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Line 1 - Value of c is 12


Line 2 - Value of c is 61


Line 3 - Value of c is 49


Line 4 - Value of c is 240
```

Line 5 - Value of c is 15

Show Examples

Operator Description Example

& Binary AND Operator copies a bit to the result if it exists in both operands. (A & B) will give 12, which is 0000 1100

| Binary OR Operator copies a bit if it exists in either operand. (A | B) will give 61, which is 0011 1101

^ Binary XOR Operator copies the bit if it is set in one operand but not both. (A ^ B) will give 49, which is 0011 0001

<< Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand. A << 2 will give 240, which is 1111 0000

>> Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand. A >> 2 will give 15, which is 0000 1111

Ternary Operator

Show Examples

Operation Description Example

? : Ternary If Condition is true? Then value X : Otherwise value Y

```
set a 10;
set b [expr $a == 1 ? 20: 30]
puts "Value of b is $b\n"
set b [expr $a == 10 ? 20: 30]
puts "Value of b is $b\n"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Value of b is 30

Value of b is 20
```

Operators Precedence in Tcl

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator.

For example : x = 7 + 3 * 2; here, x is assigned 13, not 20 because operator * has higher precedence than +, so it first gets multiplied with 3 * 2 and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Show Examples

Category	Operator	Associativity
Unary	+ -	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Ternary	?:	Right to left

```
#!/usr/bin/tclsh

set a 20
set b 10
```

```
set c 15
set d 5

set  e [expr [expr $a + $b] * $c / $d ]     ;# ( 30 * 15 ) / 5
puts "Value of (a + b) * c / d is : $e\n"

set  e [expr  [expr [expr $a + $b] * $c] / $d]   ;#  (30 * 15 ) / 5]
puts "Value of ((a + b) * c) / d is  : $e\n"

set  e  [expr [expr $a + $b] * [expr $c / $d] ]   ;# (30) * (15/5)
puts "Value of (a + b) * (c / d) is  : $e\n"

set  e  [expr $a + [expr $b * $c ] / $d ] ;#  20 + (150/5)
puts "Value of a + (b * c) / d is  :  $e\n"
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Value of (a + b) * c / d is : 90

Value of ((a + b) * c) / d is  : 90

Value of (a + b) * (c / d) is  : 90

Value of a + (b * c) / d is  :  50
```

Tcl - Decisions

A statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

Sr. No. Statement & Description

1 if statement

An 'if' statement consists of a Boolean expression followed by one or more statements.

Syntax

The syntax of an 'if' statement in Tcl language is −

```
if {boolean_expression} {
   # statement(s) will execute if the Boolean expression is true
}
```

If the Boolean expression evaluates to true, then the block of code inside the if statement will be executed. If Boolean expression evaluates to false, then the first set of code after the end of the 'if' statement (after the closing curly brace) will be executed.

Tcl language uses the expr command internally and hence it's not required for us to use expr statement explicitly.

```
#!/usr/bin/tclsh
set a 10

#check the boolean condition using if statement
if { $a < 20 } {
   # if condition is true then print the following
   puts "a is less than 20"
}
puts "value of a is : $a"
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
a is less than 20
value of a is : 10
```

2 if...else statement

An 'if' statement can be followed by an optional 'else' statement, which executes when the Boolean expression is false.

Syntax

The syntax of an 'if...else' statement in Tcl language is −

```
if {boolean_expression} {
   # statement(s) will execute if the boolean expression is true
} else {
   # statement(s) will execute if the boolean expression is false
}
```

If the Boolean expression evaluates to true, then the if block of code will be executed, otherwise else block of code will be executed.

```
#!/usr/bin/tclsh
set a 100

#check the boolean condition
if {$a < 20 } {
   #if condition is true then print the following
   puts "a is less than 20"
} else {
   #if condition is false then print the following
   puts "a is not less than 20"
}
puts "value of a is : $a"
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
a is not less than 20
value of a is : 100
```

The if...else if...else Statement

An 'if' statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement.

When using if , else if, else statements there are few points to keep in mind −

An 'if' can have zero or one else's and it must come after any else if's.

An 'if' can have zero to many else if's and they must come before the else.

Once an 'else if' succeeds, none of the remaining else if's or else's will be tested.

Syntax

The syntax of an 'if...else if...else' statement in Tcl language is −

```
if {boolean_expression 1} {
   # Executes when the boolean expression 1 is true
} elseif {boolean_expression 2} {
   # Executes when the boolean expression 2 is true
} elseif {boolean_expression 3} {
   # Executes when the boolean expression 3 is true
} else {
   # executes when the none of the above condition is true
}
```

```
#!/usr/bin/tclsh
set a 100

#check the boolean condition
if { $a == 10 } {
   # if condition is true then print the following
```

```
   puts "Value of a is 10"
} elseif { $a == 20 } {
   # if else if condition is true
   puts "Value of a is 20"
} elseif { $a == 30 } {
   # if else if condition is true
   puts "Value of a is 30"
} else {
   # if none of the conditions is true
   puts "None of the values is matching"
}

puts "Exact value of a is: $a"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
None of the values is matching
Exact value of a is: 100
```

3 nested if statements

You can use one 'if' or 'else if' statement inside another 'if' or 'else if' statement(s).

It is always legal in Tcl to nest if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax

The syntax for a nested if statement is as follows −

```
if { boolean_expression 1 } {
   # Executes when the boolean expression 1 is true
   if {boolean_expression 2} {
      # Executes when the boolean expression 2 is true
   }
```

}

You can nest else if...else in the similar way as you have nested if statement.

```
#!/usr/bin/tclsh
set a 100
set b 200

# check the boolean condition
if { $a == 100 } {
   # if condition is true then check the following
   if { $b == 200 } {
      #if condition is true then print the following
      puts "Value of a is 100 and b is 200"
   }
}
puts "Exact value of a is : $a"
puts "Exact value of b is : $b"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Value of a is 100 and b is 200
Exact value of a is : 100
Exact value of b is : 200
```

4 switch statement

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each **switch case.**

Syntax

The syntax for unquoted **switch** statement in Tcl language is as follows −

switch switchingString matchString1 {body1} matchString2 {body2} ... matchStringn {bodyn}

The syntax for unquoted **switch** statement in Tcl language is as follows −

```
switch switchingString {
   matchString1 {
      body1
   }
   matchString2 {
      body2
   }
...
   matchStringn {
      bodyn
   }
}
```

The following rules apply to a switch statement −

The switchingString is used in a switch statement; used between the different blocks by comparing to the matchString.

You can have any number of matchString blocks within a switch.

A switch statement can have an optional default block, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true.

```
#!/usr/bin/tclsh


set grade C;
```

```
switch $grade  A { puts "Well done!" }  B { puts "Excellent!" }  C { puts "You passed!" } F { puts "Better try again"   }   default {     puts "Invalid grade"   }

puts "Your grade is  $grade"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
You passed!
Your grade is  C
```

Example: Quoted version

```
#!/usr/bin/tclsh

set grade B;

switch $grade {
  A {
    puts "Well done!"
  }
  B {
    puts "Excellent!"
  }

  C {
    puts "You passed!"
  }
  F {
    puts "Better try again"
  }
  default {
```

```
    puts "Invalid grade"
  }
}
puts "Your grade is  $grade"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Excellent!
Your grade is  B
```

5 nested switch statements

You can use one switch statement inside another switch statement(s).

Syntax

The syntax for a nested switch statement is as follows −

```
switch switchingString {
  matchString1 {
    body1
    switch switchingString {
      matchString1 {
        body1
      }
      matchString2 {
        body2
      }
      ...
      matchStringn {
        bodyn
      }
    }
```

```
  }
  matchString2 {
    body2
  }
...
  matchStringn {
    bodyn
  }
}
```

```
#!/usr/bin/tclsh
set a 100
set b 200

switch $a {
  100 {
    puts "This is part of outer switch"
    switch $b {
      200 {
        puts "This is part of inner switch!"
      }
    }
  }
}
puts "Exact value of a is : $a"
puts "Exact value of a is : $b"
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
```

This is part of outer switch

This is part of inner switch!

Exact value of a is : 100

Exact value of a is : 200

The ? : Operator

We have covered conditional operator ? : in previous chapter, which can be used to replace if...else statements. It has the following general form −

Exp1 ? Exp2 : Exp3;

Where Exp1, Exp2, and Exp3 are expressions. Notice the use and placement of the colon.

#!/usr/bin/tclsh

set a 10;

set b [expr $a == 1 ? 20: 30]

puts "Value of b is $b\n"

set b [expr $a == 10 ? 20: 30]

puts "Value of b is $b\n"

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Value of b is 30

Value of b is 20

Tcl - Loops

Tcl language provides the following types of loops to handle looping requirements.

Sr. No. Loop Type & Description

1 while loop

Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.

A while loop statement in Tcl language repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a while loop in Tcl language is −

```
while {condition} {
   statement(s)
}
```

Here, statement(s) may be a single statement or a block of statements. The condition may be any expression, and true is any nonzero value. The loop iterates while the condition is true.

```
#!/usr/bin/tclsh
set a 10

#while loop execution
while { $a < 20 } {
   puts "value of a: $a"
   incr a
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
```

value of a: 16

value of a: 17

value of a: 18

value of a: 19

2 for loop

Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

A for loop is a repetition control structure that allows you to efficiently write a code that needs to be executed for a specific number of times.

Syntax

The syntax of a for loop in Tcl language is −

```
for {initialization} {condition} {increment} {
   statement(s);
}
```

Here is the flow of control in a for loop −

- The initialization step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the condition is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement just after the for loop.
- After the body of the for loop executes, the flow of control jumps back up to the increment statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.

The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the for loop terminates.

#!/usr/bin/tclsh

```
# for loop execution
for { set a 10}  {$a < 20} {incr a} {
   puts "value of a: $a"
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

3 nested loops

You can use one or more loop inside any another while, for or do..while loop.

Tcl allows to use one loop inside another loop.

Syntax

The syntax for a nested for loop statement in Tcl language is as follows −

```
for {initialization} {condition} {increment} {
   for {initialization} {condition} {increment} {
      statement(s);
   }
   statement(s);
}
```

The syntax for a nested while loop statement in Tcl language is as follows −

```
while {condition} {
   while {condition} {
      statement(s);
   }
   statement(s);
}
```

A final note on loop nesting is that you can put any type of loop inside of any other type of loop. For example, a for loop can be inside a while loop or vice versa.

```
#!/usr/bin/tclsh

set j 0;
for {set i 2} {$i<100} {incr i} {
   for {set j 2} {$j <= [expr $i/$j] } {incr j} {
      if {  [expr $i%$j] == 0 } {
         break
      }
   }
   if {$j >[expr $i/$j] } {
      puts "$i is prime"
   }
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
2 is prime
3 is prime
5 is prime
7 is prime
```

11 is prime

13 is prime

17 is prime

19 is prime

23 is prime

29 is prime

31 is prime

37 is prime

41 is prime

43 is prime

47 is prime

53 is prime

59 is prime

61 is prime

67 is prime

71 is prime

73 is prime

79 is prime

83 is prime

89 is prime

97 is prime

Loop Control Statements

Tcl supports the following control statements.

Sr.No. Control Statement & Description

1 break statement

Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.

The break statement in Tcl language is used for terminating a loop. When the break statement is encountered inside a loop, the loop is immediately terminated and program control resumes at the next statement following the loop.

If you are using nested loops (i.e., one loop inside another loop), the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for a break statement in Tcl is as follows −

break;

```
#!/usr/bin/tclsh


set a 10


# while loop execution
while {$a < 20 } {
   puts "value of a: $a"
   incr a
   if { $a > 15} {
      # terminate the loop using break statement
      break
   }
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
value of a: 10
value of a: 11
value of a: 12
value of a: 13
```

value of a: 14

value of a: 15

2 continue statement

Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

The continue statement in Tcl language works somewhat like the break statement. Instead of forcing termination, however, continue forces the next iteration of the loop to take place, skipping any code in between.

For the for loop, continue statement causes the conditional test and increment portions of the loop to execute. For the while loop, continue statement passes the program control to the conditional tests.

Syntax

The syntax for a continue statement in Tcl is as follows −

```
continue;
```

```
#!/usr/bin/tclsh

set a 10
# do loop execution
while { $a < 20 } {
   if { $a == 15} {
      #skip the iteration
      incr a
      continue
   }
   puts "value of a: $a"
   incr a
}
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 16

value of a: 17

value of a: 18

value of a: 19

The Infinite Loop

A loop becomes infinite loop if a condition never becomes false. The while loop is traditionally used for this purpose. You can make an endless loop by leaving the conditional expression as 1.

```
while {1} {
   puts "This loop will run forever."
}
```

When the conditional expression is absent, it is assumed to be true. Tcl programmers more commonly use the while {1} construct to signify an infinite loop.

NOTE − You can terminate an infinite loop by pressing Ctrl + C keys.

Tcl - Arrays

An array is a systematic arrangement of a group of elements using indices. The syntax for the conventional array is shown below.

set ArrayName(Index) value

```
#!/usr/bin/tclsh

set languages(0) Tcl
```

```
set languages(1) "C Language"
puts $languages(0)
puts $languages(1)
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Tcl
C Language
[b23n_sarif_727@prac3 tck_lab]$
```

Size of Array

The syntax for calculating size array is shown below.

[array size variablename]

```
set languages(0) Tcl
set languages(1) "C Language"
puts [array size languages]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
2
```

Array Iteration

we can use array iteration to access elements of the array.

```
set languages(0) Tcl
set languages(1) "C Language"
for { set index 0 }  { $index < [array size languages] }  { incr index } {
   puts "languages($index) : $languages($index)"
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
languages(0) : Tcl
languages(1) : C Language
```

Associative Arrays

In Tcl, all arrays by nature are associative. Arrays are stored and retrieved without any specific order. Associative arrays have an index that is not necessarily a number.

```
set personA(Name) "Dave"

set personA(Age) 14

puts  $personA(Name)

puts  $personA(Age)
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Dave

14
```

Indices of Array

The syntax for retrieving indices of array is shown below.

```
[array names variablename]

set personA(Name) "Dave"

set personA(Age) 14

puts [array names personA]
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Age Name
```

Iteration of Associative Array

You can use the indices of array to iterate through the associative array.

```
set personA(Name) "Dave"

set personA(Age) 14

foreach index [array names personA] {

   puts "personA($index): $personA($index)"

}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

personA(Age): 14

personA(Name): Dave
```

Tcl - Strings

The primitive data-type of Tcl is string and often we can find quotes on Tcl as string only language.

Boolean value can be represented as 1, yes or true for true and 0, no, or false for false.

String Representations

Unlike other languages, in Tcl, you need not include double quotes when it's only a single word.

```
set myVariable hello

puts $myVariable
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

hello
```

When we want to represent multiple strings, we can use either double quotes or curly braces.

```
set myVariable "hello world"

puts $myVariable

set myVariable {hello world}

puts $myVariable
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

hello world

hello world
```

String Escape Sequence

There are certain characters in Tcl when they are preceded by a backslash they will have special meaning and they are used to represent like newline (\n) or tab (\t).

- Here, you have a list of some of such escape sequence codes −

Escape sequence	Meaning
\\	\ character
\'	' character

\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab

```
#!/usr/bin/tclsh

puts "Hello\tWorld\n\nTutorialspoint";
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Hello World

Tutorialspoint
```

String Command

The list of subcommands for string command is listed in the following table −

Sr.No. Methods & Description

1 **compare** string1 string2

Compares string1 and string2 lexographically. Returns 0 if equal, -1 if string1 comes before string2, else 1.

2 **first** string1 string2

Returns the index first occurrence of string1 in string2. If not found, returns -1.

3 **index** string index

Returns the character at index.

4 **last** string1 string2

Returns the index last occurrence of string1 in string2. If not found, returns -1.

5 **length** string

Returns the length of string.

6 **match** pattern string

Returns 1 if the string matches the pattern.

7 **range** string index1 index2

Return the range of characters in string from index1 to index2.

8 **tolower** string

Returns the lowercase string.

9 **toupper** string

Returns the uppercase string.

10 **trim** string ?trimcharacters?

Removes trimcharacters in both ends of string. The default trimcharacters is whitespace.

11 **trimleft** string ?trimcharacters?

Removes trimcharacters in left beginning of string. The default trimcharacters is whitespace.

12 **trimright** string ?trimcharacters?

Removes trimcharacters in left end of string. The default trimcharacters is whitespace.

13 **wordend** findstring index

Return the index in findstring of the character after the word containing the character at index.

14 **wordstart** findstring index

Return the index in findstring of the first character in the word containing the character at index.

String Comparison

```
set s1 "Hello"
set s2 "World"
set s3 "World"
puts [string compare $s1 $s2]
if {[string compare $s2 $s3] == 0} {
   puts "String \'s1\' and \'s2\' are same.";
}

if {[string compare $s1 $s2] == -1} {
   puts "String \'s1\' comes before \'s2\'.";
}

if {[string compare $s2 $s1] == 1} {
   puts "String \'s2\' comes after \'s1\'.";
}
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

-1

String 's1' and 's2' are same.

String 's1' comes before 's2'.

String 's2' comes after 's1'.

Index of String

```
set s1 "Hello World"
set s2 "o"
puts "First occurrence of $s2 in s1"
puts [string first $s2 $s1]
puts "Character at index 0 in s1"
puts [string index $s1 0]
puts "Last occurrence of $s2 in s1"
puts [string last $s2 $s1]
puts "Word end index in s1"
puts [string wordend $s1 20]
puts "Word start index in s1"
puts [string wordstart $s1 20]
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
First occurrence of o in s1
4
Character at index 0 in s1
H
Last occurrence of o in s1
7
Word end index in s1
11
Word start index in s1
```

6

Length of String

```
set s1 "Hello World"
puts "Length of string s1"
puts [string length $s1]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Length of string s1
11
```

Handling Cases

```
set s1 "Hello World"
puts "Uppercase string of s1"
puts [string toupper $s1]
puts "Lowercase string of s1"
puts [string tolower $s1]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Uppercase string of s1
HELLO WORLD
Lowercase string of s1
hello world
```

Trimming Characters

```
set s1 "Hello World"
set s2 "World"
puts "Trim right $s2 in $s1"
puts [string trimright $s1 $s2]
```

```
set s2 "Hello"
puts "Trim left $s2 in $s1"
puts [string trimleft $s1 $s2]

set s1 " Hello World "
set s2 " "
puts "Trim characters s1 on both sides of s2"
puts [string trim $s1 $s2]

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Trim right World in Hello World
Hello
Trim left Hello in Hello World
 World
Trim characters s1 on both sides of s2
Hello World
```

Matching Strings

```
set s1 "test@test.com"
set s2 "*@*.com"
puts "Matching pattern s2 in s1"
puts [string match "*@*.com" $s1 ]
puts "Matching pattern tcl in s1"
puts [string match {tcl} $s1]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Matching pattern s2 in s1
```

1

Matching pattern tcl in s1

0

Append Command

```
set s1 "Hello"
append s1 " World"
puts $s1
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Hello World
```

Format command

The following table shows the list of format specifiers available in Tcl −

Specifier	Use
%s	String representation
%d	Integer representation
%f	Floating point representation
%e	Floating point representation with mantissa-exponent form
%x	Hexa decimal representation

```
puts [format "%f" 43.5]
puts [format "%e" 43.5]
puts [format "%d %s" 4 tuts]
puts [format "%s" "Tcl Language"]
puts [format "%x" 40]
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
43.500000
```

4.350000e+01

4 tuts

Tcl Language

28

Scan command

Scan command is used for parsing a string based to the format specifier.

puts [scan "90" {%[0-9]} m]

puts [scan "abc" {%[a-z]} m]

puts [scan "abc" {%[A-Z]} m]

puts [scan "ABC" {%[A-Z]} m]

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

1

1

0

1

Tcl - Lists

List is one of the basic data-type available in Tcl. It is used for representing an ordered collection of items. It can include different types of items in the same list. Further, a list can contain another list.

An important thing that needs to be noted is that these lists are represented as strings completely and processed to form individual items when required. So, avoid large lists and in such cases; use array.

Creating a List

The general syntax for list is given below −

set listName { item1 item2 item3 .. itemn }

```
# or
set listName [list item1 item2 item3]
# or
set listName [split "items separated by a character" split_character]

set colorList1 {red green blue}
set colorList2 [list red green blue]
set colorList3 [split "red_green_blue" _]
puts $colorList1
puts $colorList2
puts $colorList3
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
red green blue
red green blue
red green blue
```

Appending Item to a List

The syntax for appending item to a list is given below −

```
append listName split_character value
# or
lappend listName value

set var orange
append var " " "blue"
lappend var "red"
lappend var "green"
puts $var
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

orange blue red green

Length of List

The syntax for length of list is given below −

llength listName

```
set var {orange blue red green}
puts [llength $var]
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

4

List Item at Index

The syntax for selecting list item at specific index is given below −

lindex listname index

```
set var {orange blue red green}
puts [lindex $var  1]
```

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

blue

Insert Item at Index

The syntax for inserting list items at specific index is given below.

linsert listname index value1 value2..valuen

```
set var {orange blue red green}
set var [linsert  $var 3 black white]
puts $var
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
orange blue red black white green
```

Replace Items at Indices

The syntax for replacing list items at specific indices is given below −

```
lreplace listname firstindex lastindex value1 value2..valuen
```

```
set var {orange blue red green}
set var [lreplace $var 2 3 black white]
puts $var
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
orange blue black white
```

Set Item at Index

The syntax for setting list item at specific index is given below −

```
lset listname index value
```

```
set var {orange blue red green}
lset var 0 black
puts $var
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
black blue red green
```

Transform List to Variables

The syntax for copying values to variables is given below −

lassign listname variable1 variable2.. variablen

```
set var {orange blue red green}
lassign $var colour1 colour2
puts $colour1
puts $colour2
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
orange
blue
```

Sorting a List

The syntax for sorting a list is given below −

lsort listname

```
set var {orange blue red green}
set var [lsort $var]
puts $var
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
blue green orange red
```

Tcl - Dictionary

A dictionary is an arrangement for mapping values to keys.

dict set dictname key value

or

dict create dictname key1 value1 key2 value2 .. keyn valuen

```
dict set colours  colour1 red
puts $colours
dict set colours  colour2 green
puts $colours

set colours [dict create colour1 "black" colour2 "white"]
puts $colours
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
colour1 red
colour1 red colour2 green
colour1 black colour2 white
```

Size of Dict

The syntax for getting size of dict is shown below −

[dict size dictname]

```
set colours [dict create colour1 "black" colour2 "white"]
puts [dict size $colours]
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
2
```

Dictionary Iteration

A simple dictionary iteration for printing keys and valued of the dictionary is

```
set colours [dict create colour1 "black" colour2 "white"]
```

```
foreach item [dict keys $colours] {
   set value [dict get $colours $item]
   puts $value
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
black
white
```

Value for Key in Dict

The syntax for retrieving value for key in dict is shown below −

```
[dict get $dictname $keyname]
```

```
set colours [dict create colour1 "black" colour2 "white"]
set value [dict get $colours colour1]
puts $value
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
black
```

All Keys in Dict

The syntax for retrieving all keys in dict is shown below −

```
[dict keys $dictname]
```

```
set colours [dict create colour1 "black" colour2 "white"]
set keys [dict keys $colours]
puts $keys
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
colour1 colour2
```

All Values in Dict

The syntax for retrieving all values in dict is shown below −

```
[dict values $dictname]
```

```
set colours [dict create colour1 "black" colour2 "white"]
set values [dict values $colours]
puts $values
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
black white
```

Key Exists in Dict

The syntax for checking if a key exists in dict is shown below −

```
[dict exists $dictname $key]
```

```
set colours [dict create colour1 "black" colour2 "white"]
set result [dict exists $colours colour1]
puts $result
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
1
```

Tcl - Procedures

Procedures are nothing but code blocks with series of commands that provide a specific reusable functionality. It is used to avoid same code being repeated in multiple locations. Procedures are equivalent to the functions used in many programming languages and are made available in Tcl with the help of proc command.

The syntax of creating a simple procedure is shown below −

```
proc procedureName {arguments} {
   body
}

proc helloWorld {} {
   puts "Hello, World!"
}
helloWorld
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Hello, World!
```

Procedures with Multiple Arguments

```
proc add {a b} {
   return [expr $a+$b]
}
puts [add 10 30]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
40
```

Procedures with Variable Arguments

procedure with arguments is shown below −

```
proc avg {numbers} {
   set sum 0
   foreach number $numbers {
      set sum  [expr $sum + $number]
   }
   set average [expr $sum/[llength $numbers]]
```

```
  return $average
}
puts [avg {70 80 50 60}]
puts [avg {70 80 50 }]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
65
66
```

Procedures with Default Arguments

Default arguments are used to provide default values that can be used if no value is provided.

```
proc add {a {b 100} } {
  return [expr $a+$b]
}
puts [add 10 30]
puts [add 10]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
40
110
```

Recursive Procedures

```
proc factorial {number} {
  if {$number <= 1} {
    return 1
  }
  return [expr $number * [factorial [expr $number - 1]]]
```

}

puts [factorial 3]

puts [factorial 5]

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

6

120

Tcl - File I/O

Tcl supports file handling with the help of the built in commands open, read, puts, gets, and close.

A file represents a sequence of bytes, does not matter if it is a text file or binary file.

Opening Files

Tcl uses the open command to open files in Tcl. The syntax for opening a file is as follows −

open fileName accessMode

Here, **filename** is string literal, which you will use to name your file and **accessMode** can have one of the following values −

Sr. No.	Mode & Description
1	r Opens an existing text file for reading purpose and the file must exist. This is the default mode used when no accessMode is specified.
2	w Opens a text file for writing, if it does not exist, then a new file is created else existing file is truncated.
3	a

Opens a text file for writing in appending mode and file must exist. Here, your program will start appending content in the existing file content.

4 r+

Opens a text file for reading and writing both. File must exist already.

5 w+

Opens a text file for reading and writing both. It first truncate the file to zero length if it exists otherwise create the file if it does not exist.

6 a+

Opens a text file for reading and writing both. It creates the file if it does not exist. The reading will start from the beginning, but writing can only be appended.

Closing a File

To close a file, use the close command. The syntax for close is as follows −

close fileName

Any file that has been opened by a program must be closed when the program finishes using that file. In most cases, the files need not be closed explicitly; they are closed automatically when File objects are terminated automatically.

Writing a File

Puts command is used to write to an open file.

```
puts $filename "text to write"
```

```
set fp [open "input.txt" w+]
puts $fp "test"
```

```
close $fp
```

When the above code is compiled and executed, it creates **a new file input.txt** in the directory that it has been started under (in the program's working directory).

Reading a File

Following is the simple command to read from a file −

set file_data [read $fp]

```
set fp [open "input.txt" w+]
puts $fp "test"
close $fp
set fp [open "input.txt" r]
set file_data [read $fp]
puts $file_data
close $fp
```

When the above code is compiled and executed, it reads the file created in previous section and produces the following result −

```
test
```

for reading file till end of file line by line −

```
set fp [open "input.txt" w+]
puts $fp "test\ntest"
close $fp

set fp [open "input.txt" r]
while { [gets $fp data] >= 0 } {
   puts $data
```

```
}
close $fp
```

When the above code is compiled and executed, it reads the file created in previous section and produces the following result −

```
test
test
```

Tcl - Regular Expressions

The "regexp" command is used to match a regular expression in Tcl. A regular expression is a sequence of characters that contains a search pattern. It consists of multiple rules and the following table explains these rules and corresponding use.

Sr.No. Rule & Description

1 x

Exact match.

2 [a-z]

Any lowercase letter from a-z.

3 .

Any character.

4 ^

Beginning string should match.

5 $

Ending string should match.

6 \^

Backlash sequence to match special character ^.Similarly you can use for other characters.

7 ()

Add the above sequences inside parenthesis to make a regular expression.

8 x*

Should match 0 or more occurrences of the preceding x.

9 x+

Should match 1 or more occurrences of the preceding x.

10 [a-z]?

Should match 0 or 1 occurrence of the preceding x.

11 {digit}

Matches exactly digit occurrences of previous regex expression. Digit that contains 0-9.

12 {digit,}

Matches 3 or more digit occurrences of previous regex expression. Digit that contains 0-9.

13 {digit1,digit2}

Occurrences matches the range between digit1 and digit2 occurrences of previous regex expression.

Syntax

The syntax for regex is given below −

regexp optionalSwitches patterns searchString fullMatch subMatch1 ... subMatchn

Here, regex is the command. We will see about optional switches later. Patterns are the rules as mentioned earlier. Search string is the actual string on which the regex is performed. Full match is any variable to hold the result of matched regex result. Submatch1 to SubMatchn are optional subMatch variable that holds the result of sub match patterns.

```
regexp {([A-Za-z]*)} "Tcl Tutorial" a b

puts "Full Match: $a"

puts "Sub Match1: $b"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Full Match: Tcl

Sub Match1: Tcl
```

Multiple Patterns

The following example shows how to search for multiple patterns. This is example pattern for any alphabets followed by any character followed by any alphabets.

```
regexp {([A-Za-z]*).([A-Za-z]*)} "Tcl Tutorial" a b c

puts "Full Match: $a"

puts "Sub Match1: $b"

puts "Sub Match2: $c"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Full Match: Tcl Tutorial

Sub Match1: Tcl

Sub Match2: Tutorial
```

When the above code is executed, it produces the following result −

```
Full Match: Tcl Tutorial

Sub Match1: Tcl
```

Sub Match2: Tutorial

A modified version of the above code to show that a sub pattern can contain multiple patterns is shown below –

```
regexp {([A-Za-z]*.([A-Za-z]*))} "Tcl Tutorial" a b c
puts "Full Match: $a"
puts "Sub Match1: $b"
puts "Sub Match2: $c"
```

When the above code is executed, it produces the following result −

```
Full Match: Tcl Tutorial
Sub Match1: Tcl Tutorial
Sub Match2: Tutorial
```

Switches for Regex Command

The list of switches available in Tcl are,

nocase − Used to ignore case.

indices − Store location of matched sub patterns instead of matched characters.

line − New line sensitive matching. Ignores the characters after newline.

start index − Sets the offset of start of search pattern.

Marks the end of switches

```
regexp -nocase {([A-Z]*.([A-Z]*))} "Tcl Tutorial" a b c
puts "Full Match: $a"
puts "Sub Match1: $b"
puts "Sub Match2: $c"
```

When the above code is executed, it produces the following result −

Full Match: Tcl Tutorial

Sub Match1: Tcl Tutorial

Sub Match2: Tutorial

using switches

```
regexp -nocase -line -- {([A-Z]*.([A-Z]*))} "Tcl \nTutorial" a b
puts "Full Match: $a"
puts "Sub Match1: $b"
regexp -nocase -start 4 -line -- {([A-Z]*.([A-Z]*))} "Tcl \nTutorial" a b
puts "Full Match: $a"
puts "Sub Match1: $b"
```

When the above code is executed, it produces the following result −

```
Full Match: Tcl
Sub Match1: Tcl
Full Match: Tutorial
Sub Match1: Tutorial
```

Google + YouTube + ChatGPT

TCL for VLSI — Complete Learning Roadmap

Chapter 1: Introduction to TCL & VLSI Usage

- What is TCL?
- Why TCL is important in VLSI
- Where TCL is used (DC, PT, ICC2, Innovus, VCS, Verdi)
- TCL vs Shell vs Perl

Chapter 2: TCL Basics

- TCL syntax rules
- Comments
- Variables
- `set`, `puts`
- Command substitution

Chapter 3: Data Types in TCL

- Strings
- Numbers
- Lists
- Arrays
- Difference between list & array (very important for VLSI)

Chapter 4: Operators

- Arithmetic operators
- Relational operators
- Logical operators
- String comparison

Chapter 5: Control Statements

- `if`, `elseif`, `else`
- `switch`
- Common mistakes in conditions

Chapter 6: Loops

- `for`
- `while`
- `foreach`
- Looping through cells, nets, ports (VLSI use)

Chapter 7: Procedures (Functions)

- `proc` keyword
- Arguments
- Return values
- Writing reusable scripts

Chapter 8: Lists & Arrays (Deep Dive)

- Creating lists
- List commands (`lindex`, `llength`, `lappend`)
- Arrays (`array set`, `array get`)
- Tool database handling

Chapter 9: File Handling

- Reading files
- Writing reports
- Parsing timing/power reports
- Automation scripts

Chapter 10: Regular Expressions

- `regexp`
- `regsub`
- Parsing tool logs & reports

Chapter 11: VLSI Tool Commands with TCL

- `get_cells`, `get_nets`, `get_ports`
- `get_clocks`
- Filters (`-filter`)
- Collection handling

Chapter 12: Practical VLSI Scripts

- Count number of cells

- Find max delay path
- Report unconstrained paths
- Clock/report automation
- QoR checks

Chapter 13: Debugging & Best Practices

- Common TCL errors
- Debug techniques
- Writing tool-independent scripts
- Interview-oriented tips

🚀 **Let's Start: Chapter 1 – Introduction to TCL**

🔹 What is TCL?

TCL (Tool Command Language) is a scripting language used to control and automate EDA tools.

👉 In VLSI:

You don't click buttons

You write TCL commands

Tools internally understand TCL only

🔹 Why TCL is Important in VLSI?

Because:

- All major tools are TCL-based
- Automation saves days of manual work
- Required for Design, Verification, STA, Physical Design

Example tools:

Tool	TCL Usage
Design Compiler	Synthesis scripts
PrimeTime	Timing analysis
ICC2 / Innovus	PnR automation
VCS / Verdi	Simulation & debug
Vivado	FPGA flows

🔹 TCL vs Shell vs Perl (Quick Idea)

Feature	TCL	Shell	Perl
EDA Tool Support	⭐⭐⭐⭐⭐	⭐	⭐⭐
Learning Curve	Easy	Medium	Hard
VLSI Usage	Mandatory	Limited	Optional

👉 TCL is compulsory, Perl is a bonus.

🧠 First TCL Program (Very Simple)



puts "Hello VLSI World"

[b23n_sarif_727@prac3 tck_lab]$ vi p.tcl

[b23n_sarif_727@prac3 tck_lab]$ chmod 777 p.tcl

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Hello VLSI WORLD

Explanation:

- `puts` → print command

- `"Hello VLSI World"` → string

Output:

Hello VLSI World

🧪 Where to Practice TCL?

You can practice in:

- Linux terminal: `tclsh`
- Synopsys tools shell
- Online TCL interpreters

☑ What You Learned in Chapter 1

✓ What TCL is

✓ Why TCL is mandatory for VLSI

✓ Where it is used

✓ First TCL command

Chapter 2: TCL Basics

- Variables
- Set
- Rules of TCL syntax
- Common beginner mistakes (very important for interviews)

📘 Chapter 2: TCL Basics (Very Important)

This chapter is the foundation of TCL.

Most TCL errors in VLSI come from not understanding these basics properly.

1️⃣ Basic Rule of TCL (MOST IMPORTANT)

👉 Everything in TCL is a command

General syntax:

command argument1 argument2 argument3

Example:

puts "Hello"

* `puts` → command

* `"Hello"` → argument

⚠️ No commas, no semicolons needed

2️⃣ Comments in TCL

Single-line comment

This is a comment

puts "TCL for VLSI"

> Anything after `#` is ignored

```
#!/usr/bin/tclsh

# This is a comment
puts "Hello VLSI WORLD"
~
```

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Hello VLSI WORLD

3️⃣ Variables in TCL

👉 No data type declaration needed

Creating a variable

set a 10

- set → command
- a → variable name
- 10 → value

Printing variable value

puts $a

Output:

10

⚠️ `$` is used to read the variable

#!/usr/bin/tclsh

set a 10

puts "print the value $a”

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

print the value 10

4️⃣ Common Beginner Mistake ❌

puts a

Output:

a

✓ Correct:

puts $a

#!/usr/bin/tclsh

set a 10

puts a

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

a

5️⃣ Variable with String Value

set name "Sarif Ahmad"

puts $name

Output:

Sarif Ahmad

#!/usr/bin/tclsh

set name "SARIF AHMAD"

puts $name

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

SARIF AHMAD

📌 Quotes are optional if no space:

set tool VCS

6️⃣ Command Substitution (`[ ]`)

👉 Used when one command's output becomes another command's input

Example:

set x [expr 5 + 3]

puts $x

Output:

8

Explanation:

- expr 5 + 3 is executed first
- Result (`8`) is stored in `x`

#!/usr/bin/tclsh

set x [expr 5 + 3]

puts $x

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

8

7️⃣ Arithmetic Operations (`expr`)

TCL cannot do math directly ❌ -→ treated as a single value

Wrong:

set a 5+3

Correct:

set a [expr 5 + 3]

More examples:

set b [expr 10 - 2]

set c [expr 4 * 5]

set d [expr 20 / 4]

#!/usr/bin/tclsh

set a 5+3

set x [expr 5 * 3]

puts $a

puts $x

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

5+3

15

8 Curly Braces `{ }` vs Double Quotes `" "`

Using double quotes `" "`

- Variables are substituted

set a 10

puts "Value is $a"

Output:

Value is 10

Using curly braces `{ }`

- Variables are NOT substituted

puts {Value is $a}

Output:

Value is $a

📌 Very important in VLSI scripting (performance + correctness)

#!/usr/bin/tclsh

set a 10

puts "value of $a"

puts {value is $a}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

value of 10

value is $a

9️⃣ Semicolon `;` (Optional)

Used to write multiple commands in one line:

set a 10; set b 20; puts $b

#!/usr/bin/tclsh

set a 10; set b 20; puts "value of $a & $b"

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

value of 10 & 20

🔍 **VLSI-Oriented Example**

Imagine you want to store tool name and report it:

set tool_name "Design Compiler"

puts "Running on $tool_name"

Output:

Running on Design Compiler

#!/usr/bin/tclsh

set tool_name "Design Compiler"

puts "Running on $tool_name"

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Running on Design Compiler

❗ Common Errors in Chapter 2

Mistake	Reason
Forgetting `$`	Variable not expanded
Using `=`	TCL uses `set`, not `=`
Math without `expr`	Invalid syntax
Wrong braces usage	Unexpected results

🧠 Chapter 2 Summary

✓ TCL is command-based

✓ Variables use `set`

✓ `$` is mandatory to read variable

✓ Math uses `expr`

✓ `{}` vs `""` difference

✓ `[ ]` for command substitution

☑ Quick Practice (Try Yourself)

set a 15

set b 5

set c [expr $a * $b]

puts "Result = $c"

#!/usr/bin/tclsh

set a 15

set b 5

set c [expr $a * $b]

puts "Result = $c"

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Result = 75

```
#!/usr/bin/tclsh

set a 15
set b 5
set c [expr $a * $b]
puts "Result = $c"
```

👉 Next Chapter

If you say “Next Chapter”, we’ll move to:

📘 Chapter 3: **Data Types in TCL**

* String

* Number

* List

* Array (VERY important for EDA tools)

I’ll keep it **simple, interview-ready, and VLSI-focused**.

📘 Chapter 3: Data Types in TCL (VLSI Focused)

👉 **TCL has only ONE data type internally: STRING**
But it **behaves** like multiple data types using commands.

In VLSI scripting, you’ll mainly work with:

1. Strings

2. Numbers
3. Lists
4. Arrays

1.0 String in TCL

A **string** is a sequence of characters.

Example

set name "Sarif"
puts $name

Output:

Sarif

✓ Strings can contain:

- Letters
- Numbers
- Special characters

set msg "Timing analysis completed!"
puts $msg

2.0 Numbers in TCL

Numbers are also stored as **strings**, but used in math with expr.

set a 10
set b 20
set sum [expr $a + $b]
puts $sum

Output:

30

📌 Without expr, TCL treats numbers as text.

3.0 Lists in TCL (🔥 VERY IMPORTANT FOR VLSI)

A **list** is a collection of values.

Creating a list

set mylist {1 2 3 4 5}

or

set tools [list DC PT ICC2]

Printing a list

puts $mylist

Output:

1 2 3 4 5

#!/usr/bin/tclsh

set mylist {1 2 3 4 5}
puts $mylist

set tools [list DC PT ICC2]
puts $tools
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
1 2 3 4 5
DC PT ICC2

Access list elements

set colors {red green blue}
puts [lindex $colors 0]
puts [lindex $colors 1]

Output:

red

green

📌 Index starts from **0**

List length

puts [llength $colors]

Output:

3

Add element to list

lappend colors yellow
puts $colors

Output:

red green blue yellow

#!/usr/bin/tclsh

set colors {red green blue}
puts [lindex $colors 0]
puts [lindex $colors 1]

puts [llength $colors]
lappend colors yellow
puts $colors

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
red
green
3
red green blue yellow

🔧 **VLSI Example (List)**

set clocks {clk core_clk io_clk}

puts "Total clocks = [llength $clocks]"

Used in:

- Clock analysis
- Constraints
- Reports

#!/usr/bin/tclsh

set clocks {clk core_clk io_clk}

puts "Total clocks = [llength $clocks]"

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Total clocks = 3

---

4. 0 Arrays in TCL (🔥 MOST IMPORTANT FOR EDA)

👉 **Array = Key–Value pair**

Think like a dictionary.

Creating an array

```
set cell_area(NAND) 1.2
set cell_area(NOR)  1.5
set cell_area(INV)  0.5
```

Access array value

puts $cell_area(NAND)

Output:

1.2

Get all keys

puts [array names cell_area]

Output:

NAND NOR INV

Check if key exists

info exists cell_area(NAND)

Returns:

1

#!/usr/bin/tclsh

set cell_area(NAND) 1.2
set cell_area(NOR) 1.5
set cell_area(INV) 0.5

puts $cell_area(NAND)
puts [array names cell_area]
info exists cell_area(NAND)
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
1.2
INV NAND NOR

🔧 **VLSI Example (Array)**

set timing(slack) 0.12
set timing(wns) -0.05
set timing(tns) -1.2

puts "Worst Slack = $timing(wns)"

Used heavily in:

- PrimeTime
- QoR scripts
- Report parsing

```
#!/usr/bin/tclsh


set timing(slack) 0.12

set timing(wns)  -0.05

set timing(tns)  -1.2


puts "Worst Slack = $timing(wns)"

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Worst Slack = -0.05
```

🔍 Difference: List vs Array (INTERVIEW ⭐⭐⭐)

Feature	List	Array
Index	Numeric	Named key
Order	Maintained	Not guaranteed
Use case	Collections	Properties
VLSI Usage	get_cells result	cell attributes

5️⃣ Mixed Example (Real VLSI Style)

```
set cells {U1 U2 U3}

set power(U1) 0.2
set power(U2) 0.3
set power(U3) 0.1

puts "Power of U2 = $power(U2)"
#!/usr/bin/tclsh

set cells {U1 U2 U3}

set power(U1) 0.2
```

```
set power(U2) 0.3
set power(U3) 0.1

puts "Power of U2 = $power(U2)"
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Power of U2 = 0.3
```

❗ Common Mistakes

❌ Treating array like list
❌ Forgetting $ in array access
❌ Using spaces in array keys
❌ Confusing {} with []

🧠 Chapter 3 Summary

✓ Everything is string internally
✓ Numbers need expr
✓ Lists → ordered data
✓ Arrays → key-value mapping
✓ Arrays dominate VLSI scripting

📝 Quick Practice (Try Yourself)

```
set blocks {ALU FIFO SRAM}
set area(ALU) 120
set area(FIFO) 80
set area(SRAM) 200

puts "SRAM Area = $area(SRAM)"
#!/usr/bin/tclsh

set blocks {ALU FIFO SRAM}
set area(ALU) 120
set area(FIFO) 80
set area(SRAM) 200
```

puts "SRAM Area = $area(SRAM)"
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
SRAM Area = 200

📘 Chapter 3: Data Types in TCL

(Very important for VLSI / EDA scripting)

👉 TCL does not have fixed data types like C (`int`, `float`, `char`),
but logically TCL data is handled as:

1. String
2. Number
3. List
4. Array

1.0 String in TCL

Example
set name "Sarif"
puts $name

Output:
Sarif

With spaces
set tool "Design Compiler"
puts $tool

Output:
Design Compiler
📌 Strings are used in VLSI for:

- Cell names
- Net names
- File names
- Tool names
- Report names

VLSI Example

```
set report_file "timing_report.txt"
puts "Report file is $report_file"

#!/usr/bin/tclsh

set name "Sarif"
puts $name

set tool "Design Compiler"
puts $tool

set report_file "timing_report.txt"
puts "Report file is $report_file"
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Sarif
Design Compiler
Report file is timing_report.txt
```

2.0 Number in TCL

👉 Numbers are stored as strings, but treated as numbers using `expr`.

```
### Example
set a 10
set b 20
set c [expr $a + $b]
puts $c

Output:
30

### VLSI Example
set clock_period 10
set slack [expr $clock_period - 8]
puts "Slack = $slack ns"
#!/usr/bin/tclsh

set a 10
set b 20
set c [expr $a + $b]
puts $c
```

set clock_period 10
set slack [expr $clock_period - 8]
puts "Slack = $slack ns"
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
30
Slack = 2 ns

3.0 List in TCL ⭐ ⭐ ⭐ (Very Important)

👉 A list = multiple values in one variable

Creating a list
set cells {U1 U2 U3 U4}

or
set cells "U1 U2 U3 U4"

🔹 Accessing list elements
puts [lindex $cells 0]

Output:
U1

puts [lindex $cells 2]

Output:
U3

🔹 Length of list
puts [llength $cells]

Output:
4

🔹 Append element to list
lappend cells U5
puts $cells

Output:
U1 U2 U3 U4 U5

🔹 Looping through list
```
foreach cell $cells {
    puts $cell
}
```

Output:
```
U1
U2
U3
U4
U5
```

```
#!/usr/bin/tclsh

set cells {U1 U2 U3 U4}
puts [lindex $cells 0]

set cells "U1 U2 U3 U4"
puts [lindex $cells 2]

puts [llength $cells]

lappend cells U5
puts $cells

foreach cell $cells {
    puts $cell
}
```
```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
U1
U3
4
U1 U2 U3 U4 U5
U1
U2
U3
U4
U5
```

🧠 VLSI List Example
```
set clocks {clk1 clk2 clk3}

foreach clk $clocks {
```

```
    puts "Analyzing clock: $clk"
}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Analyzing clock: clk1
Analyzing clock: clk2
Analyzing clock: clk3
```

4. 0 Array in TCL ⭐⭐⭐⭐ (MOST IMPORTANT FOR EDA)

👉 Array = key–value pair
(Like dictionary / hash map)

```
### Creating array
set delay(clk1) 5
set delay(clk2) 7
set delay(clk3) 10

##  🔹 Access array value
puts $delay(clk1)
Output:
5

##  🔹 Print all keys
puts [array names delay]

Output:
clk1 clk2 clk3

##  🔹 Loop through array
foreach key [array names delay] {
    puts "$key delay = $delay($key)"
}
Output:
clk1 delay = 5
clk2 delay = 7
clk3 delay = 10

#!/usr/bin/tclsh
```

```
set delay(clk1) 5
set delay(clk2) 7
set delay(clk3) 10

puts $delay(clk1)

puts [array names delay]

foreach key [array names delay] {
    puts "$key delay = $delay($key)"
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
5
clk2 clk3 clk1
clk2 delay = 7
clk3 delay = 10
clk1 delay = 5
```

⚔️ List vs Array (Interview Question ⭐)

Feature	List	Array
Access	index based	key based
Example	`cells(0)` ❌	`delay(clk1)` ✓
Order	Ordered	Unordered
Usage	collections	database info
VLSI use	cell lists	attributes

🧠 Real VLSI Analogy

List
```
set all_cells {U1 U2 U3 U4}
```

👉 Only names

Array
```
set cell_type(U1) NAND
set cell_type(U2) NOR
set cell_type(U3) INV
```

👉 Property mapping

❗ Common Mistakes

❌

puts $cells(0)   ;# WRONG
✓
puts [lindex $cells 0]

❌
set delay(clk1 clk2) 5

✓

set delay(clk1) 5

☑ Chapter 3 Summary

✓ String → text data
✓ Number → math using `expr`
✓ List → multiple values
✓ Array → key–value mapping
✓ Lists used for collections
✓ Arrays used for attributes / properties

🧪 Practice Exercise

set nets {n1 n2 n3}
lappend nets n4

set power(n1) 10
set power(n2) 12
set power(n3) 15
set power(n4) 8

foreach net $nets {
   puts "Net $net power = $power($net)"
}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Net n1 power = 10

Net n2 power = 12
Net n3 power = 15
Net n4 power = 8

👉 Next Chapter

📘 Chapter 4: Operators in TCL

- Arithmetic
- Relational
- Logical
- String operators

📘 Chapter 4: Operators in TCL
(Very important for conditions, checks, and automation in VLSI)
In TCL, operators are mostly used inside `expr` and control statements.

1.0 Arithmetic Operators

Used for math calculations (timing, slack, count, iterations).

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulus

Example

```
set a 20
set b 6
puts [expr $a + $b]
puts [expr $a - $b]
puts [expr $a * $b]
puts [expr $a / $b]
puts [expr $a % $b]
```

```
#!/usr/bin/tclsh

set a 20
set b 6
```

```
puts [expr $a + $b]
puts [expr $a - $b]
puts [expr $a * $b]
puts [expr $a / $b]
puts [expr $a % $b]
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
26
14
120
3
2
```

🧠 VLSI Example

```
set clock_period 10
set data_delay 7
set slack [expr $clock_period - $data_delay]

puts "Slack = $slack ns"

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Slack = 3 ns
[b23n_sarif_727@prac3 tck_lab]$
```

2. 0 Relational (Comparison) Operators

Used to compare values → result is `1` (true) or `0` (false).

Operator	Meaning
`==`	Equal
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater or equal
`<=`	Less or equal

Example

```
set a 10
set b 20
puts [expr $a < $b]

Output:
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
1
```

🧠 **VLSI Example**
set slack -2

if {[expr $slack < 0]} {
    puts "Timing VIOLATION"
}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Timing VIOLATION

3. 0 Logical Operators
Used to combine multiple conditions.

Operator	Meaning
`&&`	AND
`|| `	OR
`!`	NOT

Example
set a 10
set b 20

if {[expr ($a < 15) && ($b > 10)]} {
    puts "Condition satisfied"
}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Condition satisfied

🧠 VLSI Example
set slack -1
set hold_slack 0.5

if {[expr ($slack < 0) || ($hold_slack < 0)]} {
    puts "Timing violation detected"
}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Timing violation detected

4.0 String Comparison Operators

⚠ Do NOT use `==` for strings outside `expr`
Use:

- `string equal`
- `string compare`
- `eq`, `ne` (inside `expr`)

Correct String Comparison

```
set tool "PrimeTime"

if {$tool eq "PrimeTime"} {
   puts "Running STA"
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Running STA
```

Wrong ❌

```
if {$tool == "PrimeTime"} {
   puts "Running STA"
}
```

5.0 `eq` vs `==` (Interview Favorite ⭐)

Operator	Use
`==`	Numeric comparison
`eq`	String comparison

Example

```
set a "10"
set b 10

if {[expr $a == $b]} {
   puts "Numeric compare"
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Numeric compare
```

```
if {$a eq $b} {
   puts "String compare"
```

}

6.0 Ternary Operator (`?:`)

Short-hand `if-else`.

Syntax
expr {condition ? value1 : value2}

Example
set slack -2
set status [expr {$slack < 0 ? "FAIL" : "PASS"}]

puts $status

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
FAIL

7. 0 Common Operator Mistakes ❌

Mistake	Why
Math without `expr`	Invalid TCL
Using `==` for strings	Wrong result
Forgetting braces `{}`	Unexpected expansion
Mixing string & number	Logical bugs

🧠 **Practical VLSI Script Example**
set path_delay 12
set clock_period 10

if {[expr $path_delay > $clock_period]} {
    puts "Setup violation"
} else {
    puts "Timing OK"
}

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Setup violation

☑ Chapter 4 Summary

✓ Arithmetic operators → math
✓ Relational operators → comparison
✓ Logical operators → combined conditions
✓ `eq` for strings
✓ `expr` is mandatory
✓ Used heavily in STA & automation

📘 Chapter 5: Control Statements in TCL

- `if`, `elseif`, `else`
- `switch`
- Real VLSI decision scripts
- Interview traps & best practices

📘 Chapter 5: Control Statements in TCL

(Decision making – extremely important in VLSI automation & STA scripts)

Control statements decide what action to take based on conditions.

1. 0 `if` Statement (Basic)

Syntax

```
if {condition} {
   commands
}
```

Example

```
set slack -1
if {$slack < 0} {
   puts "Timing violation"
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Timing violation
```

2. 0 `if – else`

Syntax

```
if {condition} {
   commands
} else {
   commands
}
```

Example

```
set slack 2
if {$slack < 0} {
   puts "FAIL"
} else {
   puts "PASS"
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
PASS
```

🧠 VLSI Example

```
set setup_slack 0.3
if {$setup_slack < 0} {
   puts "Setup violation"
} else {
   puts "Setup clean"
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Setup clean
```

3.0 `if – elseif – else`

Used when multiple conditions exist.

Syntax

```
if {cond1} {
   commands
} elseif {cond2} {
   commands
} else {
   commands
}
```

Example

```
set slack -0.5
if {$slack > 1} {
   puts "Excellent timing"
} elseif {$slack >= 0} {
   puts "Timing OK"
} else {
   puts "Timing violation"
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Timing violation
```

4. 0 IMPORTANT RULE: Always use `{}`

❌ Wrong

```
if ($slack < 0) {
   puts "Violation"
}
```

✓ Correct

```
if {$slack < 0} {
   puts "Violation"
}
```

- TCL does not use parentheses `( )` for conditions.

5.0 `switch` Statement ⭐

Used when checking **one variable against multiple values**

Very common for **tool-based decisions**.

Syntax

```
switch value {
   pattern1 { commands }
   pattern2 { commands }
   default  { commands }
}
```

Example

```
set mode "setup"
switch $mode {
   setup { puts "Running setup analysis" }
   hold  { puts "Running hold analysis" }
   default { puts "Unknown mode" }
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Running setup analysis
```

🧠 VLSI Tool Example

```
set tool "PT"
```

```
switch $tool {
   DC  { puts "Synthesis flow" }
   PT  { puts "STA flow" }
   ICC { puts "PnR flow" }
   default { puts "Unsupported tool" }
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
STA flow
```

6. 0 `switch` with Multiple Matches

```
set cell_type "INV"
switch $cell_type {
   INV -
   BUF { puts "Low delay cell" }
   NAND { puts "Logic cell" }
   default { puts "Other cell" }
}
```

OUTPUT: Low delay cell

7.0 Common Control-Statement Mistakes ❌

Mistake	Issue
Using `()`	Invalid TCL
Missing `{}`	Script breaks
Using `==` for strings	Wrong result
No `default` in switch	Unhandled case

8.0 Real VLSI Decision Script

```
set slack -0.2
set report "timing.rpt"

if {$slack < 0} {
   puts "Violation found"
   puts "Check report: $report"
} else {
   puts "Timing clean"
}
```

OUTPUT: Violation found

Check report: timing.rpt

🧠 Interview Tip ⭐

Q: Difference between `if` and `switch`?

A:

- `if` → range-based or complex conditions
- `switch` → exact value matching (faster & cleaner)

☑ Chapter 5 Summary

✓ `if`, `elseif`, `else` for decisions

✓ `{}` mandatory in conditions

✓ `switch` for multiple value matching

✓ Used heavily in STA, PnR, automation

✓ Avoid parentheses `( )`

🧪 Practice Exercise

```
set slack 0.1
```

```
if {$slack < 0} {
   puts "FAIL"
} elseif {$slack < 0.5} {
   puts "MARGINAL"
} else {
   puts "PASS"
}
```

Output: MARGINAL

📘 Chapter 6: Loops in TCL

- `for`
- `while`
- `foreach`
- Looping over cells, nets, clocks (real VLSI usage)

📘 Chapter 6: Loops in TCL

(One of the MOST used topics in VLSI automation)

Loops are used to repeat commands over:

- Cells
- Nets
- Ports
- Clocks
- report lines

Without loops, VLSI TCL scripting is impossible.

1.0 `for` Loop

Syntax

```
for {initialization} {condition} {increment} {
   commands
```

}

Simple Example

```
for {set i 0} {$i < 5} {incr i} {
   puts $i
}
```

Output:

0

1

2

3

4

🧠 **VLSI Example (Index-based)**

```
for {set i 1} {$i <= 4} {incr i} {
   puts "Checking path $i"
}
```

Output:

Checking path 1

Checking path 2

Checking path 3

Checking path 4

2. 0 `incr` Command

Used to increment variables.

```
set a 5
incr a
puts $a
```

Output:

6

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

6

With step:

incr a 2

3.0 `while` Loop

Syntax

```
while {condition} {
   commands
}
```

Example

```
set i 0
while {$i < 3} {
   puts $i
   incr i
}
```

Output:

0

1

2

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

0

1

2

🧠 VLSI Example

```
set slack -3
while {$slack < 0} {
   puts "Fixing violation, slack = $slack"
   incr slack
}
```

Output:

```
Fixing violation, slack = -3
Fixing violation, slack = -2
Fixing violation, slack = -1
```

4. 0 `foreach` Loop ⭐ ⭐ ⭐ (MOST IMPORTANT)

👉 Used to loop over lists and collections

👉 Most common loop in EDA tools

Syntax

```
foreach var list {
   commands
}
```

Simple Example

```
set cells {U1 U2 U3}
foreach cell $cells {
   puts $cell
}
```

Output:

```
U1
U2
```

U3

🧠 VLSI Example (Cells)

```
set cells {INV1 NAND2 NOR3}
foreach cell $cells {
   puts "Analyzing cell: $cell"
}
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Analyzing cell: INV1
Analyzing cell: NAND2
Analyzing cell: NOR3
```

🧠 **Real Tool Example**

```
set clocks {clk1 clk2 clk3}

foreach clk $clocks {
   puts "Reporting timing for $clk"
}
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Reporting timing for clk1
Reporting timing for clk2
Reporting timing for clk3
```

5.0 Nested Loops

```
set clocks {clk1 clk2}
```

```
set modes {setup hold}


foreach clk $clocks {
   foreach mode $modes {
      puts "Clock: $clk Mode: $mode"
   }
}
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Clock: clk1 Mode: setup
Clock: clk1 Mode: hold
Clock: clk2 Mode: setup
Clock: clk2 Mode: hold
```

6.0 Loop Control Commands

`break` → exit loop

```
set cells {INV BUF AND2 NAND2 OR2}
foreach cell $cells {
   if {$cell eq "NAND2"} {
      break
   }
   puts $cell
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
INV
BUF
AND2
```

`continue` → skip iteration

```
set cells {INV BUF AND2 NAND2 OR2}

foreach cell $cells {
   if {$cell eq "INV1"} {
      continue
   }
   puts $cell
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
INV
BUF
AND2
OR2
```

7. 0 Common Loop Mistakes ❌

Mistake	Problem
Using `for` for lists	Inefficient
Forgetting `incr`	Infinite loop
Using index instead of `foreach`	Bad practice
Modifying list inside loop	Unexpected behavior

🧠 Interview Tip ⭐

Q: Which loop is most used in VLSI TCL?

A: `foreach`, because tool commands return lists/collections.

8. 0 Practical VLSI Script Example

```
set slacks {1.2 -0.5 0.3 -1.1}
foreach s $slacks {
   if {$s < 0} {
      puts "Violation: slack = $s"
   }
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Violation: slack = -0.5
Violation: slack = -1.1
```

☑ Chapter 6 Summary

✓ `for` → counter-based

✓ `while` → condition-based

✓ `foreach` → list/collection-based ★

✓ `break` & `continue` control loops

✓ `foreach` is dominant in EDA tools

🧪 Practice Exercise

```
set nets {n1 n2 n3 n4}
foreach net $nets {
   puts "Checking net: $net"
}
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Checking net: n1
```

Checking net: n2

Checking net: n3

Checking net: n4

📘 Chapter 7: Procedures (Functions) in TCL

- `proc`
- Arguments
- Return values
- Writing reusable VLSI scripts
- Interview-focused examples

📘 Chapter 7: Procedures (Functions) in TCL

(Very important for reusable, clean, professional VLSI scripts)

In VLSI flows, no real project uses flat TCL.

Everything is written using procedures (`proc`).

1. 0 What is a Procedure?

👉 A procedure is a reusable block of TCL code (like a function in C).

Why we use it:

- Avoid code repetition
- Improve readability
- Easy debugging
- Mandatory for large EDA scripts

2. 0 Basic Syntax of `proc`

```
proc proc_name {arguments} {
    commands
```

```
    return value
}
```

3. 0 Simple Procedure Example

```
proc say_hello {} {
    puts "Hello VLSI"
}
say_hello
```

Output:

```
Hello VLSI
```

- `{}` means no arguments

4. 0 Procedure with Arguments

```
proc add {a b} {
    set sum [expr $a + $b]
    puts "Sum = $sum"
}
add 5 10
```

Output:

```
Sum = 15
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Sum = 15
```

5. 0 Returning a Value from Procedure ⭐

👉 `puts` prints

👉 `return` sends value back

```
proc add {a b} {
```

```
    return [expr $a + $b]
}

set result [add 4 6]
puts "Result = $result"
```

Output:

```
Result = 10
```

6. 0 Why `return` is Important in VLSI

Because:

- Tool commands return values
- We store results
- We reuse outputs

VLSI-style Example

```
proc check_slack {slack} {
    if {$slack < 0} {
        return "FAIL"
    } else {
        return "PASS"
    }
}
puts [check_slack -0.3]
puts [check_slack 0.5]
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
FAIL
```

PASS

7.0 Local vs Global Variables ★ ★ ★ (Interview Favorite)

Inside procedure → LOCAL

```
proc demo {} {
   set a 10
}
```

Outside:

```
puts $a   ; # ERROR
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
can't read "a": no such variable
   while executing
"puts $a"
   (file "./p.tcl" line 7)
```

Making variable GLOBAL

```
set a 5
proc demo {} {
   global a
   incr a
}
demo
puts $a
```

Output:

```
6
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
6
```

📌 Avoid global variables in real projects unless required.

8. 0 Default Argument Values

```
proc report_mode {{mode setup}} {
   puts "Mode = $mode"
}


report_mode
report_mode hold
```

Output:

```
Mode = setup
Mode = hold
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Mode = setup
Mode = hold
```

9. 0 VLSI-Oriented Procedure Example ★ ★ ★

```
proc report_timing_status {slack} {
   if {$slack < 0} {
      puts "Timing VIOLATION"
   } else {
      puts "Timing CLEAN"
   }
}
report_timing_status -1.2
report_timing_status 0.8
```

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Timing VIOLATION

Timing CLEAN

10.0 Procedure Calling Tool Commands (Real Life)

```
proc analyze_clock {clk} {
   puts "Analyzing clock: $clk"
   # report_timing -clock $clk
}

set clocks {clk1 clk2 clk3}

foreach clk $clocks {
   analyze_clock $clk
}
```

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Analyzing clock: clk1

Analyzing clock: clk2

Analyzing clock: clk3

❗ Common Mistakes with `proc`

Mistake	Reason
Forgetting `return`	No output
Using global unnecessarily	Hard to debug
Modifying tool DB inside proc blindly	Dangerous
Same proc name reused	Overwrites

🧠 Interview Questions ⭐

Q1: Difference between `puts` and `return`?

A:

- `puts` → print only
- `return` → send value back

Q2: Are variables inside proc global?

A: No, they are local by default.

☑ Chapter 7 Summary

✓ `proc` creates reusable code

✓ Arguments pass values

✓ `return` sends result back

✓ Variables are local by default

✓ Used heavily in synthesis, STA, PnR scripts

🧪 Practice Exercise

```
proc power_check {power} {
   if {$power > 10} {
      return "High Power"
   } else {
      return "Low Power"
   }
}
puts [power_check 8]
puts [power_check 15]
```

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Low Power

High Power

📘 Chapter 8: Lists & Arrays – Deep Dive (EDA Level)

- Advanced list commands
- Array operations
- Tool database style usage
- Interview & real-project patterns

📘 Chapter 8: Lists & Arrays – Deep Dive (EDA / VLSI Level)

🌟 This chapter is CRITICAL for real VLSI TCL scripting

Most tool commands (`get_cells`, `get_nets`, etc.) return lists / collections.

🔁 Quick Recall

List → ordered collection of values

Array → key–value mapping

🔹 PART 1: Advanced List Operations

1. 0 Creating Lists (Different Ways)

set cells {U1 U2 U3}

set nets "n1 n2 n3"

Both are valid.

2.0 `lindex` – Get Element

puts [lindex $cells 0]

puts [lindex $cells end]

Output:

U1

U3

- `end` is very useful in scripts.

3. 0 `llength` – List Size

puts [llength $cells]

Output: 3

4. 0 `lappend` – Add Elements

lappend cells U4 U5

puts $cells

Output:

U1 U2 U3 U4 U5

5. 0 `linsert` – Insert at Position

set cells [linsert $cells 1 NEW_CELL]

puts $cells

Output: U1 NEW_CELL U2 U3 U4 U5

6. 0 `lreplace` – Replace Elements

set cells [lreplace $cells 0 0 INV1]

puts $cells

Output:

INV1 NEW_CELL U2 U3 U4 U5

7.0 `lsearch` – Find Element ⭐

set idx [lsearch $cells U3]

puts $idx

Output:

2

If not found:

-1

🧠 VLSI Example (Cell Check)

set cells {INV BUF AND2 NAND2 OR2}

if {[lsearch $cells NAND2] != -1} {

    puts "NAND2 exists"

}

Output:

NAND2 exists

8.0 `lsort` – Sort List

set slacks {0.2 -1.0 0.5 -0.3}

puts [lsort -real $slacks]

Output:

-1.0 -0.3 0.2 0.5

- Very useful for worst slack detection

🧠 STA Example

set worst_slack [lindex [lsort -real $slacks] 0]

puts "Worst slack = $worst_slack"

Output:

-1.0 -0.3 0.2 0.5

Worst slack = -1.0

🔹 PART 2: Arrays – EDA Style Usage ⭐⭐⭐⭐

Arrays are used to store attributes of objects.

9. 0 Creating Array Entries

set delay(U1) 1.2

set delay(U2) 0.8

set delay(U3) 1.5

10.0 Accessing Array

puts $delay(U2)

Output:

0.8

11.0 Get All Keys

set keys [array names delay]

puts $keys

Output:

U3 U1 U2

12.0 Loop Over Array ⭐

foreach cell [array names delay] {

    puts "$cell delay = $delay($cell)"

}

Output:

U3 delay = 1.5

U1 delay = 1.2

U2 delay = 0.8

13. 0 Check If Key Exists

```
if {[info exists delay(U4)]} {
   puts "Delay exists"
} else {
   puts "No delay info"
}
```

Output:

No delay info

14.0 Remove Array Element

```
unset delay(U2)
```

15.0 Clear Entire Array

```
array unset delay
```

🔁 List vs Array (REAL VLSI VIEW)

List → Object names

```
set cells {U1 U2 U3}
```

Array → Object attributes

```
set cell_type(U1) NAND
set cell_type(U2) INV
```

📌 This pattern is everywhere in EDA scripts

🧠 Real VLSI-Style Script Example ⭐⭐⭐

```
set cells {U1 U2 U3}

set power(U1) 12
set power(U2) 5
set power(U3) 9

foreach cell $cells {
   if {$power($cell) > 10} {
      puts "$cell is HIGH POWER"
   }
}
```

Output:

```
U1 is HIGH POWER
```

❗ Common Mistakes (Very Important)

Mistake	Why Wrong
`$cells(0)`	List is NOT array
`[lindex delay U1]`	Array is NOT list
Modifying list inside foreach	Risky
Assuming array order	Arrays are unordered

🧠 Interview Questions ⭐

Q: What do tool commands return?

A: Lists / collections.

Q: When to use array over list?

A: When mapping attributes to objects.

☑ Chapter 8 Summary

✓ Advanced list commands (`lindex`, `lsearch`, `lsort`)

✓ Arrays store object properties

✓ Arrays heavily used in EDA tools

✓ List = objects, Array = attributes

✓ Essential for STA / PnR scripts

🧪 Practice Exercise (Strongly Recommended)

```
set paths {P1 P2 P3}

set slack(P1) 0.2
set slack(P2) -1.1
set slack(P3) 0.5

foreach p $paths {
   if {$slack($p) < 0} {
      puts "$p has violation"
   }
}
```

Output:

P2 has violation

📘 Chapter 9: File Handling in TCL

- Reading reports
- Writing output files
- Parsing timing/power reports
- Real VLSI automation examples

📘 Chapter 9: File Handling in TCL

⭐ Extremely important for VLSI automation

(Parsing reports, generating summaries, logs, QoR files)

In real projects, 90% TCL scripts read or write files.

1. 0 Why File Handling is Needed in VLSI?

- Read timing reports
- Read power reports
- Generate summary reports
- Log violations automatically
- Post-process tool outputs

2. 0 Basic File Operations Flow

👉 **3 simple steps:**

1. Open file
2. Read / Write
3. Close file

3. 0 Opening a File

Syntax

set fp [open filename mode]

Mode	Meaning
`r`	Read
`w`	Write (Overwrite)
`a`	Append

4. 0 Writing to a File

Example

set fp [open "report.txt" w]

puts $fp "Timing Analysis Report"

puts $fp "---------------------"

close $fp

- ✓ `puts $fp` → writes to file
- ✓ `puts` alone → prints to terminal

🧠 VLSI Example (Write Summary)

set fp [open "summary.rpt" w]

puts $fp "Block : ALU"

```
puts $fp "Status: PASS"

close $fp
```

5. 0 Appending to a File

```
set fp [open "summary.rpt" a]
puts $fp "Clock: clk1"
close $fp
```

6.0 Reading a File Line-by-Line ★★★

Syntax

```
while {[gets $fp line] != -1} {
    commands
}
```

Example

```
set fp [open "data.txt" r]
while {[gets $fp line] != -1} {
    puts $line
}
close $fp
```

7.0 Reading Entire File at Once

```
set fp [open "data.txt" r]
set content [read $fp]
close $fp
```

```
puts $content
```

- Useful for small files only

8. 0 Real VLSI Example: Parse Slack Values

Assume `**timing.rpt**` contains:

```
Path 1 slack = 0.3
Path 2 slack = -1.2
Path 3 slack = 0.5
```

Script

```
set fp [open "timing.rpt" r]

while {[gets $fp line] != -1} {
    if {[string match "*slack*" $line]} {
        puts $line
    }
}
close $fp
```

Output:

Explanation:

set fp [open "timing.rpt" r]

👉 Opens the file `timing.rpt` in read mode (`r`)

👉 `fp` is the file handle used to access the file

```
while {[gets $fp line] != -1} {
```

👉 Reads the file one line at a time

👉 Each line is stored in variable `line`

👉 Loop continues until end of file

```
   if {[string match "*slack*" $line]} {
```

👉 Checks whether the current line contains the word `slack`

👉 `*slack*` means slack can be anywhere in the line

```
       puts $line
```

👉 If the line contains `slack`, print that line

```
   }
}
```

👉 Ends the `if` and `while` blocks

```
close $fp
```

👉 Closes the file (good practice)

In one sentence (VLSI context):

👉 **This script scans a timing report line by line and prints only the lines that contain slack information ⏱️ .**

9.0 Extract Data from Line (Simple Parsing)

```
set words [split $line " "]
set slack [lindex $words end]


puts "Slack = $slack"
```

- This technique is used everywhere in report parsing.

10.0 Writing Violation Report Automatically ⭐⭐⭐

```
set in  [open "timing.rpt" r]
set out [open "violations.rpt" w]

while {[gets $in line] != -1} {
    if {[string match "*slack*" $line]} {
        set slack [lindex [split $line " "] end]
        if {$slack < 0} {
            puts $out $line
        }
    }
}
close $in
close $out
```

🔥 **This is REAL VLSI automation**

11. 0 File Exists Check

```
if {[file exists "timing.rpt"]} {
    puts "Report found"
}
```

12. 0 Common File Handling Mistakes ❌

| Mistake | Issue |

| ---------- | ---------- |

| Forgetting `close` | File corruption |

| Using `read` for big files | Memory issue |

| Not checking file exists | Script crash |

| Hardcoding paths | Non-portable scripts |

🧠 Interview Questions ⭐

Q: How do you read a report line by line in TCL?

A: Using `gets` inside a `while` loop.

Q: Difference between `w` and `a` mode?

A:

* `w` → overwrite

* `a` → append

☑ Chapter 9 Summary

✓ Open, read, write, close files

✓ Parse reports line-by-line

✓ Extract data using `split`

✓ Generate violation summaries

✓ Core skill for STA / PnR

🧪 Practice Exercise

Read numbers from file and print only negative values

(You can try this once you create a small text file.)

📘 Chapter 10: Regular Expressions in TCL

- `regexp`
- `regsub`
- Powerful report parsing
- Interview + real tool examples

📘 Chapter 10: Regular Expressions in TCL

⭐ Power tool for report parsing in VLSI

This chapter separates basic TCL users from industry-ready VLSI engineers.

1. 0 Why Regular Expressions are Needed?

In real VLSI:

- Reports are unstructured text
- `split` is sometimes not enough
- You need pattern matching

👉 This is where `regexp` and `regsub` are used.

2. 0 `regexp` — Pattern Matching

Basic Syntax

regexp pattern string

Returns:

- `1` → match found
- `0` → no match

Simple Example

set line "Slack = -0.25"

if {[regexp {Slack} $line]} {

    puts "Slack info found"

}

Output:

Slack info found

3. 0 Extracting Values using `regexp` ★ ★ ★

Syntax

regexp pattern string variable

Example: Extract Slack Value

set line "Slack = -0.25"

regexp {Slack = ([\-0-9\.]+)} $line slack

puts $slack

Output:

-0.25

- Parentheses `()` capture values
- `[]` defines allowed characters

4.0 Understanding the Pattern (Important)

Pattern:

([\-0-9\.]+)

Means:

- `-` optional negative

- digits `0–9`
- dot `.`
- `+` → one or more characters

👉 Perfect for timing / power values.

5. 0 Real VLSI Example: Parse Timing Report



Report line:

Path 2 Slack = -1.12

Script:

```
set fp [open "timing.rpt" r]

while {[gets $fp line] != -1} {
    if {[regexp {Slack = ([\-0-9\.]+)} $line slack]} {
        if {$slack < 0} {
            puts "Violation slack = $slack"
        }
    }
}
close $fp
```

🔥 This is exactly how STA automation works

6. 0 Multiple Capture Groups

```
set line "Path 3 Slack = -0.8"
regexp {Path ([0-9]+) Slack = ([\-0-9\.]+)} $line path slack
```

puts "Path=$path Slack=$slack"

Output:

Path=3 Slack=-0.8

7. 0 `regsub` — Replace Text ★ ★ ★

Used to modify strings

Syntax

regsub pattern string replacement new_string

Example

set line "Slack = -1.25"

regsub {Slack = } $line "" result

puts $result

Output:

-1.25

8. 0 Clean Report Lines (VLSI Use Case)

set line " Slack =   -0.50 "

regsub -all {\s+} $line " " clean

puts $clean

- `-all` → replace everywhere
- `\s` → whitespace

Output:

Slack = -0.50

set line " Slack =   -0.50 "

👉 Stores a string in variable `line`

👉 Notice it has extra spaces at the start, middle, and end

regsub -all {\s+} $line " " clean

👉 `regsub` = replace text using a pattern

Breakdown:

- `-all` → replace every match
- `\s+` → one or more whitespace characters (spaces, tabs)
- `$line` → input string
- `" "` → replace with single space
- `clean` → store result here

So this line removes extra spaces and keeps only one space between words.

puts $clean

👉 Prints the cleaned string

☑ **Output**

Slack = -0.50

9.0 `string match` vs `regexp` (Interview ⭐)

Feature	string match	regexp
Simple pattern	✓	✓
Capture values	❌	✓
Complex parsing	❌	✓
Performance	Faster	Slower

👉 **Use:**

- `string match` → quick checks
- `regexp` → extraction

10.0 Common Regex Mistakes ❌

Mistake	Why
Forgetting `{}`	Escaping issues
Overusing regex	Slow scripts
Not checking match	Script crash
Greedy patterns	Wrong data

🧠 Interview Questions ⭐

Q: How do you extract slack from timing report?

A: Use `regexp` with capture groups.

Q: Difference between `split` and `regexp`?

A:

* `split` → delimiter based
* `regexp` → pattern based

☑ Chapter 10 Summary

✓ `regexp` for matching & extraction

✓ Capture groups using `()`

✓ `regsub` for text replacement

✓ Essential for report parsing

✓ Used heavily in STA & signoff

🍼 Practice Exercise

set line "Power = 12.5 mW"

Extract numeric power value using regexp

set line "Power = 12.5 mW"

regexp {([0-9.]+)} $line power

puts $power

Output:

12.5

📘 Chapter 11: EDA Tool Commands & Collections

- `get_cells`, `get_nets`
- Collections vs lists
- Filters (`-filter`)
- Real Design Compiler / PrimeTime examples

This is where TCL meets real tools 🚀

📘 Chapter 11: EDA Tool Commands & Collections (VLSI Core Chapter)

⭐ This is where pure TCL becomes VLSI TCL

Up to now, everything you learned works in `tclsh`.

From this chapter onward, commands are EDA-tool specific (DC / PT / ICC2 / Innovus).

1. 0 What Are EDA Tool Commands?

EDA tools expose their design database using TCL commands.

Examples:

- `get_cells`

- `get_nets`
- `get_ports`
- `get_pins`
- `get_clocks`

These commands return collections.

2. 0 What is a Collection? ⭐ ⭐ ⭐ (Very Important)

👉 A collection is a tool-managed list of design objects

It behaves similar to a TCL list, but not exactly the same.

Example:

set cells [get_cells]

- `cells` is a collection, not a normal TCL list.

3.0 Common `get_*` Commands

Command	What it returns
`get_cells`	All cells
`get_nets`	All nets
`get_ports`	IO ports
`get_pins`	Pins
`get_clocks`	Clocks

Example

set all_cells [get_cells]

puts "Total cells = [sizeof_collection $all_cells]"

4.0 `sizeof_collection` ⭐ ⭐ ⭐

👉 Used to count objects in a collection

```
set nets [get_nets]
puts [sizeof_collection $nets]
```

❌ `llength` does NOT work reliably on collections

5. 0 Looping Over Collections ⭐⭐⭐⭐

❌ Wrong (Common Mistake)

```
foreach c $cells {
   puts $c
}
```

✓ Correct Way

```
foreach_in_collection c $cells {
   puts [get_object_name $c]
}
```

- `foreach_in_collection` is mandatory for collections

6. 0 `get_object_name` ⭐⭐⭐

Collections store handles, not names.

```
foreach_in_collection c $cells {
   puts [get_object_name $c]
}
```

7. 0 Filtering Objects (`-filter`) ⭐⭐⭐⭐

Used to select specific objects

Syntax

get_cells -filter "condition"

Example: Only sequential cells

set seq_cells [get_cells -filter "is_sequential == true"]

Example: Cells with high area

get_cells -filter "area > 10"

8. 0 VLSI Practical Example: Count Flops

set flops [get_cells -filter "is_sequential == true"]

puts "Total flops = [sizeof_collection $flops]"

- Very common interview + project script

9. 0 Working with Clocks

set clocks [get_clocks]

foreach_in_collection clk $clocks {

  puts [get_object_name $clk]

}

10.0 Access Object Attributes ★ ★ ★

Syntax

get_attribute object attribute_name

Example

```
foreach_in_collection c [get_cells] {
   set area [get_attribute $c area]
   puts "Cell area = $area"
}
```

11.0 Combining Filter + Attribute

```
foreach_in_collection c [get_cells -filter "area > 5"] {
   puts "[get_object_name $c] : [get_attribute $c area]"
}
```

12.0 Collection → TCL List (If Needed)

```
set cell_list [get_object_name [get_cells]]
puts $cell_list
```

- Now `foreach` works normally.

❗ Common Mistakes (CRITICAL)

Mistake	Why wrong
Using `foreach` on collections	Breaks scripts
Using `llength`	Invalid for collections
Printing object directly	Shows handle
Forgetting `get_object_name`	Unreadable output

🧠 Interview Questions ⭐⭐⭐

Q: Difference between list and collection?

A:

- List → TCL managed
- Collection → Tool managed

Q: How to loop over cells?

A: Use `foreach_in_collection`

Q: How to filter cells?

A: Using `-filter`

☑ Chapter 11 Summary

✓ `get_*` commands return collections

✓ Collections ≠ lists

✓ Use `sizeof_collection`

✓ Use `foreach_in_collection`

✓ `-filter` selects objects

✓ `get_attribute` accesses properties

🧪 Practice Exercise

Print name and area of all cells with area > 2

```
# Get all cells in the design
set cells [get_cells]

# Loop through each cell
foreach_in_collection cell $cells {
    # Get cell name
    set name [get_object_name $cell]
```

```
    # Get cell area
    set area [get_attribute $cell area]

    # Print only if area > 2
    if {$area > 2} {
        puts "Cell: $name   Area: $area"
    }
}
```

- get_cells → collects all cells in the design
- foreach_in_collection → loops over a **collection**
- get_attribute area → reads the **area** of each cell
- if {$area > 2} → filters required cells
- puts → prints **name + area**

Cell: U12   Area: 2.45

Cell: U27   Area: 3.10

Cell: U45   Area: 5.60

📘 Chapter 12: Practical VLSI TCL Scripts

- Count cells, flops, gates
- Find worst slack
- Unconstrained paths
- QoR automation
- Real interview-level scripts

This is the most important chapter for jobs.

So far, in a proper TCL-for-VLSI learning flow, we've already covered:

- Basics & syntax
- Variables, `set`, `puts`
- Expressions & math

- Strings & lists
- Conditionals (`if`, `switch`)
- Loops (`for`, `while`, `foreach`)
- Procedures (`proc`)
- Regular expressions (`regexp`, `regsub`)
- File handling (basic)

📘 Chapter 11: Arrays & Dictionaries in TCL (Very Important for VLSI)

In VLSI tools (PrimeTime, Innovus, DC, ICC2), arrays and dictionaries are heavily used to store:

- timing data
- cell properties
- reports info
- key–value pairs

🔹 11.1 TCL Arrays (Associative Arrays)

👉 TCL arrays are NOT numeric index arrays

👉 They are key–value pairs (like hash maps)

◇ Syntax

set arrayName(key) value

◇ Example 1: Simple Array

set timing(slack) -0.25

set timing(wns) -0.50

set timing(tns) -12.3

puts $timing(slack)

☑ **Output:**

-0.25

- Use in VLSI: storing slack, WNS, TNS from reports

◇ Example 2: Print All Array Elements

```
foreach key [array names timing] {
   puts "$key = $timing($key)"
}
```

☑ **Output:**

```
slack = -0.25
wns = -0.50
tns = -12.3
```

◇ Example 3: Check If Key Exists

```
if {[info exists timing(wns)]} {
   puts "WNS exists"
}
```

- Very common in tool scripting to avoid errors

◇ Example 4: Delete Array Element

```
unset timing(tns)
```

◇ Example 5: Delete Entire Array

```
unset timing
```

◇ 11.2 Arrays from Loops (Real VLSI Style)

```
set cells {NAND2 NOR2 INV DFF}

foreach cell $cells {
   set area($cell) [expr {int(rand()*10 + 1)}]
}

foreach cell [array names area] {
   puts "Cell $cell has area $area($cell)"
}
```

Output:

Cell DFF has area 8

Cell INV has area 9

Cell NOR2 has area 10

Cell NAND2 has area 5

- Used when parsing cell libraries or reports

◇ 11.3 Dictionaries (Modern TCL – Preferred)

👉 Dictionaries are cleaner and safer than arrays

👉 Used heavily in advanced tool scripting

🔸 **Create Dictionary**

```
set dictData [dict create slack -0.2 wns -0.5 tns -10.2]
```

🔹 **Access Dictionary Value**

```
puts [dict get $dictData slack]
```

🔹 **Add / Update Value**

```
dict set dictData hold_slack 0.12
```

🔹 **Print All Dictionary Data**

```
dict for {key value} $dictData {
    puts "$key = $value"
}
```

🔹 **Check If Key Exists**

```
if {[dict exists $dictData wns]} {
    puts "WNS present"
}
```

🔹 Arrays vs Dictionaries (Interview Favorite)

Feature	Array	Dictionary
Syntax	old style	modern
Safety	less	more
Nested data	hard	easy
Tool usage	common	preferred

- Interview Tip:
- **Use dictionary when data is passed between procedures.**

🧠 **Mini VLSI Example (Timing Summary Script)**

```
set timing_dict [dict create]

dict set timing_dict WNS -0.45
dict set timing_dict TNS -15.3
dict set timing_dict Violations 120

dict for {k v} $timing_dict {
   puts "$k : $v"
}
```

☑ **Chapter 11 Summary**

You learned:

- TCL arrays (key-value storage)
- Looping through arrays
- Dictionary creation & usage
- Practical VLSI-style examples

📘 Chapter 12: File Parsing & Report Analysis (Timing / Power / Area)

🔥 This is CORE SKILL for VLSI automation & interviews

📘 Chapter 12: File Parsing & Report Analysis in TCL (VLSI Core Skill)

In VLSI tools, you will constantly:

- read timing / power / area reports
- extract numbers (slack, WNS, TNS, cell count)
- take decisions based on report data

👉 This chapter makes you job-ready for DC / PT / Innovus scripting.

◇ 12.1 Opening & Reading a File

🔸 Basic Syntax

```
set fp [open "report.txt" r]
set data [read $fp]
close $fp
```

- `r` → read mode

◇ Example 1: Print File Content

```
set fp [open "timing.rpt" r]
set content [read $fp]
puts $content
close $fp
```

◇ 12.2 Reading File Line-by-Line (MOST USED)

👉 This is how real report parsing is done.

```
set fp [open "timing.rpt" r]

while {[gets $fp line] != -1} {
    puts "LINE: $line"
}
close $fp
```

- `gets` reads one line at a time

◇ 12.3 Search for a Keyword (Slack / WNS / TNS)

🔹 **Example**: Detect Slack Line

```
set fp [open "timing.rpt" r]

while {[gets $fp line] != -1} {
   if {[regexp {Slack} $line]} {
      puts "Found: $line"
   }
}
close $fp
```

- Used to locate important report sections

◇ 12.4 Extract Numbers from Report Lines

🔹 **Example Timing Report Line**

```
Slack (MET) :   -0.35
```

🔹 **Extract Slack Value**

```
regexp {Slack.*:\s+(-?\d+\.\d+)} $line match slack
puts "Slack = $slack"
```

🔹 **12.5 Store Extracted Data in Array**

```
set timing(slack) $slack
```

🔹 **12.6 Complete Real-Style Example (Timing Report)**

📄 timing.rpt

WNS : -0.45

TNS : -12.3

Violations : 85

📜 TCL Script

```
set fp [open "timing.rpt" r]

while {[gets $fp line] != -1} {

    if {[regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns]} {
        set timing(WNS) $wns
    }

    if {[regexp {TNS\s*:\s*(-?\d+\.\d+)} $line -> tns]} {
        set timing(TNS) $tns
    }

    if {[regexp {Violations\s*:\s*(\d+)} $line -> vio]} {
        set timing(Violations) $vio
    }
}
close $fp

puts "WNS = $timing(WNS)"
puts "TNS = $timing(TNS)"
```

```
puts "Violations = $timing(Violations)"
```

🔹 12.7 Decision Making from Reports (Automation)

🔸 Example: Timing Pass/Fail

```
if {$timing(WNS) >= 0} {
   puts "☑ Timing PASSED"
} else {
   puts "❌ Timing FAILED"
}
```

- This is used in flow scripts.

🔸 12.8 Write Output to a File

```
set out [open "summary.txt" w]

puts $out "WNS = $timing(WNS)"
puts $out "TNS = $timing(TNS)"
puts $out "Violations = $timing(Violations)"

close $out
```

🔹 12.9 Append Mode (Add Data)

```
set out [open "summary.txt" a]
puts $out "Run completed at [clock format [clock seconds]]"
close $out
```

🧠 Interview-Level Insight

90% of VLSI TCL scripting = file parsing + regexp + arrays

If you master this chapter → you are already ahead of many candidates.

☑ Chapter 12 Summary

You learned:

- Open/read files
- Line-by-line parsing
- Extract timing numbers using `regexp`
- Store report data
- Make decisions
- Write summary reports

📘 Chapter 13: Tool Commands & Querying Design Objects (get_cells, get_pins, get_nets)

🔥 This is what interviewers LOVE

You want Chapter 12 specifically as “Practical VLSI Scripts”, not just theory.

📓 Chapter 12: Practical VLSI TCL Scripts (Industry-Style)

This chapter focuses on ready-to-use scripts you’ll see in:

- Design Compiler (DC)
- PrimeTime (PT)
- ICC2 / Innovus

◇ 12.1 Script 1: Check Timing Pass / Fail (MOST COMMON)

🎯 Task

Check WNS and print PASS / FAIL

```
set wns [get_attribute [get_timing_paths -max_paths 1] slack]

if {$wns >= 0} {
   puts "☑ TIMING PASSED (WNS = $wns)"
} else {
   puts "❌ TIMING FAILED (WNS = $wns)"
}
```

🧠 Explanation

- `get_timing_paths` → gets worst timing path
- `slack` → slack of that path
- `if` → decision making

- Used in nightly regression flows.

◇ 12.2 Script 2: Count Total Cells, FFs, Comb Cells

```
set total_cells [llength [get_cells *]]
set ff_cells    [llength [get_cells -filter "is_sequential == true"]]
set comb_cells  [llength [get_cells -filter "is_combinational == true"]]

puts "Total cells : $total_cells"
puts "FF cells    : $ff_cells"
puts "Comb cells  : $comb_cells"
```

- Interview favorite question

◇ 12.3 Script 3: List High Area Cells

🎯 Task

Print cells with area > 20

```
foreach cell [get_cells *] {
   set area [get_attribute $cell area]
   if {$area > 20} {
      puts "$cell  area=$area"
   }
}
```

- Used for area optimization

🔹 12.4 Script 4: Find Cells on Critical Path

```
set path [get_timing_paths -max_paths 1]
set cells [get_cells -of_objects $path]
foreach cell $cells {
   puts "Critical cell: $cell"
}
```

- Asked in PrimeTime interviews

◇ 12.5 Script 5: Find Nets with High Fanout

```
foreach net [get_nets *] {
   set fanout [get_attribute $net number_of_sinks]
   if {$fanout > 10} {
      puts "$net fanout = $fanout"
   }
}
```

- Used for buffer insertion analysis

🔹 12.6 Script 6: Dump Timing Summary to File

```
set fp [open "timing_summary.txt" w]

set wns [get_attribute [get_timing_paths -max_paths 1] slack]
set tns [get_attribute [get_timing_paths] total_negative_slack]

puts $fp "WNS = $wns"
puts $fp "TNS = $tns"
close $fp
```

- Used in automation pipelines

🔹 12.7 Script 7: Clock Information Script

```
foreach clk [get_clocks *] {
   set period [get_attribute $clk period]
   puts "Clock: $clk  Period: $period"
}
```

- Very common in CTS/debug

◇ 12.8 Script 8: Detect Unconnected Pins

```
foreach pin [get_pins *] {
   if {[get_nets -of_objects $pin] == ""} {
      puts "Unconnected pin: $pin"
   }
}
```

- Used in design sanity checks

🔹 12.9 Script 9: Store Design Info in Dictionary (Clean Way)

```
set design_info [dict create]

dict set design_info total_cells [llength [get_cells *]]
dict set design_info wns [get_attribute [get_timing_paths -max_paths 1] slack]

dict for {k v} $design_info {
   puts "$k : $v"
}
```

- Preferred in large scripts

🧠 Interview Insight (VERY IMPORTANT)

If interviewer asks:

“How do you use TCL in VLSI?”

👉 Your answer:

Query design objects, extract attributes, automate analysis, and make decisions.

☑ Chapter 12 Summary

You now know how to:

- Write real tool scripts
- Extract timing, area, fanout
- Automate reports
- Use TCL like an EDA engineer

📓 Chapter 13: Advanced Timing & Path-Based Scripting (PrimeTime Focus)

📘 Chapter 13: Debugging & Best Practices in TCL for VLSI

🔹 13.1 Why Debugging Matters in VLSI

In large scripts:

- Thousands of lines of TCL
- Multiple tool commands (`get_cells`, `get_timing_paths`)
- Reports change every run

❌ Without debugging:

- Script silently fails
- Wrong report analysis
- Time wasted in design flows

🔸 13.2 Basic Debugging Techniques

1. 0 Print Statements (`puts`)

```
set wns [get_attribute [get_timing_paths -max_paths 1] slack]
puts "WNS = $wns"
```

- Use liberally to see intermediate values
- Helps check if variables hold expected data

2. 0 Using `info` Commands

```
info exists timing(WNS)   ;# Check if variable exists
info commands             ;# List all defined procedures
info procs                ;# List all procs
```

3. 0 Check Collections vs Lists

```
set cells [get_cells *]
puts [llength $cells]          ;# WRONG if $cells is collection
puts [sizeof_collection $cells] ;# Correct
```

- Mistaking collections for lists is the most common source of errors.

4. 0 Error Handling with `catch`

```
if {[catch {set wns [get_attribute [get_timing_paths -max_paths 1] slack]} err]} {
   puts "Error occurred: $err"
} else {
   puts "WNS = $wns"
}
```

- Prevents script from aborting abruptly
- Useful for nightly automation flows

◇ 13.3 Best Practices for VLSI TCL

1. 0 Always Use Procedures (`proc`)

- Avoid repeated code
- Improves readability

```
proc report_slack {path} {
   set wns [get_attribute $path slack]
   puts "Slack for $path = $wns"
}
```

2.0 Use Meaningful Variable Names

```
set worst_slack -0.45
set total_ff_cells 1200
```

- Easier to debug large scripts

3. 0 Comment Your Scripts

```
# Get all sequential cells in design
set flops [get_cells -filter "is_sequential == true"]
```

4. 0 Check Tool Data Before Access

```
if {[llength [get_timing_paths]] == 0} {
   puts "No paths found"
```

```
} else {
    set wns [get_attribute [get_timing_paths -max_paths 1] slack]
}
```

5. 0 Use Arrays / Dictionaries for Clean Data Storage

```
dict set design_info WNS $wns
dict set design_info TNS $tns
dict set design_info Violations $vio
```

- Avoids global variables
- Easy to pass between procedures

6. 0 Logging to Files

```
set fp [open "debug_log.txt" a]
puts $fp "WNS = $wns"
close $fp
```

- Always keep a log for regression scripts

7.0 Avoid Hardcoding

```
# BAD
set cell_name U123

# GOOD
set cell_name [lindex [get_cells *] 0]
```

- Makes scripts reusable for any design

8. 0 Modular Scripts

- Split into small procs
- Each proc does one task
- Easier to debug and maintain

◇ 13.4 Common Mistakes in VLSI TCL Scripts

Mistake	Consequence	Fix
Using `llength` on a collection	Returns wrong number	Use `sizeof_collection`
Forgetting `close` after `open`	File corruption	Always close files
Hardcoding design objects	Script fails on new design	Use `get_cells`, `get_ports` dynamically
No error handling	Script crashes	Use `catch` or `if` checks
Global variables	Hard to track	Use arrays or dictionaries

◇ 13.5 Mini Debugging Workflow

1. Add `puts` to track variable values
2. Check collection vs list usage
3. Catch errors using `catch`
4. Validate report data
5. Log results for automation

◇ 13.6 Real Example: Debugging Timing Script

set path [get_timing_paths -max_paths 1]

```
if {[llength $path] == 0} {
   puts "No timing paths found, exiting"
} else {
   if {[catch {set wns [get_attribute $path slack]} err]} {
      puts "Error: $err"
   } else {
      puts "WNS = $wns"
   }
}
```

☑ Chapter 13 Summary

- Always debug using `puts` and `catch`
- Prefer procedures, arrays, dictionaries
- Log outputs to files
- Check collections carefully
- Avoid hardcoding and global variables
- Split scripts into small, reusable modules

💡 Interview tip:

“How do you debug TCL scripts in VLSI?”

Answer with: puts, catch, arrays, collections, logs, modular procs

Interview Questions

The use of "", {} and [] in TCL scripting language with an example:

In TCL scripting, the characters `""`, `{}`, and `[]` each have a different purpose.

1. 0 Double Quotes: `""`

- Use: " " allows variable substitution and command substitution inside the string.
- Variables (like `$var`) are replaced with their value.
- Backslashes like `\n` or `\t` are interpreted.

Example:

set name "Sarif"

set age 26

puts "Hello, my name is $name and I am $age years old"

Output:

Hello, my name is Sarif and I am 26 years old

☑ Inside "", **`$name`** is replaced with its value.

2. 0 Curly Braces: {}

- Use: {} prevents variable substitution and prevents command substitution.
- The content is treated literally.

Example:

set name "Sarif"

puts {Hello, my name is $name}

Output:

Hello, my name is $name

☑ $name is not expanded because of {}.

Good for writing code blocks or long strings literally.

3. 0 Square Brackets: []

- Use: [] are for command substitution.
- TCL **executes** the command inside [] first, then replaces it with the result.

Example:

set a 5

set b 10

puts "The sum is [expr {$a + $b}]"

Output:

The sum is 15

☑ [expr {$a + $b}] is evaluated first, then replaced inside the string.

4. 0 Mixed Use Example

set name "Sarif"

set age 26

Using all three

puts "Hello $name, next year you will be [expr {$age + 1}] years old"

puts {Hello $name, next year you will be [expr {$age + 1}] years old}

puts "{ Hello $name, next year you will be [expr {$age + 1}] years old }"

Output:

Hello Sarif, next year you will be 27 years old

Hello $name, next year you will be [expr {$age + 1}] years old

{ Hello Sarif, next year you will be 27 years old }

☑ First line uses "" → variables and command substituted

☑ Second line uses {} → everything is literal, no substitution

5. 0 Another Practical Example: Loop

```
set list {1 2 3 4 5}

foreach i $list {
    puts "Number: $i"
}
```

Output:

Number: 1

Number: 2

Number: 3

Number: 4

Number: 5

- {} in `foreach` block → code block, no early substitution
- "" inside puts → $i is replaced with current value

☑ Summary Table

Symbol	Purpose	Example
""	Variable & command substitution	"Hello $name" → Hello Sarif`
{}	Literal string or code block, no substitution	{Hello $name} → Hello $name
[]	Command substitution	[expr {$a + $b}] → evaluates to 15

1.0 Exercise: Print net count using variable substitution

```
# Suppose we have a variable for number of nets
set net_count 45
```

```
# Use double quotes to print the value
puts "The design has $net_count nets"
```

Output:

```
The design has 45 nets
```

☑ "" allows $net_count to be replaced.

2 Exercise: Literal code block for multiple commands

```
# Use curly braces to group commands without substitution
set a 5
set b 10

if {$a < $b} {
    puts "a is less than b"
} else {
    puts "a is not less than b"
}
```

Output:

```
a is less than b
```

☑ {} prevents premature evaluation in the `if` condition or code block.

3.0 Exercise: Evaluate expression inside string

```
set a 7
set b 8

puts "The sum of $a and $b is [expr {$a + $b}]"
```

Output:

```
The sum of 7 and 8 is 15
```

☑ [expr {}] evaluates first, then prints the result inside " ".

4. 0 Exercise: Iterating over design pins using foreach

```
set pins {clk rst data_in data_out}

foreach pin $pins {
    puts "Configuring pin: $pin"
}
```

Output:

```
Configuring pin: clk
Configuring pin: rst
Configuring pin: data_in
Configuring pin: data_out
```

☑ {} used for the loop block, " " used to substitute `$pin`.

5.0 Exercise: Combine all three in one VLSI-like command

set width 8

set height 16

Create a rectangle with evaluated area

puts "Creating rectangle of size $width x $height, area = [expr {$width * $height}]"

Output:

Creating rectangle of size 8 x 16, area = 128

☑ " " → variable substitution

☑ [expr {}] → expression evaluation

☑ {} inside [expr {}] → prevent early evaluation

◇ Quick Interview Notes

1. `""` → variable substitution inside string

2. `{}` → literal strings or command blocks

3. `[]` → command substitution (evaluate first, then replace)

1. 0 Example: Concatenating strings using double quotes `""`

set cell_name "NAND2"

set instance "X1"

puts "Instantiating cell: $cell_name$instance"

Output:

Instantiating cell: NAND2X1

☑ `" "` allows variable substitution and concatenation inside the string.

2. 0 Example: Prevent substitution using curly braces `{}`

```
set cell "INV"

# Using braces prevents substitution
puts {Instantiating cell: $cell}
```

Output:

```
Instantiating cell: $cell
```

☑ `{}` treats everything literally → no variable substitution.

Useful for writing code blocks or multi-line strings in scripts.

3. 0 Example: Evaluating arithmetic using square brackets `[]`

```
set width 5
set height 3

puts "Total area = [expr {$width * $height}]"
```

Output:

```
Total area = 15
```

☑ `[ ]` evaluates a command (here `expr`) first, then substitutes its result in the string.

4. 0 Example: Using `[]` to call another TCL procedure

```
proc double {x} {
    return [expr {$x * 2}]
}

set value 7
puts "Double of $value is [double $value]"
```

Output:

Double of 7 is 14

☑ `[double $value]` executes the `double` procedure first, then substitutes the result.

5. 0 Example: Using `{}` for multi-line command blocks

```
set cells {NAND2 NOR2 INV DFF}

foreach cell $cells {
    puts "Processing cell: $cell"
    # Additional commands can go here
}
```

Output:

Processing cell: NAND2

Processing cell: NOR2

Processing cell: INV

Processing cell: DFF

☑ `{}` groups multiple commands for the `foreach` loop without early substitution.

☑ `$cell` inside `puts` is substituted because we use `""` there.

☑ Quick Interview Summary

Symbol	Purpose	Example Use
`""`	Variable substitution inside string	`"Cell name: $cell"`
`{}`	Literal string or command block	`{Cell name: $cell}`
`[]`	Command substitution	[expr {$a + $b}], [proc_name arg]

Mini VLSI TCL Script: Count Pins and Print Cells with >3 Pins

```
# List of cells with number of pins
set cell_pins {
   NAND2 4
   NOR2 3
   INV   2
   DFF   5
   XOR2  4
}

puts "Checking cells with more than 3 pins..."
puts "-----------------------------------"

# Iterate over the cell list
foreach item $cell_pins {
```

```
    # Split the cell name and pin count
    set cell_name [lindex $item 0]
    set pin_count [lindex $item 1]

    # Use [] to evaluate a condition, "" to substitute variables
    if {[expr {$pin_count > 3}]} {
        puts "Cell $cell_name has $pin_count pins → Eligible for optimization"
    } else {
        # Use {} to print literal string without evaluating variables
        puts {Cell $cell_name has 3 or fewer pins → Skip optimization}
    }
}
puts "-----------------------------------"
puts "Done!"
```

Explanation of TCL Symbols Used

1. `""` (double quotes)

- Used in `puts "Cell $cell_name has $pin_count pins"`
- Allows variable substitution for dynamic printing.

2. `{}` (curly braces)

- Used in `puts {Cell $cell_name has 3 or fewer pins → Skip optimization}`
- Prevents variable substitution → prints literally.
- Good for fixed messages.

3. `[]` (square brackets)

- Used in `if {[expr {$pin_count > 3}]}`
- `[expr {...}]` evaluates the arithmetic/boolean expression first.

- Can also be used for procedure calls.

Expected Output

Checking cells with more than 3 pins...

Cell NAND2 has 4 pins → Eligible for optimization

Cell NOR2 has 3 or fewer pins → Skip optimization

Cell INV has 3 or fewer pins → Skip optimization

Cell DFF has 5 pins → Eligible for optimization

Cell XOR2 has 4 pins → Eligible for optimization

Done!

I want to perform the addition operation on index 1

```
#!/usr/bin/tclsh

set cell_pins {
    {NAND2 4}
    {NOR2  3}
    {INV   2}
    {DFF   5}
    {XOR2  4}
}

set total_pins 0

foreach item $cell_pins {
```

```
    set pin_count [lindex $item 1]
    set total_pins [expr {$total_pins + $pin_count}]
}

puts "Total pins (sum of index 1) = $total_pins"
```

Output:

[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl

Total pins (sum of index 1) = 18

top 20 TCL scripting interviews question for VLSI domain

🔥 Top 20 TCL Scripting Interview Questions for VLSI

1.0 What is TCL and why is it used in VLSI?

Answer:

TCL (Tool Command Language) is used to automate EDA tools like Design Compiler, Prime Time, and Innovus. It helps in querying design data, parsing reports, and building automated flows.

2 Difference between list and array in TCL?

Answer:

List: Ordered collection of elements

Array: Key–value pair (associative array)

```
set mylist {a b c}
set myarray(key) value
```

3 What is a collection in VLSI TCL?

Answer:

A collection is a tool-specific object returned by commands like `get_cells`. It is not a normal TCL list.

4 Difference between `llength` and `sizeof_collection`?

Answer:

- `llength` → for TCL lists
- `sizeof_collection` → for EDA tool collections

5 What is `get_cells` and how is it used?

Answer:

`get_cells` is used to query cell instances from the design database.

get_cells -filter "is_sequential == true"

- **Gets only sequential cells** from the design.

Break it down

get_cells

- Collects all cells in the current design

-filter

- Applies a condition to select only some cells

"is_sequential == true"

- is_sequential is a cell attribute
- true means the cell has memory behavior

6 How do you filter objects in TCL?

Answer:

Using the `-filter` option with logical expressions.

get_cells -filter "area > 10 && is_combinational == true"

7 How do you get an attribute of a cell?

Answer:

Using `get_attribute`.

get_attribute [get_cells U1] area

- get_cells U1 → selects the cell named **U1**
- get_attribute ... area → asks for the **area value** of that cell

👉 So this command **returns the physical area of cell U1**.

In one line:

"Get the area of cell U1."

8 How do you check timing pass/fail using TCL?

Answer:

```
set wns [get_attribute [get_timing_paths -max_paths 1] slack]
if {$wns >= 0} {
   puts "Timing PASSED"
} else {
   puts "Timing FAILED"
}
```

- get_timing_paths -max_paths 1 → gets the **worst timing path 1 only**
- get_attribute ... slack → extracts its **slack value**
- wns stores that slack

Then:

- If wns ≥ 0 → **Timing PASSED**
- If wns < 0 → **Timing FAILED**

👉 In short: **the script checks worst slack and reports pass/fail.**

9 What is `get_timing_paths` used for?

Answer:

It extracts timing paths for STA analysis (worst paths, endpoints, slack).

10 How do you parse a timing report in TCL?

Answer: parse==analysis

Using file handling + `regexp`.

regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns

👉 It extracts the WNS value (number) from a text line and stores it in `wns`.

- Breakdown (easy words):
- `regexp` → search using a pattern
- `{WNS\s*:\s*(-?\d+\.\d+)}` → look for
- `WNS :` (with any spaces)
- followed by a number (can be negative)
- `$line` → the text being searched
- `wns` → variable where the found number is saved

Example:

If `$line` is:

WNS : -0.23

Then:

wns = -0.23

Pattern:

{WNS\s*:\s*(-?\d+\.\d+)}

Meaning (piece by piece):

`WNS`

→ **literally matches the text WNS**

`\s*`

→ **matches any spaces (0 or more)**

(space, tab, etc.)

`:`

→ **matches the colon `:`**

`\s*`

→ **again matches any spaces after the colon**

`(` `)`

→ **capture group**

(whatever matches inside is saved into `wns`)

`-?`

→ **optional minus sign**

(handles negative slack like `-0.25`)

`\d+`

→ **one or more digits**

(0–9)

`\.`

→ **decimal point**

`\d+`

→ **digits after the decimal**

In short:

👉 It matches text like:

WNS : 0.12

WNS:-0.45

WNS : -1.03

And captures only the number (e.g. `-0.45`) into `wns`.

11 What is `regexp` in TCL?

Answer:

Used for pattern matching and extracting data from strings or reports.

12 Difference between `regexp` and `regsub`?

Answer:

- `regexp` → match/extract
- `regsub` → search and replace

13 How do you loop over all cells in a design?

Answer:

```
foreach cell [get_cells *] {
   puts $cell
}
```

14 How do you find high-fanout nets?

Answer:

```
foreach net [get_nets *] {
   if {[get_attribute $net number_of_sinks] > 10} {
      puts $net
   }
```

}

Explanation:

`foreach net [get_nets *]`

→ Go through **each net** in the design one by one.

`get_attribute $net number_of_sinks`

→ Find **how many sinks (loads)** are connected to that net.

`if {[...] > 10}`

→ Check **if the net has more than 10 sinks**.

`puts $net`

→ If yes, **print the net name.**

In one line:

👉 This script prints all nets that are connected to more than 10 loads (high-fanout nets).

15 What is `catch` used for?

Answer:

Used for error handling to prevent script crash.

catch {command} error_msg

16 Difference between blocking and non-blocking in TCL?

Answer:

TCL executes commands sequentially; blocking/non-blocking is not applicable like Verilog.

17 How do you write reusable TCL code?

Answer:

Using procedures (`proc`) and passing arguments.

18 How do you store design data cleanly in TCL?

Answer:

Using dictionaries or arrays.

dict set info WNS -0.3

Explanation:

`dict set info WNS -0.3`

👉 **This stores data in a dictionary.**

- `info` → dictionary name
- `WNS` → key (label)
- `-0.3` → value

Meaning:

You are saving WNS = -0.3 inside a dictionary called `info`.

19 Common mistakes in VLSI TCL scripting?

Answer:

- Using `llength` on collections
- Hardcoding cell names
- No error handling
- No logging

20 How do you debug a TCL script in VLSI tools?

Answer:

Using:

- `puts` for printing
- `catch` for errors
- checking collections
- logging output to files

🎯 Final Interview Tip

If interviewer asks:

“How good are you in TCL?”

Your confident answer:

“I use TCL to query design objects, automate timing/area analysis, parse reports, and build reusable scripts for EDA flows.”

🔥 Top 10 TRICKY TCL Interview Questions for VLSI Domain —

🔥 Top 10 Tricky TCL Interview Questions (VLSI Focus)

1 Why does `llength [get_cells *]` sometimes give wrong results?

☑ Answer:

Because `get_cells` **returns a collection**, not a TCL list.

✓ Correct way:

sizeof_collection [get_cells *]

🧠 This is one of the MOST COMMON interview traps.

2 What happens if `get_cells` returns an empty collection? How do you handle it?

☑ Answer:

The script may fail when accessing attributes.

✓ Safe check:

set cells [get_cells -filter "is_sequential == true"]

if {[sizeof_collection $cells] == 0} {

    puts "No sequential cells found"

}

3 Difference between `get_attribute` and `report_attribute`?

☑ Answer:

- `get_attribute` → used in scripts, returns value
- `report_attribute` → prints info, not script-friendly

✓ Interview rule:

Use `get_attribute` for automation.

4 Why does this script fail sometimes?

set wns [get_attribute [get_timing_paths -max_paths 1] slack]

☑ Answer:

Because `get_timing_paths` may return no paths.

✓ Correct:

set path [get_timing_paths -max_paths 1]

```
if {[sizeof_collection $path] > 0} {
   set wns [get_attribute $path slack]
}
```

5 Can we use `foreach` directly on a collection?

- Yes and No
- Some tools allow it
- Best practice is to convert to list

✓ Safe way:

```
foreach_in_collection cell [get_cells *] {
   puts $cell
}
```

🧠 Interviewers LOVE `foreach_in_collection`

6 Difference between `==` and `eq` in TCL?

☑ Answer:

- `==` → numeric comparison
- `eq` → string comparison

✓ Example:

```
if {$mode eq "FAST"} { }
if {$slack >= 0} { }

set a "10"
set b "010"
```

```
set c "10"

# Numeric comparison
puts [expr {$a == $b}]     ;# 10 == 10 → true

# String comparison (different strings)
puts [expr {$a eq $b}]     ;# "10" != "010" → false

# String comparison (same strings)
puts [expr {$a eq $c}]     ;# "10" == "10" → true
```

Output:

1

0

1

Simple takeaway:

- == → compares **numbers**
- eq → compares **strings exactly (character by character)**

Explanation (simple):

- == → **numeric comparison**
  👉 "10" and "010" are both **number 10**, so result is **1 (true)**
- eq → **string comparison**
  👉 "10" and "010" are **different strings**, so result is **0 (false)**

7 Why is hardcoding object names a bad idea in VLSI TCL?

☑ Answer:

Because scripts should work for any design.

Hardcoding breaks reuse and automation.

❌ Bad:

get_cells U123

✓ Good:

get_cells -filter "is_sequential == true"

Example2

❌ Bad (hardcoded object name)

set area [get_attribute [get_cells U1] area]

puts $area

👉 **This works only if cell `U1` exists.**

If the design changes and `U1` is renamed or removed → script fails.

☑ **Good (no hardcoding, reusable)**

foreach cell [get_cells] {

  puts [get_attribute $cell area]

}

👉 **This works for any design, any cell names.**

Simple point:

Hardcoding = design-dependent script

No hardcoding = reusable & automation-friendly script ☑

8 What is the difference between `regexp` and `scan`?

☑ Answer:

- `regexp` → flexible pattern matching (used for reports)
- `scan` → fixed formatted input

✓ VLSI preference:

- `regexp` is **preferred for parsing reports.**

Example

Example input line

set line "WNS : -0.45"

Using `regexp` (flexible)

regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns

puts $wns

☑ **Works even if spaces change (`WNS:-0.45`, `WNS  :  -0.45`)**

Using `scan` (fixed format)

scan $line "WNS : %f" wns

puts $wns

- **Works only if the format is exactly `WNS : number`**

One-line takeaway:

- `regexp` → best for parsing reports (format may change)
- `scan` → best for strict, known formats

9 Why do EDA scripts prefer dictionaries over global variables?

☑ Answer:

- Cleaner data handling
- Easy to pass between procedures
- Avoids name conflicts

✓ **Example:**

dict set timing WNS -0.4

🛑 **Problem with global variables**

Global variables can:

- Get overwritten by mistake
- Cause name conflicts in big scripts
- Make debugging hard

☑ Dictionary = Clean & Safe Data Container

A dictionary groups related data under one name.

Simple Example (Timing data)

Using dictionary

dict set timing WNS -0.4

dict set timing TNS -12.3

puts "WNS = [dict get $timing WNS]"

puts "TNS = [dict get $timing TNS]"

🔍 What's happening?

- `timing` → **single container**
- `WNS`, `TNS` → keys
- `-0.4`, `-12.3` → values

🎯 **Why this is better than globals?**

❌ Global way (bad practice)

set WNS -0.4

set TNS -12.3

➡ Risky in large EDA scripts (names can clash)

☑ Dictionary way (preferred)

timing(WNS), timing(TNS) ;# logically grouped

🧠 **Interview One-Liner (VERY IMPORTANT)**

“EDA scripts use dictionaries because they keep related data together, avoid global variable conflicts, and are easier to pass between procedures.”

10 How do you debug a TCL script that fails silently?

☑ Answer:

1. Add `puts` at each step
2. Check collections with `sizeof_collection`
3. Use `catch`
4. Log outputs to files

✓ Example:

catch {set wns [get_attribute $path slack]} err

puts $err

Ultimate Interview One-Liner

“I always differentiate between TCL lists and EDA collections, use `foreach_in_collection`, handle empty queries, and write reusable scripts using procedures and dictionaries.”

What is TCL, and how have you used it in the VLSI domain?

“TCL (Tool Command Language) is a scripting language used to control and automate EDA tools. In the VLSI domain, I use TCL with tools like Design Compiler and PrimeTime to query design objects, extract timing and area information, parse reports, and automate analysis such as timing pass/fail checks and design summaries.”

What is the difference between a TCL list and a VLSI collection?

“A TCL list is a normal TCL data structure managed by the interpreter, while a VLSI collection is a tool-managed object returned by commands like **get_cells** or **get_nets**. Collections cannot be treated as normal lists, so commands like **llength** may give incorrect results; instead, we use **sizeof_collection** or **foreach_in_collection**.”

Why should we use sizeof_collection instead of llength in VLSI TCL?

“We use **sizeof_collection** because VLSI tool commands like **get_cells** return collections, not normal TCL lists. **llength** works only on TCL lists and can give wrong results for collections. **sizeof_collection** is tool-aware and correctly returns the number of objects in a collection.”

“llength **is for TCL lists,** sizeof_collection **is for EDA collections.”**

How do you check timing pass or fail using TCL in PrimeTime?

“In PrimeTime, I check timing pass or fail by extracting the worst slack using **get_timing_paths** and then applying an **if–else** condition. If the WNS is greater than or equal to zero, timing passes; otherwise, it fails.”

Example script:

```
set path [get_timing_paths -max_paths 1]

set wns  [get_attribute $path slack]


if {$wns >= 0} {

   puts "Timing PASSED"

} else {

   puts "Timing FAILED"

}
```

Basic TCL Interview Questions (With Answers)

The kind asked in **entry-level / freshers** / basic VLSI tool rounds.

1 What is TCL?

Answer:

TCL (Tool Command Language) is a scripting language used to automate tasks and control EDA tools in the VLSI domain.

2 Why is TCL popular in VLSI?

Answer:

Because most EDA tools like Design Compiler, PrimeTime, and Innovus support TCL for automation, reporting, and flow control.

3 How do you create a variable in TCL?

Answer:

set a 10

4 How do you print output in TCL?

Answer:

puts "Hello TCL"

5 How do you write comments in TCL?

Answer:

This is a comment

6 How do you perform arithmetic operations in TCL?

Answer:

set sum [expr {$a + $b}]

7 Difference between `"` and `{}` in TCL?

Answer:

- `"` allows variable substitution
- `{}` prevents variable substitution

puts "$a"

puts {$a}

8 What is a TCL list?

Answer:

A list is an ordered collection of elements.

set mylist {1 2 3}

9 How do you find the length of a list?

Answer:

llength $mylist

10 What is `foreach` used for?

Answer: iteration property 0 to last element

Used to loop through list elements.

```
foreach item $mylist {
    puts $item
}
```

11 How do you write an if-else condition in TCL?

Answer:

```
if {$a > 5} {
    puts "Greater"
} else {
    puts "Smaller"
}
```

12 What is a procedure in TCL?

Answer:

A procedure (`proc`) is used to create reusable code.

```
proc add {a b} {
    return [expr {$a + $b}]
}
```

13 How do you return a value from a procedure?

Answer:

Using the `return` command.

14 What is an array in TCL?

Answer:

An array is a key-value data structure.

```
set area(cell1) 10
```

15 How do you check if a variable exists?

Answer:

```
info exists var_name
```

16 What is `regexp` used for?

Answer:

Used for pattern matching and extracting data from strings.

17 What is file handling in TCL?

Answer:

Reading or writing files using `open`, `read`, `gets`, and `close`.

18 What is `catch` in TCL?

Answer:

Used for error handling to prevent script failure.

catch {command} error_msg

19 Difference between `break` and `continue`?

Answer:

- `break` → exits the loop
- `continue` → skips current iteration

20 is TCL case-sensitive?

Answer:

Yes, TCL is case-sensitive.

🎯 Quick Interview Tip

If you are a fresher, focus on:

- Variables
- Loops
- Lists
- Procedures
- Basic EDA commands (`get_cells`, `get_attribute`)

Top 20 TCL interviews questions for Design and verification engineers

1 What is TCL and where do you use it as a Design/Verification Engineer?

Answer:

TCL is a scripting language used to automate EDA tools. As a Design or Verification Engineer, it is used to run simulations, control tool flows, parse reports, and automate checks.

2 How is TCL different from Verilog/SystemVerilog?

Answer:

TCL is a scripting and automation language, while Verilog/SystemVerilog are hardware description and verification languages.

3 How do you create and access variables in TCL?

set a 10

puts $a

4 What is the difference between `"` and `{}` in TCL?

Answer:

`"` allows variable substitution, `{}` does not.

5 What is a TCL list and where is it used?

Answer:

A list is an ordered collection of elements, commonly used to store file names, test names, or signals.

6 What is a TCL array?

Answer:

An array is a key–value data structure used to store named data like test results or configuration values.

7 What is a procedure (`proc`) and why is it important?

Answer:

A procedure allows reusable and modular scripting, which is important for large design and verification flows.

8 How do you use loops in TCL?

```
foreach test $test_list {
    puts "Running $test"
}
```

9 What is `regexp` and how is it used in verification?

Answer:

`regexp` is used to extract information from logs and simulation reports, such as pass/fail status or errors.

10 How do you parse a simulation log in TCL?

Answer:

By reading the file line-by-line and matching patterns using `regexp`.

11 What is `catch` used for?

Answer:

It is used for error handling to prevent scripts from crashing when a command fails.

12 How do you automate regression runs using TCL?

Answer:

By looping over testcases, invoking simulator commands, capturing logs, and checking results automatically.

13 What is the role of TCL in UVM or verification environments?

Answer:

TCL controls simulation execution, regression management, and post-simulation log analysis, while UVM handles functional verification.

14 How do you run multiple testcases using TCL?

set tests {test1 test2 test3}

```
foreach t $tests {
    exec vsim $t
}
```

15 How do you check PASS/FAIL of a simulation in TCL?

Answer:

By searching the log file for keywords like “PASS”, “FAIL”, or “ERROR”.

16 What is the difference between blocking and non-blocking execution in TCL?

Answer:

TCL executes commands sequentially; blocking/non-blocking concepts are not applicable like in Verilog.

17 How do you write output to a file in TCL?

```
set fp [open "result.txt" w]
puts $fp "PASS"
close $fp
```

18 What is `exec` used for in TCL?

Answer:

`exec` is used to run OS or tool commands from a TCL script.

19 What are common TCL scripting mistakes in design/verification?

Answer:

- Hardcoding paths
- No error handling
- Not checking log results
- Poor modularization

❌ Bad TCL Script (common mistakes)

```
set fp [open "/home/user/design/timing.rpt" r]
while {[gets $fp line] >= 0} {
   if {[regexp {WNS} $line]} {
      puts $line
   }
}
close $fp
```

Why this is bad (mapped to your points)

1️⃣ **Hardcoding paths**

"/home/user/design/timing.rpt"

👉 Breaks if run on another machine or project.

2️⃣ **No error handling**

👉 If file doesn’t exist → script crashes.

3️⃣ Not checking log results

👉 Just prints WNS line, doesn’t decide PASS/FAIL.

4️⃣ Poor modularization

👉 All logic is written in one place, not reusable.

One-line takeaway (interview-ready):

A bad TCL script works only once; a good script works for every design.

20 How do you debug TCL scripts in verification flows?

Answer:

Using `puts`, `catch`, log files, and incremental testing of scripts.

🎯 Interview Tip for Design & Verification Roles

When asked about TCL, always connect it to:

- Automation
- Regression
- Log parsing
- Tool control

That shows real hands-on experience.

Explanation

In VLSI Design & Verification, TCL is not for theory — it's used to automate repetitive work, run regressions, parse logs, and control EDA tools.

So, in interviews, you should connect TCL to real usage, not just syntax.

☑ One Practical Example (covers all 4 points)

Scenario:

You ran multiple timing reports and want to automatically check timing.

```
set reports {run1.rpt run2.rpt run3.rpt}


foreach rpt $reports {
   set fp [open $rpt r]
   while {[gets $fp line] >= 0} {
      if {[regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns]} {
         if {$wns < 0} {
            puts "$rpt : TIMING FAILED"
         } else {
            puts "$rpt : TIMING PASSED"
         }
         break
      }
   }
   close $fp
}
```

🔍 How this map to interview keywords

Interview Word	Where it appears
Automation	Script checks timing automatically
Regression	Loops over multiple reports
Log parsing	Uses `regexp` to extract WNS
Tool control	Can be plugged into PrimeTime / Genus / VCS

🧠 One-Line Interview Answer

“I use TCL mainly for automating flows, running regressions, parsing tool reports, and controlling EDA tools.”

🔹 TCL Coding Problems for Verification Engineers

🟢 Level 1: Basics (Interview Warm-up)

Q1. Print numbers from 1 to 10

```
for {set i 1} {$i <= 10} {incr i} {
    puts $i
}
```

Output:

1

2

3

4

5

6

7

8

9

10

Q2. Check whether a number is even or odd

set num 7

```
if {[expr {$num % 2}] == 0} {
    puts "Even"
} else {
    puts "Odd"
}
```

Output:

Odd

Q3. Find length of a list

```
set tests {test1 test2 test3}
puts [llength $tests]
```

Output:

3

🟢 Level 2: Verification-Style Problems

Q4. Loop through testcases and print test names

```
set tests {sanity regression stress}
```

```
foreach t $tests {
   puts "Running test: $t"
}
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./p.tcl
Running test: sanity
Running test: regression
Running test: stress
```

- Used in regression control scripts

Q5. Count PASS and FAIL from a result list

```
set results {PASS FAIL PASS PASS FAIL}

set pass 0
set fail 0

foreach r $results {
   if {$r eq "PASS"} {
      incr pass
   } else {
      incr fail
   }
}

puts "PASS = $pass"
```

```
puts "FAIL = $fail"
```

Output:

PASS = 3

FAIL = 2

Q6. Store testcase result using array

```
set result(test1) PASS
set result(test2) FAIL

puts $result(test1)
```

Output:

PASS

🟢 Level 3: File & Log Parsing (VERY IMPORTANT)

Q7. Read a simulation log and print ERROR lines

```
set fp [open "sim.log" r]

while {[gets $fp line] != -1} {
    if {[regexp {ERROR} $line]} {
        puts $line
    }
}

close $fp
```

- Most common verification TCL task

Q8. Check PASS / FAIL from log file

Assume log contains:

TEST STATUS : PASS

```
set status "UNKNOWN"

set fp [open "sim.log" r]
while {[gets $fp line] != -1} {
    if {[regexp {TEST STATUS\s*:\s*(PASS|FAIL)} $line -> result]} {
        set status $result
    }
}
close $fp

puts "Simulation Result = $status"
```

Q9. Write regression summary to file

```
set fp [open "summary.txt" w]
puts $fp "Regression Completed"
puts $fp "Total Tests = 10"
puts $fp "PASS = 9"
puts $fp "FAIL = 1"
close $fp
```

Level 4: Practical Regression Automation

Q10. Run multiple testcases using TCL

```
set tests {test1 test2 test3}

foreach t $tests {
   puts "Running $t"
   exec vsim -c work.top +TEST=$t -do run.do > $t.log
}
```

- Real UVM regression flow

Explanation:

set tests {test1 test2 test3}

👉 Creates a list of test names to run in simulation.

foreach t $tests {

👉 Loop through each test in the list, one by one.

   puts "Running $t"

👉 Prints which test is currently running

(example: `Running test1`).

   exec vsim -c work.top +TEST=$t -do run.do > $t.log

👉 Runs the simulator:

- `vsim` → ModelSim/Questa simulator
- `-c` → command-line (no GUI)
- `work.top` → top module
- `+TEST=$t` → passes test name to the testbench
- `-do run.do` → runs commands from `run.do`
- `> $t.log` → saves output into a log file (`test1.log`, etc.)

}

👉 Ends the loop after all tests are run.

🧠 One-line summary (interview ready):

This TCL script runs multiple tests automatically, passes test names to the simulator, and saves logs for regression checking.

Q11. Stop regression if any test FAILS

```
foreach t $tests {
   exec vsim -c +TEST=$t > $t.log

   set fp [open "$t.log" r]
   set fail_found 0

   while {[gets $fp line] != -1} {
      if {[regexp {FAIL} $line]} {
         set fail_found 1
         break
      }
   }
   close $fp

   if {$fail_found} {
      puts "Stopping regression due to FAIL in $t"
      break
   }
}
```

Output:

🔥 Interview-level script

Q12. Use procedure to check log status

```
proc check_status {logfile} {
   set fp [open $logfile r]
   while {[gets $fp line] != -1} {
      if {[regexp {FAIL} $line]} {
         close $fp
         return "FAIL"
      }
   }
   close $fp
   return "PASS"
}

puts [check_status "test1.log"]
```

Output:

🧠 VERY COMMON INTERVIEW QUESTION

👉 “How do you use TCL in verification?”

“I use TCL mainly for **automation in verification** — running regressions, controlling simulators, parsing logs, and checking results automatically.”

“For example, I use TCL to loop over multiple tests, run the simulator in command-line mode, capture logs, and parse them to report PASS/FAIL without manual effort.”

That answer clearly shows hands-on verification experience ☑

☑ Simple Verification Example (Regression + Log)

```
set tests {test1 test2}


foreach t $tests {
   puts "Running $t"
   exec vsim -c work.top +TEST=$t -do "run -all; quit" > $t.log
}
```

Simple explanation:

- Runs multiple tests automatically
- Passes test name to the testbench
- Saves simulation output into log files
- Used in verification regression

Interview one-liner:

“I use TCL to automate running multiple verification tests and collect logs during regression.”

💡 Perfect Answer:

“I use TCL to automate simulations, manage regressions, parse logs for PASS/FAIL and errors, and generate summary reports.”

Regression means:

👉 Running the same set of tests again and again after a code change to make sure nothing is broken.

Simple example:

- You fix a bug in RTL
- You rerun all old tests
- If all tests pass → change is safe ☑

- If something fails → new bug introduced ❌

One-line interview answer:

Regression is repeatedly running test suites to ensure new changes don’t break existing functionality.

🎯 Your Current Level (Honest)

Based on earlier interaction:

- ☑ Basics: Good
- ☑ Concepts: Good

1-Page TCL Interview Revision Sheet — crisp, practical, and VLSI/Verification-focused.

You can literally revise this one night before interview.

🧠 TCL Interview Quick Revision (VLSI / Verification)

🔹 What is TCL?

- TCL = Tool Command Languag
- Used for automation, regression control, log parsing, EDA tool scripting
- Widely used in Synopsys, Cadence, Mentor tools

🔹 Basic Syntax

set a 10

puts $a

- Variables accessed using `$`
- No data types (everything is string internally)

◇ Common Commands

Command	Use
`set`	Assign value
`puts`	Print
`expr`	Arithmetic
`if / else`	Decision
`for / foreach / while`	Loop
`proc`	Procedure
`regexp`	Pattern matching
`open / gets / close`	File handling

◇ If–Else

```
if {$a > 5} {
   puts "Greater"
} else {
   puts "Smaller"
}
```

🔹 Loops

```
for {set i 0} {$i < 5} {incr i} {
   puts $i
}

foreach t {test1 test2} {
   puts $t
}
```

🔹 List

set l {a b c}

llength $l

lindex $l 0

- Normal TCL data structure

🔹 Collection (EDA Tools)

set cells [get_cells *]

sizeof_collection $cells

🔑 Difference

| List | Collection |

| Pure TCL | Tool-managed |

| `llength` | ❌ |

| `sizeof_collection` | ☑ |

| Faster for large data | ✓ |

◇ Why sizeof_collection instead of llength?

👉 Collections are handles, not real lists

👉 `llength` expands data → slow + memory heavy

◇ File Handling (VERY IMPORTANT)

set fp [open "log.txt" r]

while {[gets $fp line] != -1} {

```
    puts $line
}
close $fp
```

◇ Log Parsing (Interview Favorite)

```
if {[regexp {ERROR} $line]} {
    puts "Error Found"
}
```

Explanation:

```
if {[regexp {ERROR} $line]} {
```

👉 Checks whether the word `ERROR` exists anywhere inside the variable `line`.

- `regexp {ERROR} $line`

→ returns 1 (true) if `ERROR` is found

→ returns 0 (false) if not found

```
    puts "Error Found"
```

👉 If `ERROR` is found, it prints this message on the screen.

```
}
```

👉 Ends the `if` condition.

🧠 One-line summary:

This code scans a line of text and prints “Error Found” if the word ERROR appears in it.

◇ Procedure

```
proc add {a b} {
   return [expr {$a+$b}]
}
puts [add 3 4]
```

Output:

7

◇ Arrays

```
set result(test1) PASS
puts $result(test1)
```

Output:

PASS

◇ PASS / FAIL Check

```
if {[regexp {FAIL} $line]} {
   puts "Test Failed"
}
```

◇ Regression Use

- Run simulations
- Control test list
- Parse logs
- Generate summary

◇ Common Interview Questions

✓ Difference between list and collection

✓ How to parse log files

✓ Why TCL is used in verification

✓ How to automate regression

✓ How to stop regression on FAIL

◇ One-Line Interview Answer

“I use TCL to automate simulations, manage regressions, parse logs for errors and PASS/FAIL, and generate reports.”

🎯 Your Readiness

- Design + Verification background ✓
- TCL basics ✓
- Tool awareness ✓

10 Tricky TCL Interview Questions (VLSI Focus)

1 Why does `llength` not work on `get_cells` output?

Answer:

- `get_cells` returns a collection, not a TCL list.
- `llength` works only on TCL lists, while collections must be handled using `sizeof_collection`.

sizeof_collection [get_cells *]

2 What happens if you use `llength` on a large collection?

Answer:

It expands the collection into a list → slow execution + high memory usage → may crash the tool.

3 Why are collections faster than lists in EDA tools?

Answer:

Collections are **tool-managed handles**, not real lists.

They avoid copying large data and operate directly inside the tool database.

##4Differencebetween`foreach`and`foreach_in_collection`?

Answer:

- `foreach` → works on TCL lists
- `foreach_in_collection` → works on tool collections

```
foreach_in_collection c [get_cells *] {
   puts [get_object_name $c]
}
```

5 Why `{}` is preferred over `"` in if conditions?

Answer:

`{}` prevents double substitution and improves performance.

```
if {$a > 5} { puts "OK" }
```

6 What is double substitution in TCL?

Answer:

TCL first substitutes variables, then evaluates the expression again — causing bugs or slowdown.

7 How do you safely handle errors in TCL scripts?

Answer:

Using `catch`.

```
if {[catch {read_verilog top.v} err]} {
   puts "Error: $err"
}
```

8 Why is `expr {}` recommended instead of `expr ""`?

Answer:

Curly braces prevent re-evaluation and protect against unwanted variable expansion.

```
expr {$a + $b}
```

9 How do you stop a script when timing slack is negative?

Answer:

```
if {$slack < 0} {
   error "Timing violation found"
}
```

10 How do you parse a PrimeTime report and count violations?

Answer (Concept):

- Open report file
- Read line by line
- Use `regexp` to detect violations
- Increment counter

```
if {[regexp {VIOLATED} $line]} {
   incr count
}
```

🎯 What Interviewers Are Checking

✓ Tool awareness

✓ Performance understanding

✓ Real-world scripting

✓ Debugging ability

💡 **Golden Interview Line (Use This!)**

“I always use collections and `sizeof_collection` instead of lists when handling large design data to avoid performance issues.”

🧩 5 Real TCL Coding Interview Problems (VLSI Focus)

1 Count number of cells in a design

Problem:

Count total cells in a design using TCL (EDA tool).

☑ Solution

set cells [get_cells *]

puts "Total cells = [sizeof_collection $cells]"

🧠 Why interviewer likes this

- Shows collection handling
- Correct use of `sizeof_collection`

2 Print all cell names one by one

Problem:

Print each cell name from the design.

☑ Solution

```
foreach_in_collection c [get_cells *] {
   puts [get_object_name $c]
}
```

🧠 Concept tested

- `foreach_in_collection`
- Tool database access

3 Parse a log file and count ERRORs

Problem:

Count number of `ERROR` messages in a simulation log.

☑ Solution

```
set fp [open "sim.log" r]
set err_count 0

while {[gets $fp line] != -1} {
   if {[regexp {ERROR} $line]} {
      incr err_count
   }
}
close $fp
```

puts "Total Errors = $err_count"

🧠 Concept tested

- File handling
- `regexp`
- Loop + condition

4 Stop script if timing slack is negative

Problem:

Exit script if slack < 0.

☑ **Solution**

if {$slack < 0} {

    error "Timing violation detected!"

}

🧠 Used in

- PrimeTime signoff
- Automated flow stop

5 Check PASS / FAIL in regression

Problem:

Print FAIL if any test failed.

☑ Solution

set status PASS

```
if {[regexp {FAIL} $line]} {
   set status FAIL
}
puts "Regression Status: $status"
```

🧠 Verification relevance

- Regression summary
- Log parsing

🎯 How to Answer in Interview

If asked:

“Have you written TCL scripts?”

Say:

“Yes, I’ve written TCL scripts for automation like counting cells, parsing logs, detecting timing violations, and regression status generation.”

⭐ Bonus Tip

Never use:

```
llength [get_cells *]
```

❌ Interviewers hate this 😄

Always use:

```
sizeof_collection
```

☑

Top 25 BASIC TCL Interview Questions for Design & Verification Engineers:

👉 PLUS basic coding questions (very commonly asked) (VLSI-focused).

🔹 **Top 25 Basic TCL Interview Questions (Design & Verification)**

🔷 PART-A: Conceptual Questions (1–15)

1 What is TCL?

TCL is a scripting language used to automate EDA tools in VLSI.

2 Why is TCL used in VLSI?

For automation, flow control, report generation, and regression handling.

3 Is TCL case-sensitive?

Yes.

4 How do you declare a variable in TCL?

set a 10

5 How do you print a variable?

puts $a

6 How do you perform arithmetic in TCL?

expr {$a + $b}

7 Difference between `{}` and `""`?

- `{}` → no substitution
- `""` → variable substitution

8 What is a TCL list?

An ordered collection of elements.

set l {1 2 3}

9 How do you find list length?

llength $l

10 What is a collection in VLSI tools?

A tool-managed object returned by commands like `get_cells`.

11 Difference between list and collection?

List	Collection
Normal TCL	Tool managed
`llength`	`sizeof_collection`
Slow for big data	Fast

12 Why use `sizeof_collection`?

Because collections are not real lists.

13 What is `foreach` used for?

To iterate over lists.

14 What is `foreach_in_collection`?

Used to iterate over tool collections.

15 What is a procedure (`proc`)?

Reusable block of TCL code.

◇ PART-B: Control & Utility Questions (16–20)

16 How do you write if-else in TCL?

if {$a > 5} { puts OK } else { puts NO }

17 How do you read a file in TCL?

Using `open`, `gets`, `close`.

18 What is `regexp` used for?

Pattern matching in strings/logs.

19 What is `catch`?

Error handling in TCL scripts.

20 Where is TCL used in verification?

Regression control, log parsing, result summary.

◇ PART-C: Tool-Related Basics (21–25)

21 How do you get all cells in design?

get_cells *

22 How do you print cell names?

get_object_name

23 How do you stop script on error?

error "Message"

24 How do you count objects in a design?

sizeof_collection

25 Is TCL dynamically typed?

Yes, everything is string internally.

🧩 PART-D: Basic TCL Coding Questions (VERY IMPORTANT)

Coding Q1: Add two numbers

set a 5

set b 6

puts [expr {$a + $b}]

Output:

11

Coding Q2: Check even or odd

```
if {[expr {$n % 2}] == 0} {
    puts "Even"
```

```
} else {
   puts "Odd"
}
```

Output:

Coding Q3: Count cells

```
puts [sizeof_collection [get_cells *]]
```

Coding Q4: Parse log for ERROR

```
if {[regexp {ERROR} $line]} {
   incr count
}
```

Coding Q5: PASS / FAIL check

```
set status PASS
if {[regexp {FAIL} $line]} {
   set status FAIL
}
```

🎯 One-Line Interview Answer (Golden)

“I use TCL mainly for automating EDA tools, handling collections, parsing logs, and managing regressions in verification.”

✓ Basics strong

✓ VLSI-oriented understanding

✓ Ready for junior–mid interviews

◇ 5 Tricky TCL Debugging Questions (VLSI Focus)

1 Problem: Using `llength` on `get_cells` causes wrong results

```
set total_cells [llength [get_cells *]]
puts $total_cells
```

❌ Issue:

`get_cells` returns a tool collection, not a TCL list.

`llength` expands the collection → incorrect count or slow execution.

☑ Fix:

```
set total_cells [sizeof_collection [get_cells *]]
puts $total_cells
```

💡 **Interview Tip: Always use `sizeof_collection` for tool collections.**

2 Problem: Script fails when no cells match filter

```
set regs [get_cells -filter "is_sequential==1"]
puts [get_attribute $regs name]
```

❌ Issue:

If the filter returns empty collection, `get_attribute` fails.

☑ Fix:

```
set regs [get_cells -filter "is_sequential==1"]
if {[sizeof_collection $regs] > 0} {
```

```
    puts [get_attribute $regs name]
} else {
    puts "No sequential cells found"
}
```

💡 Lesson: Always check for empty collections.

3 Problem: Log parsing fails when line has extra spaces

```
if {[regexp {ERROR} $line]} {
    puts $line
}
```

❌ Issue:

`ERROR` may have spaces / different cases: `error`, `ERROR`, `Error`

☑️ Fix:

```
if {[regexp -nocase {\bERROR\b} $line]} {
    puts $line
}
```

💡 Lesson: Use `-nocase` and `\b` for robust parsing.

```
if {[regexp -nocase {\bERROR\b} $line]} {
```

👉 This line checks whether the word `ERROR` appears in the text stored in `line`.

Break it down:

- `regexp` → searches using a regular expression

- `-nocase` → ignores case
- matches `ERROR`, `error`, `Error`, etc.
- `\bERROR\b` → matches only the full word `ERROR`
- `\b` = word boundary
- avoids matching `ERRORS` or `MYERROR123`
- `$line` → the text being checked

If the pattern is found, the condition becomes true.

puts $line

👉 Prints the entire line that contains the word `ERROR`.

}

👉 Ends the `if` block.

🧠 One-line summary (interview-ready):

This code scans a line (case-insensitive) and prints it only if the exact word ERROR is found.

4 Problem: Arithmetic fails in if-condition

if {$a/$b > 1} {

puts "OK"

}

❌ Issue:

- Division of strings without `expr` causes error
- `$b` may be 0 → divide by zero

☑ Fix:

if {[expr {$b != 0 && $a/$b > 1}]} {

puts "OK"

}

💡 Lesson: Always use `expr` and safe checks.

5 Problem: Script prints nothing when using variables in loops

```
set tests {t1 t2 t3}
foreach test $tests {
   puts "Running $test"
}
```

❌ Issue:

- Variable name shadowing or typo may cause `$test` to be empty
- Example: using `{}` incorrectly

☑ Fix:

```
foreach test $tests {
   puts "Running $test"
}
```

Output:

Running t1

Running t2

Running t3

💡 **Lesson:**

- Avoid `{}` around loop variable in `foreach`
- Always test loops incrementally

🎯 Key Debugging Takeaways for Interviews

1. Check collection vs TCL list → `sizeof_collection`

2. Always handle empty collections

3. Use `regexp -nocase` for robust log parsing

4. Use `expr` for arithmetic

5. Test loops & variables step-by-step

10 common TCL coding questions with solutions, interview-oriented for Design & Verification.

1. 0 Print numbers from 1 to 10

Question: Write a TCL script to print numbers 1 to 10.

```
for {set i 1} {$i <= 10} {incr i} {
    puts $i
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

2 Check if a number is even or odd

```
set num 7
if {[expr {$num % 2 == 0}]} {
    puts "Even"
} else {
    puts "Odd"
}
```

Output:

Odd

3 Find length of a list

```
set cells {INV NAND NOR DFF}
puts [llength $cells]
```

Output:

4

4. 0 Loop through a list and print elements

```
set tests {test1 test2 test3}

foreach t $tests {
    puts $t
}
```

Output:

test1

test2

test3

5. 0 Read a file line by line

```
set fp [open "data.txt" r]


while {[gets $fp line] >= 0} {
    puts $line
}
close $fp
```

6. 0 Search for a word in a line (log parsing)

```
set line "ERROR: Simulation failed"


if {[regexp {ERROR} $line]} {
    puts "Error found"
}
```

Output:

Error found

7 Extract a number using `regexp`

```
set line "WNS : -0.45"
```

regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns

puts $wns

Output:

-0.45

🔹 Code

set line "WNS : -0.45"

☑ Creates a string variable `line`

It contains timing report text.

- Value:

"WNS : -0.45"

regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns

This is the main line. Let’s split it:

🔸 `regexp`

- Command used to search text using a pattern

🔸 `{WNS\s*:\s*(-?\d+\.\d+)}`

This is the pattern (regular expression):

Part	Meaning
`WNS`	Match literal text `WNS`
`\s*`	Any number of spaces
`:`	Match colon `:`
`\s*`	Spaces again

| `()` | Capture group |

| `-?` | Optional minus sign |

| `\d+` | One or more digits |

| `\.` | Decimal point |

| `\d+` | Digits after decimal |

👉 So it matches:

WNS : -0.45

🔸 `$line`

- Input string where regexp searches

🔸 `-> wns`

- `->` skips full match
- `wns` stores only the captured number.
- Result:

wns = -0.45

puts $wns

☑ Prints the extracted value

🖨 Output

-0.45

🎯 Interview One-Line Summary

- “This TCL code extracts the numeric WNS value from a timing report line using `regexp`.”

8 Count number of ERROR lines in a log

```
set count 0

while {[gets $fp line] >= 0} {
    if {[regexp {ERROR} $line]} {
        incr count
    }
}
puts "Total Errors = $count"
```

9 Simple procedure (function) example

```
proc add {a b} {
    return [expr {$a + $b}]
}
puts [add 5 3]
```

Output:

```
8
```

10 PASS / FAIL decision based on value

```
set wns -0.12

if {$wns >= 0} {
    puts "TIMING PASSED"
} else {
```

```
    puts "TIMING FAILED"
}
```

Output:

TIMING FAILED

🧠 Interview Tip

If asked “Are you comfortable with TCL coding?”

“Yes, I use TCL for loops, file handling, regexp-based log parsing, and automation in verification and STA flows.”

◇ 10 Advanced TCL Coding Questions with Solutions — focused on Design & Verification interviews.

1 Remove duplicate elements from a list

Question: Remove duplicates from a list.

```
set lst {INV NAND INV NOR NAND}
set uniq [lsort -unique $lst]
puts $uniq
```

Output:

INV NAND NOR

2 Count how many times each word appears in a log

```
set counts {}
```

```
while {[gets $fp line] >= 0} {
    foreach word $line {
        dict incr counts $word
    }
}
puts $counts
```

Output:

```
word1 3 word2 5 word3 1
```

3 Print only failing tests from multiple logs

```
foreach log [glob *.log] {
    set fp [open $log r]
    while {[gets $fp line] >= 0} {
        if {[regexp {FAIL} $line]} {
            puts "$log : FAILED"
            break
        }
    }
    close $fp
}
```

4 Extract multiple slack values from a report

```
set slacks {}
```

```
while {[gets $fp line] >= 0} {
    if {[regexp {slack.*(-?\d+\.\d+)} $line -> s]} {
        lappend slacks $s
    }
}
puts $slacks
```

Output:

5. 0 Find worst (minimum) slack

```
set wns [lindex [lsort -real $slacks] 0]
puts "WNS = $wns"
```

6 Procedure to check timing PASS / FAIL

```
proc check_timing {wns} {
    if {$wns >= 0} {
        return "PASS"
    } else {
        return "FAIL"
    }
}
puts [check_timing -0.12]
```

Output:

FAIL

7 Skip header lines while parsing a report

```
while {[gets $fp line] >= 0} {
    if {[regexp {^-+} $line]} {
        continue
    }
    puts $line
}
```

Explanation:

➞ Reads the file line by line using file pointer `fp`

➞ Stores each line in variable `line`

```
if {[regexp {^-+} $line]} {
```

➞ Checks if the line starts with dashes (`-----`)

➞ Such lines are usually headers or separators in reports

```
continue
```

➞ Skips the current line

➞ Moves directly to the next line of the file

```
puts $line
```

➞ Prints only the useful data lines (non-header lines)

```
}
```

➞ Ends the loop

🎯 Interview One-Liner

“This code skips header or separator lines in a report and processes only meaningful data lines.”

8 Run regression and stop on first failure

```
foreach t {test1 test2 test3} {
   exec vsim -c work.top +TEST=$t -do "run -all; quit" > $t.log
   if {[catch {exec grep FAIL $t.log}]} {
      continue
   } else {
      puts "$t FAILED — stopping regression"
      break
   }
}
```

9 Use `dict` to store report summary

```
dict set summary WNS -0.34
dict set summary TNS -12.4
dict set summary STATUS FAIL

puts $summary
```

Output:

WNS -0.34 TNS -12.4 STATUS FAIL

10 Extract numeric columns safely from space-variant data

set cols [regexp -all -inline {\S+} $line]

set value [lindex $cols 3]

puts $value

Output:

✓ \S+ → matches non-space characters

✓ Extracts the **4th column** (index starts from 0)

✓ Prints the extracted value

Use of `" "`, `{ }` and `[ ]` in TCL (with 5 simple examples)

Think of TCL like this 👇

👉 `[ ]` = execute command

👉 `" "` = substitute variables

👉 `{ }` = treat as literal (no substitution)

☑ 1 Square Brackets `[ ]` → Command Execution

TCL executes the command inside `[ ]` first.

Example-1

set a [expr {5 + 3}]

Explanation:

✓ `expr` runs first

✓ result `8` is stored in `a`

Example-2

puts [llength {a b c}]

✓ `llength` runs

✓ output = `3`

☑ 2 Double Quotes `" "` → Variable Substitution

Variables are expanded inside `" "`.

Example-3

set name TCL

puts "I am learning $name"

✓ Output → `I am learning TCL`

Example-4

set a 10

puts "$a is value"

✓ `$a` is replaced by `10`

☑ 3 Curly Braces `{ }` → No Substitution (Safe & Fast)

Everything inside `{ }` is treated as it is.

Example-5

set a 10

puts {$a is value}

✓ Output → `$a is value`

❌ No substitution

🔑 Most Important Interview Rule

Symbol	Meaning
`[ ]`	Execute command
`" "`	Substitute variables
`{ }`	Literal text (no substitution)

- Best Practice:

Use `{ }` in `if` and `expr` to **avoid bugs and improve speed.**

📘 20 Basic TCL Coding Problems (Interview Prep)

☑ 1.0 Print numbers from 1 to 10 using a loop

Using `for` loop

```
for {set i 1} {$i <= 10} {incr i} {

   puts $i

}
```

Explanation

- `set i 1` → start from 1
- `$i <= 10` → loop till 10
- `incr i` → increase `i` by 1
- `puts $i` → print number

⭐ Interview Tip

If interviewer asks why `{}` is used:

Curly braces prevent unnecessary variable substitution and make the loop faster and safer.

Print numbers from 1 to 10 using foreach in TCL

```
foreach i {1 2 3 4 5 6 7 8 9 10} {
   puts $i
}
```

☑ 2.0 Print even numbers between 1 to 20

Method-1: Using `for` loop (most preferred in interviews)

```
for {set i 1} {$i <= 20} {incr i} {
   if {[expr {$i % 2 == 0}]} {
      puts $i
   }
}
```

Explanation:

- Loop runs from 1 to 20
- `% 2 == 0` → checks even number
- `puts $i` prints only even values

☑ Method-2: Optimized way (best practice)

```
for {set i 2} {$i <= 20} {incr i 2} {
   puts $i
}
```

★ Why this is better

- No `if` condition
- Faster and cleaner
- Shows good TCL understanding in interview

One-Line Interview Answer

“I prefer incrementing by 2 to directly print even numbers instead of checking every number.”

3.0 To check whether a number is even or odd

```
set num 7


if {$num % 2 == 0} {
   puts "Even number"
} else {
   puts "Odd number"
}
```

Explanation:

- `set num 7` → number to check
- `$num % 2` → gets remainder after dividing by 2
- `== 0` → remainder zero means even

- otherwise → odd

4.0 To find the sum of first N natural numbers

🔹 **Using a loop**

```
set N 10
set sum 0

for {set i 1} {$i <= $N} {incr i} {
   set sum [expr $sum + $i]
}

puts "Sum = $sum"
```

🔸 **Explanation:**

- `set N 10` → number of terms
- `sum` starts from 0
- loop adds each number from 1 to N
- final sum is printed

🔹 **Interview tip** 💡

You can also do it without a loop:

```
set N 10
set sum [expr $N * ($N + 1) / 2]
puts "Sum = $sum"
```

5.0 To find the largest of three numbers

```
set a 10
```

```
set b 25
set c 15

if {$a >= $b && $a >= $c} {
    puts "Largest number is $a"
} elseif {$b >= $a && $b >= $c} {
    puts "Largest number is $b"
} else {
    puts "Largest number is $c"
}
```

Explanation:

- Compare `a`, `b`, and `c` using `if–elseif–else`
- `&&` means AND
- The greatest value gets printed

🔹 **Interview shortcut** 💡

Using `expr`:

```
set max [expr max($a, max($b, $c))]
puts "Largest number is $max"
```

6.0 To reverse a string

🔹 Using a loop

```
set str "HELLO"
set rev ""

for {set i [string length $str]} {$i > 0} {incr i -1} {
```

```
    append rev [string index $str [expr $i-1]]
}
puts "Reversed string: $rev"
```

Explanation:

- `string length` → gets string length
- loop runs from last character to first
- `string index` → picks each character
- `append` → builds reversed string

◇ Interview shortcut-

Using built-in command:

```
set str "HELLO"
puts [string reverse $str]
```

7.0 To count the number of words in a string.

```
set str "I love TCL scripting"
set word_count [llength $str]

puts "Number of words: $word_count"
```

Explanation:

- TCL treats a string with spaces as a list
- `llength` → counts list elements (words)
- Result = number of words

🔹 **Safer interview-friendly way-**

```
set str "  I   love   TCL scripting  "
set words [split $str]
set word_count [llength $words]

puts "Number of words: $word_count"
```

8.0 To find the length of a list.

```
set mylist {10 20 30 40 50}

set len [llength $mylist]
puts "Length of list = $len"
```

Explanation:

- `mylist` → a TCL list
- `llength` → counts number of elements in the list
- prints the list length

🔹 **Interview tip** 💡

Even strings can be treated as lists:

```
set data "A B C D"
puts [llength $data]
```

Output:

4

9.0 To print all elements of a list using `foreach`

```
set mylist {10 20 30 40 50}

foreach item $mylist {
```

```
    puts $item
}
```

Explanation:

- `set mylist {...}` → defines the list
- `foreach item $mylist` → loops through each element
- `puts $item` → prints each element

◇ Interview tip 💡

You can also use multiple variables:

```
set data {1 2 3 4}

foreach a b $data {
    puts "a=$a b=$b"
}
```

Output:

10.0 To remove duplicate elements from a list-

🔹 Using `lsort -unique` (recommended)

```
set data {10 20 30 20 10 40 30}

set unique_list [lsort -unique $data]
puts $unique_list
```

Explanation:

- `lsort` → sorts the list
- `-unique` → removes duplicate elements

- Result is a list with only unique values.

🔹 **Interview-friendly (without sorting)**

(Preserves original order)

```
set data {10 20 30 20 10 40 30}
set unique_list {}

foreach item $data {
    if {[lsearch -exact $unique_list $item] == -1} {
        lappend unique_list $item
    }
}

puts $unique_list
```

🔹 **Output**:

```
10 20 30 40
```

11.0 To check whether an element exists in a list-

```
set mylist {10 20 30 40 50}
set elem 30

if {[lsearch -exact $mylist $elem] != -1} {
    puts "Element exists in the list"
} else {
    puts "Element does not exist in the list"
}
```

Explanation:

- `lsearch` → searches an element in a list
- `-exact` → exact match
- returns index if found
- returns -1 if not found

◇ Interview shortcut 💡

Case-insensitive search:

lsearch -nocase $mylist $elem

12.0 To sort a list numerically-

◇ Numeric sorting

set mylist {10 2 25 4 1}

set sorted_list [lsort -integer $mylist]

puts $sorted_list

Explanation:

- `lsort` → sorts a list
- `-integer` → numeric (not string) sorting
- Result: `1 2 4 10 25`

◇ **Interview pitfall** ⚠

Without `-integer`, sorting is string-based:

lsort $mylist

Output: 1 10 2 25 4

🔹 **Useful options** 💡

- Descending order:

lsort -integer -decreasing $mylist

13.0 To read a file line by line and print it-

set fp [open "input.txt" r]

while {[gets $fp line] >= 0} {
    puts $line
}
close $fp

◇ Explanation:

- `open "input.txt" r` → opens file in read mode
- `gets` → reads one line at a time
- loop continues till end of file
- `puts` → prints each line
- `close` → closes the file

◇ **Interview tip** 💡

Check file existence safely:

if {[file exists "input.txt"]} {
    # read file
}

14.0 To count how many times a word appears in a file-

🔹 Example: Count occurrences of the word `"ERROR"`

```
set filename "input.txt"
set search_word "ERROR"
set count 0

set fp [open $filename r]

while {[gets $fp line] >= 0} {
    foreach word $line {
        if {$word eq $search_word} {
            incr count
        }
    }
}
close $fp
puts "Word '$search_word' appears $count times"
```

Explanation:

- Opens the file in read mode
- Reads file line by line
- Splits each line into words
- Compares each word with `search_word`
- Increments `count` when matched

🔹 **Interview-friendly (regex-based, more robust)** 💡

```
set count 0
```

```
set fp [open "input.txt" r]

while {[gets $fp line] >= 0} {
    incr count [regexp -all {\bERROR\b} $line]
}
close $fp

puts "ERROR count = $count"
```

◇ EDA use-case ❁

Very common for log analysis (ERROR, WARNING, FAIL).

15.0 To search for a word (like `ERROR`) in a log file-

🔹 Print all lines containing `ERROR`

```
set filename "log.txt"
set fp [open $filename r]
while {[gets $fp line] >= 0} {
    if {[regexp {\bERROR\b} $line]} {
        puts $line
    }
}
close $fp
```

Explanation:

- Opens the log file in read mode
- Reads file line by line
- `regexp` checks if the line contains `ERROR`

- Prints only matching lines

🔹 Interview / EDA-friendly version 💡

Case-insensitive search:

```
if {[regexp -nocase {\berror\b} $line]} {
   puts $line
}
```

🔹 Extra (count + print)

```
set count 0
if {[regexp {\bERROR\b} $line]} {
   puts $line
   incr count
}
```

This is very common in VLSI log/debug scripting (VCS, ICC, PrimeTime).

16.0 To extract a number from a string using `regexp`-

◇ Basic example

```
set str "Delay is 125 ns"

regexp {([0-9]+)} $str match number
puts "Extracted number: $number"
```

◇ Explanation:

- `{([0-9]+)}` → matches one or more digits
- `match` → stores the full matched text (optional)

- `number` → stores the captured digits
- `puts` → prints the extracted number

◇ Floating-point number extraction 💡

```
set str "Slack = -0.35 ns"
regexp {(-?[0-9]+\.?[0-9]*)} $str match value
puts $value
```

🔹 **EDA use-case** ⚙️

Very common for extracting:

- Slack values
- Timing numbers
- Counts from reports

17.0 To check if a file exists before opening it-

```
set filename "input.txt"

if {[file exists $filename]} {
   set fp [open $filename r]
   puts "File opened successfully"
   close $fp
} else {
   puts "File does not exist"
}
```

Explanation:

- `file exists` → checks whether the file is present

- If it exists → safely opens the file
- Else → prints an error message

◇ Interview / EDA best practice 💡

Also check read permission:

```
if {[file exists $filename] && [file readable $filename]} {
   set fp [open $filename r]
}
```

This is very common in VLSI scripts (log files, reports, netlists).

18.0 Write a simple procedure to add two numbers-

```
proc add_numbers {a b} {
   set sum [expr $a + $b]
   return $sum
}

puts [add_numbers 10 20]
```

Explanation:

- `proc add_numbers {a b}` → defines a procedure with two inputs
- `expr $a + $b` → performs addition
- `return` → sends result back
- `puts` → prints the returned value

🔹 Interview tip 💡

One-liner version:

```
proc add {a b} { expr {$a + $b} }
```

19.0 To print PASS / FAIL based on a value

◇ Basic example

```
set value 75

if {$value >= 50} {
   puts "PASS"
} else {
   puts "FAIL"
}
```

Explanation:

- `value` → input number
- `>= 50` → pass condition
- Prints PASS or FAIL

🔹 EDA / Verification-style example 💡

(Check status flag)

```
set status 1
if {$status} {
   puts "PASS"
} else {
   puts "FAIL"
}
```

◇ One-liner (interview friendly)

puts [expr {$value >= 50 ? "PASS" : "FAIL"}]

Very common in regression scripts & log checks.

20.0 To print only specific columns from space-separated data-

🔹 Example: Print 1st and 3rd columns

set line "10 20 30 40"

set fields [split $line]

puts "[lindex $fields 0] [lindex $fields 2]"

Explanation:

- `split` → splits the line by spaces into a list
- `lindex` → accesses specific column (index starts from 0)
- Prints selected columns only

🔹 From a file (real interview / EDA use-case) 💡

set fp [open "data.txt" r]

while {[gets $fp line] >= 0} {

    set f [split $line]

    puts "[lindex $f 0] [lindex $f 2]"

}

close $fp

🔹 Safer version (avoid index error)

```
if {[llength $f] >= 3} {
   puts "[lindex $f 0] [lindex $f 2]"
}
```

- This is very common in report parsing (timing, power, area).

☑ Print odd numbers between 1 to 20 (TCL)

Method-1: Using `for` loop + condition

```
for {set i 1} {$i <= 20} {incr i} {
   if {[expr {$i % 2 != 0}]} {
      puts $i
   }
}
```

Explanation

- Loop runs from 1 to 20
- `% 2 != 0` → checks odd number
- Prints only odd values

☑ Method-2: Optimized & Best Practice

```
for {set i 1} {$i <= 20} {incr i 2} {
   puts $i
}
```

💡 One-line interview answer

“I start from 1 and increment by 2 to directly print odd numbers.”

◇ Sum of ODD numbers between 1 and 20

☑ **Sum of odd numbers between 1 to 20 (TCL)**

Best & clean approach

```
set sum 0

for {set i 1} {$i <= 20} {incr i 2} {
    set sum [expr {$sum + $i}]
}
puts "Sum of odd numbers = $sum"
```

Explanation

- `set sum 0` → initialize sum
- Start loop from 1 (first odd number)
- `incr i 2` → jumps only through odd numbers
- Add each odd number to `sum`
- Print final result

🧠 What numbers are added?

`1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19`

➡ Result = 100

★ Alternate (using condition – less efficient)

```
set sum 0
for {set i 1} {$i <= 20} {incr i} {
    if {[expr {$i % 2 != 0}]} {
        set sum [expr {$sum + $i}]
```

```
    }
}
puts $sum
```

🎯 **Interview One-liner**

- “I start from the first odd number and increment by 2 to efficiently calculate the sum.”

To print numbers divisible by 3

🔹 **Example: from 1 to 30**

```
for {set i 1} {$i <= 30} {incr i} {
    if {$i % 3 == 0} {
        puts $i
    }
}
```

Explanation:

- Loop goes from 1 to 30
- `% 3 == 0` → number is divisible by 3
- Prints only those numbers

🔹 One-liner (interview style 💡)

```
for {set i 3} {$i <= 30} {incr i 3} {puts $i}
```

To check whether a number is prime or not

```
set num 17
set isPrime 1
```

```
if {$num < 2} {
    set isPrime 0
} else {
    for {set i 2} {$i <= [expr {sqrt($num)}]} {incr i} {
        if {$num % $i == 0} {
            set isPrime 0
            break
        }
    }
}

if {$isPrime} {
    puts "Prime number"
} else {
    puts "Not a prime number"
}
```

◇ **Explanation:**

- Numbers < 2 are not prime
- Try dividing `num` from 2 to √num
- If divisible → not prime
- Otherwise → prime

To check whether a number is a palindrome

◇ Basic palindrome number check

```
set num 121
set original $num
```

```
set rev 0

while {$num > 0} {
    set digit [expr {$num % 10}]
    set rev   [expr {$rev * 10 + $digit}]
    set num   [expr {$num / 10}]
}

if {$rev == $original} {
    puts "Palindrome number"
} else {
    puts "Not a palindrome number"
}
```

Explanation:

- Store original number
- Reverse the digits using `% 10`
- Compare reversed number with original
- Same → palindrome

◇ Shortcut (string method) 💡

```
set n "121"
if {$n eq [string reverse $n]} {
    puts "Palindrome number"
} else {
    puts "Not a palindrome number"
}
```

To count digits in a number

◇ Using loop (number method)

```
set num 12345
set count 0
while {$num > 0} {
   incr count
   set num [expr {$num / 10}]
}

puts "Number of digits = $count"
```

Explanation:

- Divide the number by 10 each time
- Count how many times until it becomes 0
- Count = number of digits

🔹 Shortcut (string method) 💡

```
set num 12345
puts "Number of digits = [string length $num]"
```

☑ Factorial of a number in TCL (Using `for` loop)

🔹 Example: **Factorial of 5**

```
set n 5
set fact 1
```

```
for {set i 1} {$i <= $n} {incr i} {
    set fact [expr {$fact * $i}]
}

puts "Factorial of $n is $fact"
```

Explanation

- `set n 5` → number whose factorial we want
- `set fact 1` → factorial starts from 1
- Loop runs from `1` to `n`
- Multiply `fact` by each number
- Print the final result

👉 Output:

```
Factorial of 5 is 120
```

☑ Alternate: Using `while` loop

```
set n 5
set fact 1
set i 1

while {$i <= $n} {
    set fact [expr {$fact * $i}]
    incr i
}
puts $fact
```

⭐ Interview One-Liner

“I calculate factorial by initializing result to 1 and multiplying it with numbers from 1 to N using a loop.”

🧠 Common Interview Follow-up

Q: What is factorial of 0?

A: `0! = 1`

```
set n 0
set fact 1
puts $fact
```

☑ Fibonacci Series in TCL (First N numbers)

Example: Print first 10 Fibonacci numbers

```
set n 10
set a 0
set b 1

puts $a
puts $b

for {set i 3} {$i <= $n} {incr i} {
    set c [expr {$a + $b}]
    puts $c
    set a $b
    set b $c
```

}

Explanation

- Fibonacci series starts with 0, 1
- Each next number = sum of previous two
- `a` → first number
- `b` → second number
- `c` → next Fibonacci number
- Values are shifted for next iteration

👉 Output:

0

1

1

2

3

5

8

13

21

34

☑ Alternate: Using `while` loop

set n 10

set a 0

set b 1

set count 1

while {$count <= $n} {

```
    puts $a
    set temp [expr {$a + $b}]
    set a $b
    set b $temp
    incr count
}
```

⭐ **Interview One-Liner**

“I generate Fibonacci numbers by storing the previous two values and repeatedly adding them inside a loop.”

🧠 **Common Interview Follow-ups**

Q: What are the first two Fibonacci numbers?

A: `0 and 1`

Q: Time complexity?

A: `O(n)`

☑ Reverse a Number in TCL

Example: Reverse 1234 → 4321

```
set num 1234
set rev 0

while {$num > 0} {
    set digit [expr {$num % 10}]
    set rev   [expr {$rev * 10 + $digit}]
```

```
    set num   [expr {$num / 10}]
}
puts "Reversed number = $rev"
```

Explanation

- `num % 10` → gets last digit
- `rev * 10 + digit` → builds reversed number
- `num / 10` → removes last digit
- Loop continues until number becomes `0`

👉 Output

```
Reversed number = 4321
```

⭐ Interview One-Liner

“I reverse a number by extracting digits using modulo and rebuilding the number inside a loop.”

🧠 Common Follow-up Questions

Q: What if number is `0`?

A: Output remains `0`

Q: Time complexity?

A: `O(n)` where `n` is number of digits

🔥 Bonus: Reverse a number using `for` loop

```
set num 5678
set rev 0
```

```
for {} {$num > 0} {} {
    set digit [expr {$num % 10}]
    set rev   [expr {$rev * 10 + $digit}]
    set num   [expr {$num / 10}]
}
puts $rev
```

☑ Check Palindrome String in TCL

Example: `madam`

```
set str "madam"
set rev ""

for {set i [expr {[string length $str] - 1}]} {$i >= 0} {incr i -1} {
    append rev [string index $str $i]
}

if {$str eq $rev} {
    puts "Palindrome string"
} else {
    puts "Not a palindrome"
}
```

Explanation

- `string length` → gets string length
- Loop runs from last character to first
- `string index` → extracts one character

- `append` → builds reversed string
- Compare original and reversed string

👉 Output

Palindrome string

⭐ **Interview One-Liner**

“I reverse the string character by character and compare it with the original string.”

🔥 Bonus: Case-insensitive Palindrome

set str "Madam"

set str [string tolower $str]

🧠 Common Interview Follow-ups

Q: How do you compare strings in TCL?

A: Using `eq` or `ne`

Q: Difference between `==` and `eq`?

A:

- `==` → numeric comparison
- `eq` → string comparison

☑ Remove spaces from a string in TCL

🔹 Example

Input string:

"VLSI TCL SCRIPT"

Output:

"VLSITCLSCRIPT"

☑ Method-1: Using `string map` (BEST & MOST COMMON)

set str "VLSI TCL SCRIPT"

set result [string map {" " ""} $str]

puts $result

Explanation

- `" "` → space character
- `""` → replace with nothing
- `string map` replaces all spaces at once

✓ Fast

✓ Clean

✓ Interview favorite

☑ Method-2: Using `regsub` (Very popular in log parsing)

set str "VLSI TCL SCRIPT"

regsub -all {\s+} $str "" result

puts $result

Explanation

- `\s+` → matches one or more spaces
- `-all` → replaces all occurrences
- Stores output in `result`

✓ Handles multiple spaces

✓ Used heavily in EDA log parsing

⭐ Interview One-Liner

“I remove spaces using `string map` or `regsub -all` since they are faster and cleaner than loops.”

🧠 Which one should YOU say in interview?

Say this 👇

✓ `string map` for simple replacement

✓ `regsub` for complex patterns or logs

Count vowels in a string using TCL.

☑ Count vowels in a string (TCL)

🔸 Example string

"VLSI Automation"

☑ Method-1: Using loop + condition (BEST for interviews)

set str "VLSI Automation"

set str [string tolower $str]

set count 0

foreach ch [split $str ""] {

```
    if {$ch eq "a" || $ch eq "e" || $ch eq "i" || $ch eq "o" || $ch eq "u"} {
        incr count
    }
}
puts "Vowel count = $count"
```

Explanation

- Convert string to lowercase (handle `A` and `a`)
- Split string into characters
- Check if character is a vowel
- Increase count

👉 **Output**

Vowel count = 6

☑ Method-2: Using `regexp` (Clean & Powerful)

```
set str "VLSI Automation"
set vowels [regexp -all -inline -nocase {[aeiou]} $str]
puts "Vowel count = [llength $vowels]"
```

🔍 Why interviewers like this-

✓ Uses `regexp`

✓ Short and efficient

✓ Great for log/string parsing

⭐ Interview One-Liner-

“I count vowels either by looping through characters or by using `regexp -all` and counting matches.”

To count vowels in a string-

🔹 Using loop + regexp

```
set str "Hello World"
set count 0

foreach ch [split $str ""] {
   if {[regexp -nocase {[aeiou]} $ch]} {
      incr count
   }
}

puts "Number of vowels = $count"
```

Explanation:

- `split $str ""` → splits string into characters
- `regexp` → checks vowel (a, e, i, o, u)
- `-nocase` → ignores case
- `incr` → increments count

🔹 One-line interview trick 💡

```
regexp -all -nocase {[aeiou]} $str count
puts "Vowels = $count"
```

Output:

Number of vowels = 3

Extract numbers from a string in TCL

☑ Extract numbers from a string (TCL)

Example string

set str "Cell U12 has delay 45 ns and power 120 mW"

☑ Method-1: Using `regexp` (BEST & MOST USED)

set numbers [regexp -all -inline {\d+} $str]

puts $numbers

Explanation

- `\d+` → matches one or more digits
- `-all` → find all numbers
- `-inline` → return matches as a list

👉 Output

12 45 120

- Interview favorite method-

☑ Method-2: Extract only the FIRST number

regexp {\d+} $str match

puts $match

👉 Output

12

☑ Method-3: Extract numbers with decimal values

```
set str "Slack is -0.25 ns and delay is 3.5 ns"
set nums [regexp -all -inline {-?\d+\.?\d*} $str]
puts $nums
```

👉 Output

-0.25 3.5

- Very useful in PrimeTime timing reports.

☑ Method-4: Sum all numbers extracted

```
set sum 0
foreach n [regexp -all -inline {\d+} $str] {
   incr sum $n
}
puts "Sum = $sum"
```

⭐ Interview One-Liner

“I extract numbers from strings using `regexp -all -inline` since it’s fast and ideal for parsing VLSI reports and logs.”

VERY common basic coding question that interviewers love (especially for TCL / scripting / VLSI roles):

🔹 **Reverse each word in a string**

👉 “Hello World” → “olleH dlroW”

☑ TCL Solution

```
set str "Hello World"
set result ""

foreach word $str {
    append result "[string reverse $word] "
}
puts [string trim $result]
```

◇ Explanation:

- TCL treats string as a list of words
- `foreach word $str` → picks one word at a time
- `string reverse` → reverses each word
- `append` → builds final string

◇ Why interviewers ask this?

- Tests loops
- Tests string handling
- Tests TCL list behavior

Count the frequency of each word in a file (or string)

☑ TCL Solution (string version)

```
set str "apple banana apple orange banana apple"
set counts {}

foreach word $str {
```

```
    dict incr counts $word
}
puts $counts
```

◇ Output:

```
apple 3 banana 2 orange 1
```

Explanation:

- `dict` → key-value storage
- `dict incr` → increments count automatically
- Very efficient and clean

◇ File version (real interview use-case)

```
set fp [open "input.txt" r]
set counts {}

while {[gets $fp line] >= 0} {
    foreach word $line {
        dict incr counts $word
    }
}
close $fp
puts $counts
```

🔹 **Why interviewers love this?**

✓ Uses loop + dict

✓ Real log/report parsing

✓ Shows TCL maturity

◇ Problem: Parse a timing report and print only the paths with negative slack:-

☑ Sample report lines

Path 1 : Slack = 0.23

Path 2 : Slack = -0.15

Path 3 : Slack = 1.02

Path 4 : Slack = -0.08

☑ TCL Solution

```
set fp [open "timing.rpt" r]

while {[gets $fp line] >= 0} {
   if {[regexp {Slack\s*=\s*(-[0-9.]+)} $line match slack]} {
      puts "VIOLATION: $line"
   }
}
close $fp
```

Explanation:

- Reads report line by line
- `regexp` extracts negative slack
- If slack is negative → timing violation
- Prints only violating paths

◇ Output

VIOLATION: Path 2 : Slack = -0.15

VIOLATION: Path 4 : Slack = -0.08

◇ Count number of lines in a file

☑ Simple TCL Solution

```
set filename "input.txt"
set count 0

set fp [open $filename r]

while {[gets $fp line] >= 0} {
   incr count
}
close $fp
puts "Total lines = $count"
```

Explanation:

- `open` → opens file in read mode
- `gets` → reads one line at a time
- Loop runs until EOF
- `incr count` → counts each line

◆ Interview-safe version (check file first) 💡

```
set filename "input.txt"
set count 0

if {[file exists $filename]} {
   set fp [open $filename r]
   while {[gets $fp line] >= 0} {
```

```
        incr count
    }
    close $fp
    puts "Total lines = $count"
} else {
    puts "File not found"
}
```

Count frequency of duplicate elements in a list (TCL)

☑ Problem

Given a list, print only the elements that appear more than once along with their count.

☑ TCL Solution (clean & interview-preferred)

```
set data {10 20 30 20 40 10 50 30 10}
set freq {}

# Count frequency
foreach item $data {
    dict incr freq $item
}

# Print only duplicates
foreach {key value} $freq {
    if {$value > 1} {
        puts "$key appears $value times"
    }
}
```

◇ Output

10 appears 3 times

20 appears 2 times

30 appears 2 times

Explanation:

- `dict incr` → automatically counts occurrences
- Second loop filters only duplicates (`value > 1`)
- Prints element + frequency

◇ Sum of digits of a number (TCL)

☑ Example: `1234 → 1 + 2 + 3 + 4 = 10`

🔹 Solution (number method – interview preferred)

```
set num 1234
set sum 0

while {$num > 0} {
   set digit [expr {$num % 10}]
   set sum   [expr {$sum + $digit}]
   set num   [expr {$num / 10}]
}
puts "Sum of digits = $sum"
```

◇ Explanation:

- `% 10` → extracts last digit
- `/ 10` → removes last digit
- Add digits one by one
- Loop continues until number becomes 0

◇ Shortcut method (string-based)-

```
set num "1234"
set sum 0

foreach d [split $num ""] {
    incr sum $d
}

puts "Sum of digits = $sum"
```

◇ Why interviewers ask this

✓ Loops

✓ Arithmetic ops

✓ Logic clarity

Armstrong number check-

◆ What is an Armstrong number?

A number is Armstrong if

sum of its digits raised to the power of number of digits = the number itself

👉 Example: 153

[1^3 + 5^3 + 3^3 = 153]

☑ TCL Program: Armstrong Number Check

```
set num 153
```

```
set original $num
set sum 0

# Count digits
set digits [string length $num]

# Calculate Armstrong sum
while {$num > 0} {
    set digit [expr {$num % 10}]
    set sum   [expr {$sum + pow($digit, $digits)}]
    set num   [expr {$num / 10}]
}

# Check result
if {$sum == $original} {
    puts "Armstrong number"
} else {
    puts "Not an Armstrong number"
}
```

Explanation:

- Count number of digits
- Extract each digit using `% 10`
- Raise digit to `digits` power
- Add all values
- Compare with original number

◇ Output

Armstrong number

Perfect Number Check-

🔸 What is a Perfect Number?

A number is perfect if the sum of its proper divisors (excluding the number itself) equals the number.

👉 Example: 6

Divisors: `1, 2, 3` → `1 + 2 + 3 = 6` ☑

☑ TCL Program: Perfect Number Check

```
set num 28
set sum 0

for {set i 1} {$i <= $num/2} {incr i} {
    if {$num % $i == 0} {
        incr sum $i
    }
}

if {$sum == $num} {
    puts "Perfect number"
} else {
    puts "Not a perfect number"
}
```

Explanation:

- Proper divisors are always ≤ num/2
- Check each number from `1` to `num/2`
- Add divisors
- Compare sum with original number

◇ Output

Perfect number

◇ Interview follow-up 💡

Print perfect numbers in a range:

```
for {set n 1} {$n <= 1000} {incr n} {
   set sum 0
   for {set i 1} {$i <= $n/2} {incr i} {
      if {$n % $i == 0} {
         incr sum $i
      }
   }
   if {$sum == $n} {
      puts $n
   }
}
```

Decimal to Binary Conversion — very common in interviews and VLSI scripting.

◇ Decimal to Binary Conversion in TCL

☑ TCL Program (using loop)

```
set num 23
set binary ""
set n $num
while {$n > 0} {
    set rem [expr {$n % 2}]
    set binary "$rem$binary"
    set n [expr {$n / 2}]
}
puts "Decimal $num in binary = $binary"
```

Explanation:

- `% 2` → remainder → binary digit
- Divide number by 2 repeatedly
- Prepend each remainder to form binary string
- Result: binary representation

◇ Output

```
Decimal 23 in binary = 10111
```

◇ Shortcut (string-based / built-in method)

```
puts [format "%b" 23]
# Output: 10111
```

LCM (Least Common Multiple) of two numbers — very common in interviews and coding tests.

◆ LCM Using GCD (Recommended Method)

☑ TCL Program

```
# Function to calculate GCD
proc gcd {a b} {
    while {$b != 0} {
        set temp $b
        set b [expr {$a % $b}]
        set a $temp
    }
    return $a
}
# Numbers
set a 12
set b 18

# LCM formula: LCM(a, b) = (a * b) / GCD(a, b)
set lcm [expr {($a * $b) / [gcd $a $b]}]
puts "LCM of $a and $b = $lcm"
```

◇ Explanation:

- First, calculate GCD using Euclidean algorithm
- Use formula: LCM = (a × b) / GCD(a, b)
- Prints the least common multiple

◇ Output

LCM of 12 and 18 = 36

Basic of TCL Tool Command Language:-

TCL Lecture-1: printing and setting variables



Basics of TCL
Tool Command Language

Amit Bisaria

TCL Basics

The first program

```
puts "hello world"; #puts is for printing
                    #in command line
```

Without semicolon, we cannot add comment in same line. It will be syntax error

Output:

```
hello world
```

TCL Basics

Semicolon (;) is not required in one single line of statement.

```
puts "hello world";
puts "hello world without semicolon"
```

Output:

```
hello world
hello world without semicolon
```

TCL Basics

Semicolon (;) is significant while writing two commands in single line.

```
puts "hello world"; puts "hello world without semicolon"
#above statement is valid


puts "hello world" puts "hello world with semicolon";
#above statement is not valid and will throw syntax error
```

Semicolon marks the end of statement.

TCL Basics

Semicolon (;) is also significant if you want to put comment after the end of statement.

```
puts "hello world"     #comment
```

Above statement will throw syntax error

```
puts "hello world";    #comment after semicolon
```

Above statement is correct

Variable declaration

Set command is used for declaring variables.

Tcl compiler doesn't require to define data-type of variable.

Syntax for declaring variable is:

```
set variable_name
```

Variable declaration

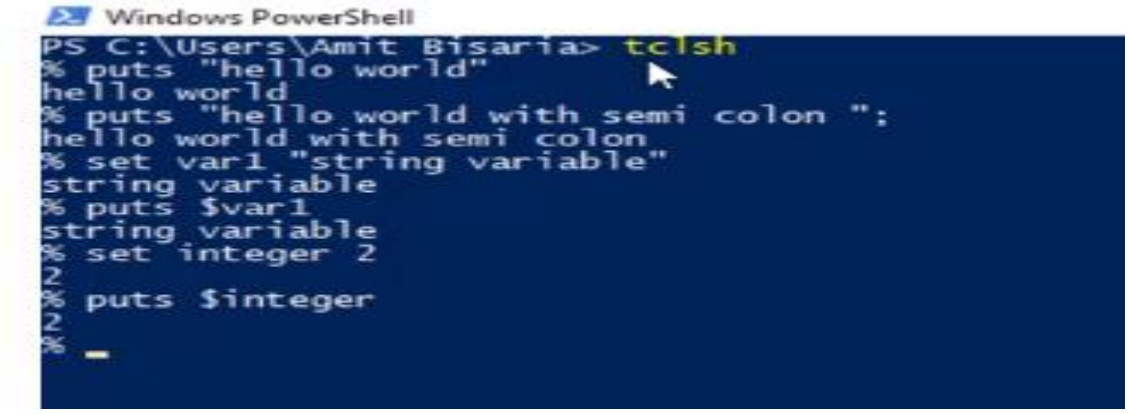
Input:

```
set str "this is str";
set value 1.5;
puts "$str";
puts "the value is $value";
```

Output:

```
this is str
this is value: 1.5
```

Dollar '$' is used as prefix with variable to print variable value in command line.



TCL Lecture2: command interpolation | square and curly braces

Square Braces

Square braces "[ ]" are used for assigning value to variables from other expressions.

Following is one example

```
set a "5";
set b $a;            #this statement will assign b to a
```

- Below statement will also assign b to a but in single statement
  `set b [set a "5"];`

- It will take statement inside square braces, evaluate it, and, assign output of it to the variable b.

Curly Braces

Braces can also be used for printing in command line. Syntax is as follows:

```
puts "str in double quotes"
puts {str in curly braces}
```

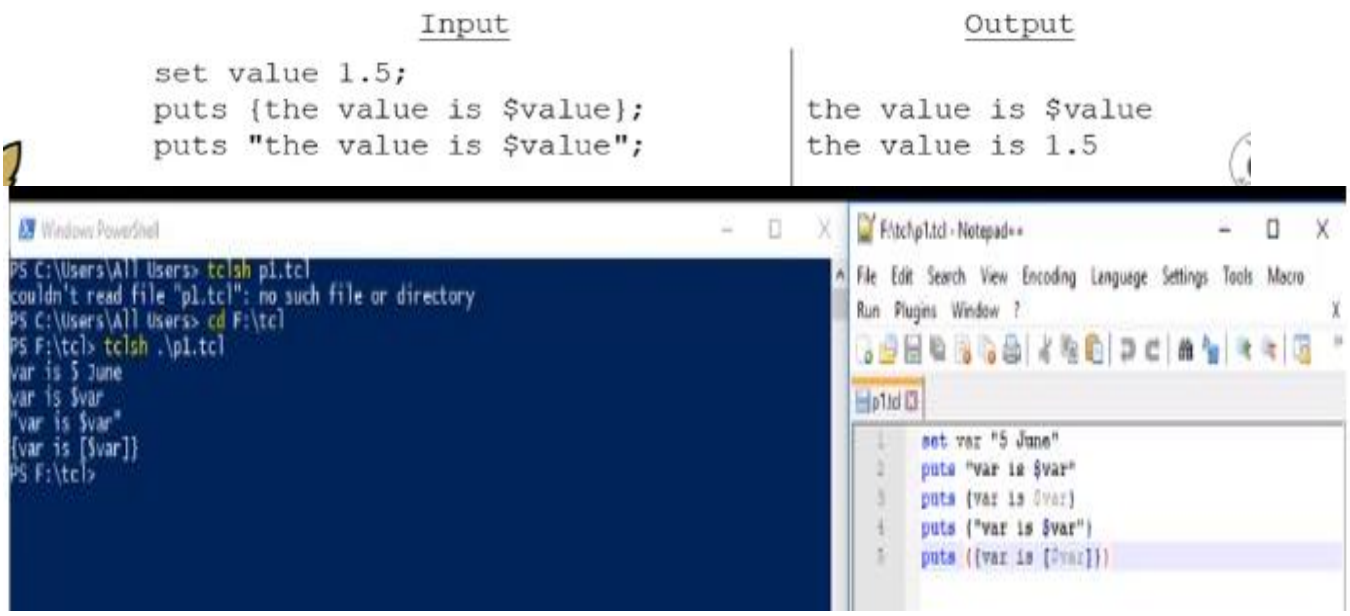
Both of the above statements will be printing the strings in command line.

```
str in double quotes
str in curly braces
```

Curly Braces

Why the curly brace '{' is required when double quote is already there for printing ?

Curly braces doesn't recognize variables. It prints all the $variables as it is.



TCL Lecture3: Arithmetic and logical operations | expr statements:

Arithmetic Operations

Expr Statement:

Expr is used to evaluate the expression.

Syntax:

Expr variable1 operator variable2

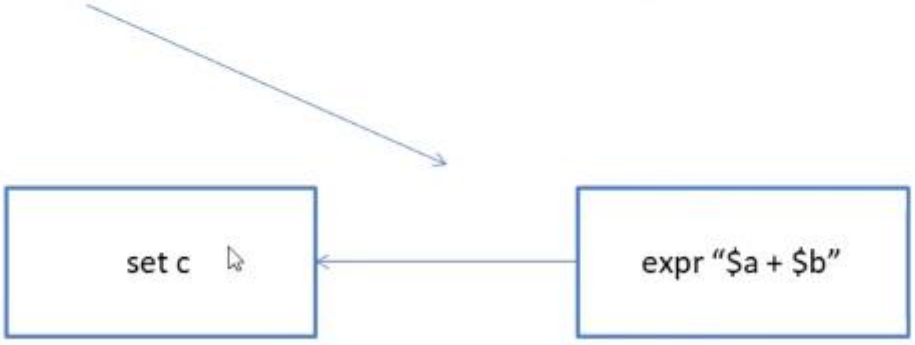
```
set a "5";
set b "3";
set c [expr "$a + $b"];
set d [expr "$a - $b"];
set e [expr "$a * $b"];
puts "addition of a and b is $c";
puts "subtraction of a and b is $d";
puts "multiply of a and b is $e";
```

```
addition of a and b is 8
subtraction of a and b is 2
multiply of a and b is 15
```

Arithmetic Operations

Following is one more example for square braces and expr put together

```
set a "5";
set b "3";
set c expr "$a + $b";        #this statement will throw error
set c [expr "$a + $b"];      #this statement is correct
```



"[]" is used to put output of expr statement into variable c

Arithmetic Operations

Expr Statement:

```
expr 10+5      ->      15
expr 10.0+5    ->      15.0
expr 12/7      ->      1
expr 12/7+3.0  ->      4.0
expr 12.0/7+3.0        ->      4.71428
```

Integers are used until a floating point number is introduced, after which floating–point number is used.

Logical Operations

variable A holds 1
variable B holds 0

Operator	Description	Example
&&	Logical AND operator	(A && B) is false.
||	Logical OR operator	(A || B) is true.
!	Logical NOT operator	!(A && B) is true.

Bitwise Operations

Bitwise operator works on bits and perform bit-by-bit operation.

& = Bitwise AND
| = Bitwise OR
^ = Bitwise XOR

Assume if A = 60; and B = 13. Now, in binary format they will be as follows –

A = 0011 1100
B = 0000 1101

A&B = 0000 1100
A|B = 0011 1101
A^B = 0011 0001

Bitwise Operations

Bitwise operator works on bits and perform bit-by-bit operation.

& = Bitwise AND
| = Bitwise OR
^ = Bitwise XOR

A = 0011 1100
B = 0000 1101

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12, which is 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) will give 61, which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49, which is 0011 0001
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240, which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15, which is 0000 1111

TCL Lecture 4: Loops in tcl | looping syntax with example

Conditional Statement and Loop: -

If Condition

If condition :

The syntax is:

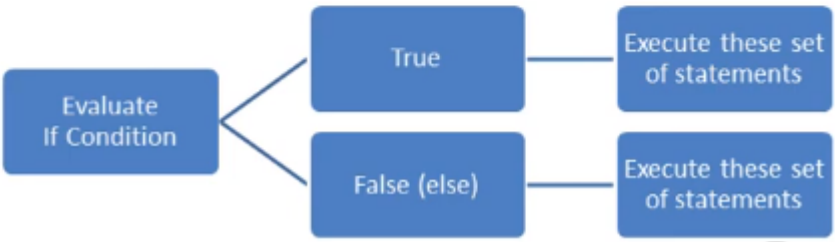
```
if expr1 ?then? body1 elseif expr2 ?then? body2 elseif ... ?else? ?bodyN?
```

```
set x 1;

if {$x == 2} {puts "x is 2"} else {puts "x is not 2"}
```

Output

```
x is not 2
```

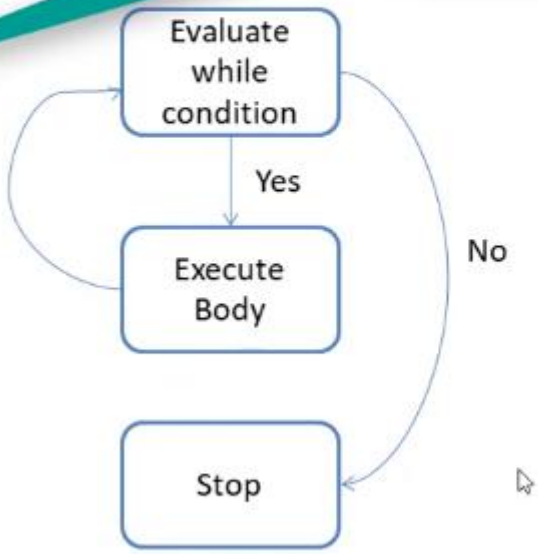


While Loop

note: opening curly brace of body should be in same line of while loop or else it will error out

While Loop:

```
# while loop.
set x 0;
while {$x < 6} {
    puts "x is $x";
    set x [expr $x + 1]
    }
```



Output:

```
x is 0
x is 1
x is 2
x is 3
x is 4
x is 5
```

For Loop

For loop syntax is bit different from while loop.

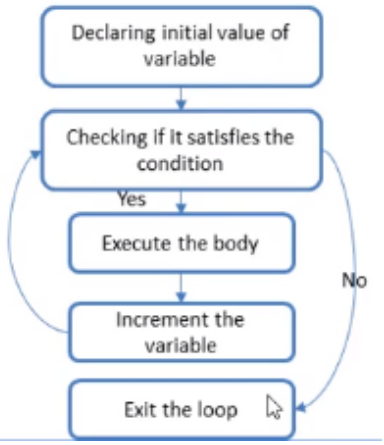
```
for {initialization} {condition} {increment} {
   statement(s);
}
```

Example:

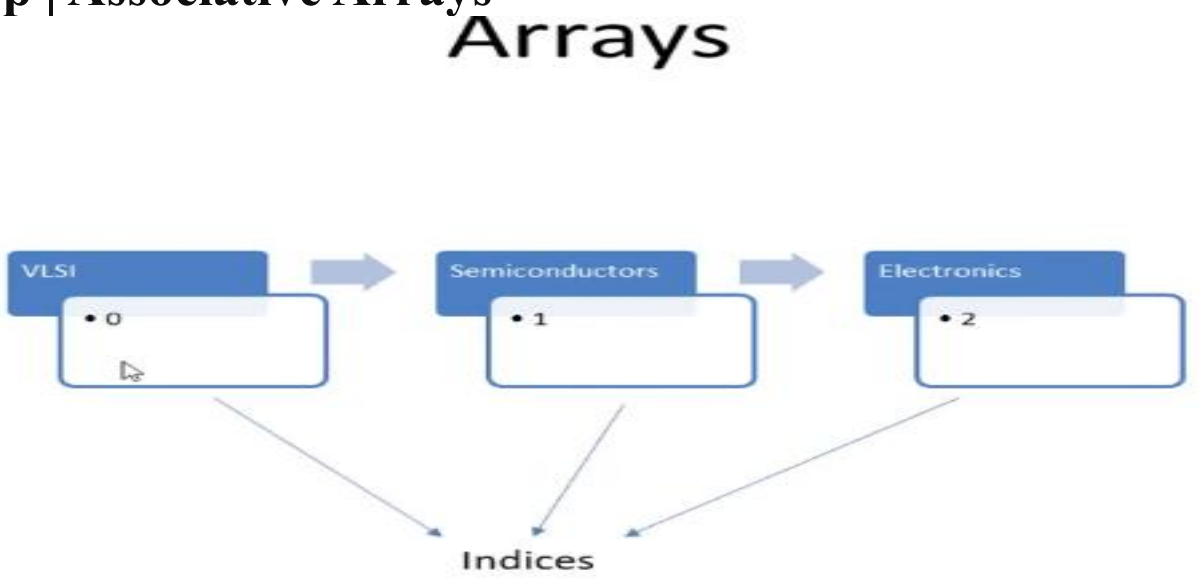
```
for {puts "Start"; set i 0} {$i < 2} {incr i; puts "I after incr: $i"; } {
  puts "I inside first loop: $i"
  }
```

Output:

```
Start
I inside first loop: 0
I after incr: 1
I inside first loop: 1
I after incr: 2
```



TCL Lecture 5 : Arrays and Loops | for loop and foreach loop | Associative Arrays



Arrays

Array Decalaration: Arrays are ordered set of values

The syntax is:

```
set ArrayName(Index) value
```

```
set institute(0) VLSI
set institute(1) Academy
set institute(2) India

puts $institute(0)
puts $institute(1)
puts $institute(2)
```

Index	Value
0	VLSI
1	Academy
2	India

Output:

```
VLSI
Academy
India
```

Arrays

Iterating over arrays: one can iterate/loop over arrays in tcl

```
set institute(0) VLSI
set institute(1) Academy
set institute(2) India

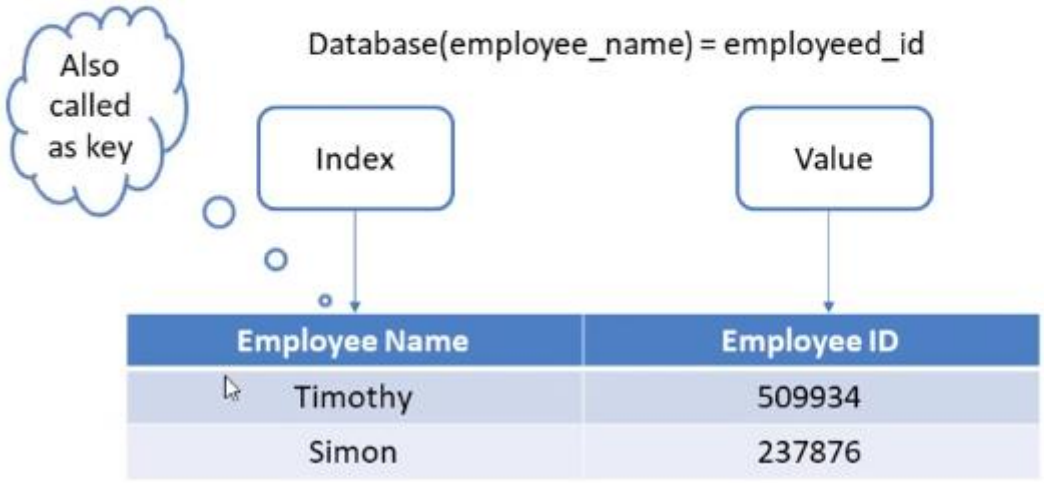
for {set index 0} {$index < [array size institute]} {incr index} {
   puts "institute($index) : $institute($index)"
}
```

Output:

```
institute(0) : VLSI
institute(1) : Academy
institute(2) : India
```

Associative Arrays

Associative Arrays:



Employee Name	Employee ID
Timothy	509934
Simon	237876

Associative Arrays

Associative Arrays: Also called as dictionaries.

- Associative arrays are unordered set of values which can be accessed with corresponding keys.
- Associative arrays have an index that is not necessarily a number, and can be sparsely populated.
- Example:

```
set Employee1(Name) "Jacob"
set Employee1(Age) 24
puts $Employee1(Name)
puts $Employee1(Age)
```

Associative Arrays

Associative Arrays:

- Syntax of retrieving indices, is as follows:

```
[array names arrayName]
```

- Example:

```
set Employee1(Name) "Latha"
set Employee1(Age) 24
puts [array names Employee1]
```

- Output:

```
Age Name
```

Foreach Loop

Foreach Loop: foreach is more useful in the event of *associative arrays*.

```
set institute(0) VLSI
set institute(1) Academy
set institute(2) India

foreach index [array names institute] {
puts "institute($index):$institute($index)"
}
```

This will be expanded to all keys present in "institute" array

Output:

```
institute(0):VLSI
institute(1):Academy
institute(2):India
```

TCL Lecture 6 : Programming in tcl | for loop and foreach loop | Associative Arrays: -

```
#program to find a given element in array in tcl
#program name: element_finding_in_array
#array of continous indices

set array1(0) a
set array1(1) b
set array1(2) c
set array1(3) d
set array1(4) e

set element_to_find d

foreach number [array names array1] {
    if {$array1($number) == $element_to_find} {
        puts "found: $array1($number)"
        }
    }
```

```
#associative array
#students(name) - age

set students(jacob) 24
set students(ryan) 27
set students(callie) 27
set students(john) 29
set students(yang) 21

set name_to_find callie

foreach name [array names students] {
    if { $name == $name_to_find } {
        puts "Name : $name "
        puts "age : $students($name)"
        }
```

TCL lecture7: exec command and escape sequences | executing shell command in tcl

Basics of TCL
Tool Command Language

executing shell commands and escape sequences

exec command

- It can be used to execute shell commands

- Example:

```
exec ls              #lists all files in directory
exec ls *.tcl        #lists all .tcl files in directory
```

```
PS F:\tcl> ls


    Directory: F:\tcl


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         6/25/2018   1:19 AM            237 6th lecture.txt
-a----         6/21/2018   2:31 AM            286 array_finder.tcl
-a----         6/24/2018   9:55 PM            329 associative.tcl
-a----         6/24/2018   9:21 PM            360 element_finding_in_arrray.tcl
-a----         7/12/2018   2:49 PM              0 escape_sequences.tcl
-a----         6/24/2018   9:52 PM            326 hash_finder.tcl
-a----          6/6/2018   2:13 AM            102 p1.tcl


PS F:\tcl>
```

```
PS F:\tcl> tclsh
% exec ls
6th lecture.txt
array_finder.tcl
associative.tcl
element_finding_in_arrray.tcl
escape_sequences.tcl
hash_finder.tcl
p1.tcl
%
```

Windows PowerShell

```
% exec ls associative*
associative.tcl
% clear
```

Windows PowerShell

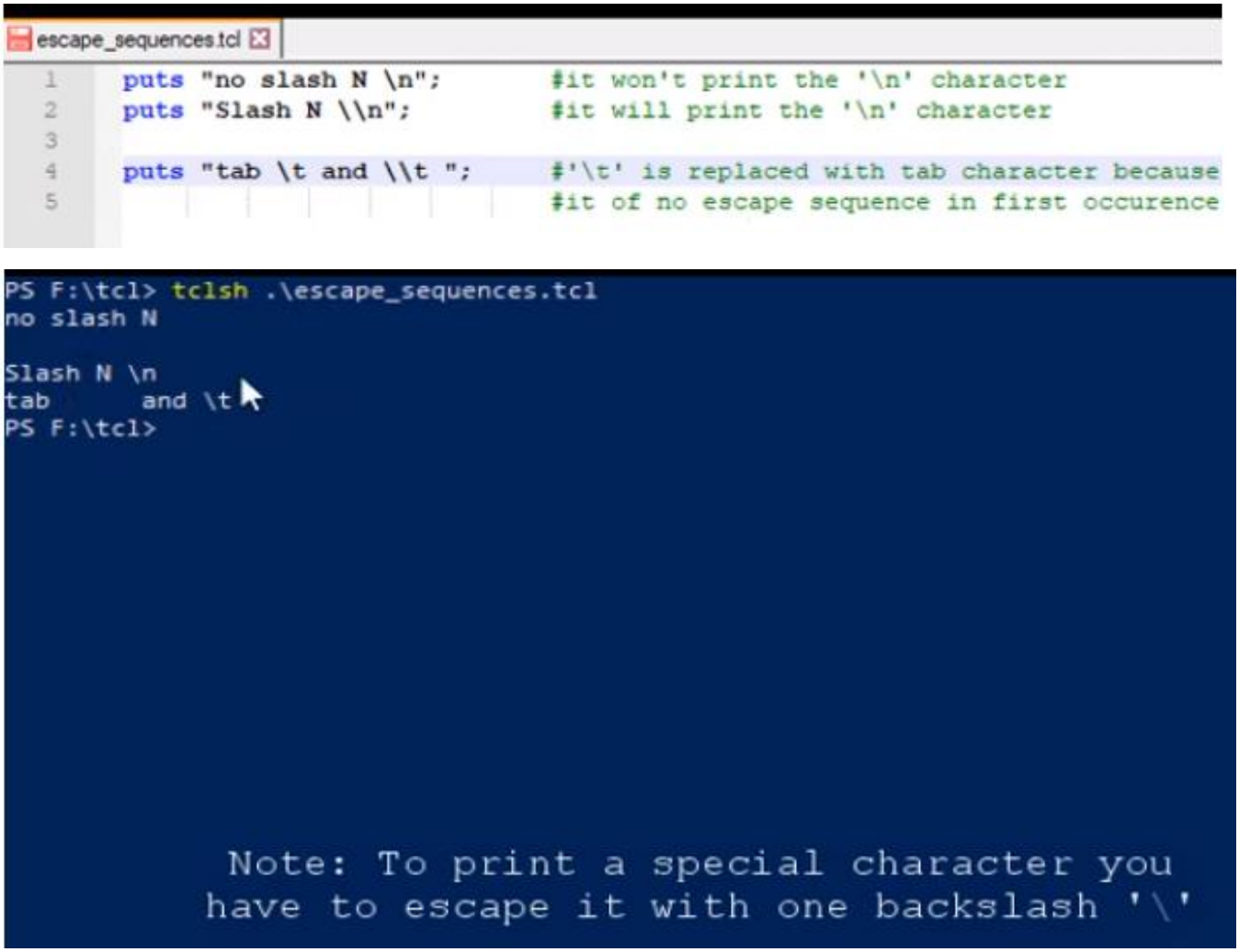
```
% exec ls -ltr
total 6
-rwxrwx---+ 1 Amit Bisaria None 102 Jun  6 02:13 p1.tcl
-rwxrwx---+ 1 Amit Bisaria None 286 Jun 21 02:31 array_finder.tcl
-rwxrwx---+ 1 Amit Bisaria None 360 Jun 24 21:21 element_finding_in_arrray.tcl
-rwxrwx---+ 1 Amit Bisaria None 326 Jun 24 21:52 hash_finder.tcl
-rwxrwx---+ 1 Amit Bisaria None 329 Jun 24 21:55 associative.tcl
-rwxrwx---+ 1 Amit Bisaria None 237 Jun 25 01:19 6th lecture.txt
-rwxrwx---+ 1 Amit Bisaria None   0 Jul 12 14:49 escape_sequences.tcl
%
```

escape_sequences.tcl

```
1  puts "no slash N \n";        #it won't print the '\n' character
2  puts "Slash N \\n";      #it will print the '\n' character
```

```
PS F:\tcl> tclsh .\escape_sequences.tcl
no slash N

Slash N \n
PS F:\tcl>
```



◆ What are escape sequences?

Escape sequences are special character combinations (start with `\`) used to represent characters that can't be typed directly or have special meaning.

◇ Common escape sequences in TCL

Escape	Meaning	Example
`\n`	New line	`puts "Hello\nWorld"`
`\t`	Tab space	`puts "Name\tDelay"`
`\\`	Backslash `\`	`puts "C:\\tools"`

| `\"` | Double quote `"` | `puts "He said \"Hi\""` |

| `\r` | Carriage return | `puts "ABC\r12"` |

◇ Examples in TCL

1 New line

puts "Hello\nTCL"

Output:

Hello

TCL

2 Tab space

puts "Name\tDelay\tLoad"

Output:

Name Delay Load

3 Print double quotes

puts "This is a \"TCL\" script"

Output:

This is a "TCL" script

4 Backslash

puts "Path: C:\\EDA\\tools"

Output:

Path: C:\EDA\tools

◇ Important note (very common interview point)

- Escape sequences work only inside double quotes `" "`
- Inside `{ }` (braces), escape sequences are NOT expanded

Example:

puts "Hello\nWorld" ;# works

puts {Hello\nWorld} ;# prints \n as text

◇ One-line interview definition

“Escape sequences in TCL represent special characters using backslash notation and work inside double-quoted strings.”

TCL lecture8: string operations in tcl

String commands

- **string compare** *string1 string2*

Compares string1 and string2 lexographically. Returns 0 if equal, -1 if string1 comes before string2, else 1.

```
puts [string compare "Golden" "Light"]        Output: -1
```

- **string index** *string index*

Returns the character at index

```
puts [string index "Timing Path" 4]           Output: n
```

- **string length** *string*

Returns length of string

```
puts [string length "Timing Path"]            output: 11
```

String commands

- **string range** *string index1 index2*

Return the range of characters in string from index1 to index2

```
puts [string range "I am studying physical design" 2 12]
```

output: `am studying`

- **string** *tolower string*

Returns the lowercase string

```
puts [string tolower "VLSI DESIGN"]
Output: vlsi design
```

- **string** *toupper string*

Returns the uppercase string

```
puts [string toupper "vlsi design"]
Output: VLSI DESIGN
```

String commands

- **trimright string ?trimcharacters?**

Removes trimcharacters in left end of string. The default trimcharacters is whitespace.

```
set s1 "VLSI Academy India Pvt Ltd"
set s2 "Pvt Ltd"
puts "Trim right: \"$s2\" in \"$s1\""
puts "new string: \"[string trimright $s1 $s2]\""
```

Output:

```
Trim right: "Pvt Ltd" in "VLSI Academy India Pvt Ltd"
new string: "VLSI Academy India"
```

String commands

- **trimleft string ?trimcharacters?**

Removes trimcharacters in left beginning of string. The default trimcharacters is whitespace.

```
set s1 "Physical Design includes stages from floorplan to routing"
set s2 "Physical"
puts "Trim left: \"$s2\" in \"$s1\""
puts "new string: \"[string trimleft $s1 $s2]\""
```

Output:

```
Trim left: "Physical" in "Physical Design includes stages from floorplan to routing"
new string: "Design includes stages from floorplan to routing"
```

String commands

- **trim string ?trimcharacters?**

Removes trimcharacters in both ends of string. The default trim characters is whitespace

```
set s1 ":::Physical Design:::"
set s2 ":"
puts "Trim on both sides- \"$s2\" in \"$s1\""
puts "new string: \"[string trim $s1 $s2]\""
```

Output:

```
Trim on both sides- ":" in ":::Physical Design:::"
new string: "Physical Design"
```

String commands

- **match pattern string**

Returns 1 if the string matches the pattern.

```
set s1 "test-id@goldenLightVLSI.com"
set s2 "*@*.com"
puts "Matching pattern $s2 in $s1"
if { [string match "*@*.com" $s1 ] } {
puts "match found"
} else { puts "match not found" }

puts "Matching pattern \"tcl\" in $s1"
if {[string match {tcl} $s1]} {
puts "match found"
} else { puts "match not found"}
```

Output:

```
Matching pattern *@*.com in test-
id@goldenLightVLSI.com

match found

Matching pattern "tcl" in test-
id@goldenLightVLSI.com

match not found
```

String commands

- **Appending string:**

It appends string to original string itself. It doesn't create a new string.

Syntax:

```
append original_string appending_string
```

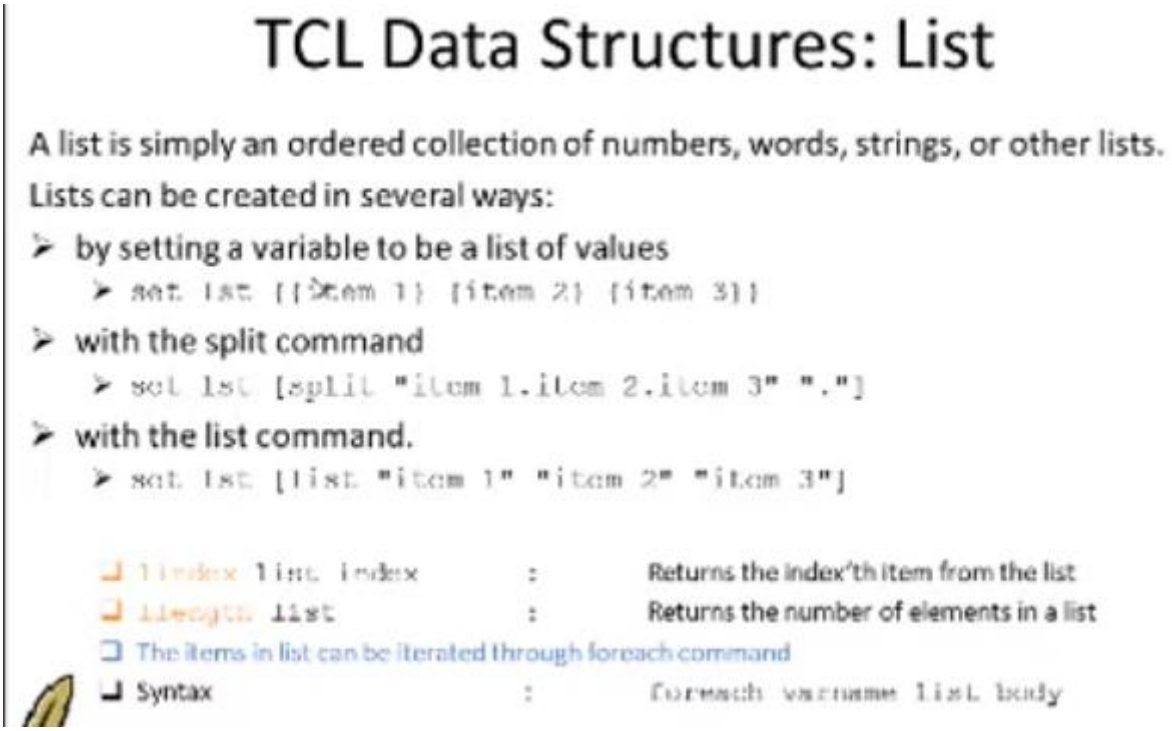
Example:

```
set s1 "We love to study tcl"
append s1 "by VLSI Academy"
puts $s1
```

Output:

```
We love to study tcl by VLSI Academy
```

TCL lecture9: TCL Data Structures | Lists in tcl | Part - 1



A list is simply an ordered collection of numbers, words, strings, or other lists.

Lists can be created in several ways:

- by setting a variable to be a list of values

set lst {{item 1} {item 2} {item 3}}

- with the split command

set lst [split "item 1.item 2.item 3" "."]

- with the list command

set lst [list "item 1" "item 2" "item 3"]

lindex list index : Returns the index'th item from the list

llength list : Returns the number of elements in a list

The items in list can be iterated through foreach command

Syntax: foreach varname list body

```
#string operations with examples
#we will learn use of lindex and split command
#we will learn how to iterate over items of list

set x "a b c"
puts "Item at index 2 of the list {$x} is : [lindex $x 2] \n"

set y [split 7/4/1776 "/"]
#printing string "We celebrate on the 4'th day of the 7'th month"
puts "We celebrate on the [lindex $y 1]'th day of the [lindex $y 0]'th montl
```

```
PS F:\tcl> tclsh .\string_operations.tcl
Item at index 2 of the list {a b c} is : c

We celebrate on the 4'th day of the 7'th month
PS F:\tcl>
```

```
#string operations with examples
#we will learn use of lindex and split command
#we will learn how to iterate over items of list

set x "a b c"
puts "Item at index 2 of the list {$x} is : [lindex $x 2] \n"

set y [split 7/4/1776 "/"]
#printing string "We celebrate on the 4'th day of the 7'th month"
puts "We celebrate on the [lindex $y 1]'th day of the [lindex $y 0]'th montl

#iterating over items of list
set i 0
foreach j $x {
puts "$j is item number $i in list x"
incr i
}
```

```
PS F:\tcl> tclsh .\string_operations.tcl
Item at index 2 of the list {a b c} is : c

We celebrate on the 4'th day of the 7'th month
a is item number 0 in list x
b is item number 1 in list x
c is item number 2 in list x
PS F:\tcl>
```

TCL lecture10: TCL Data Structures | Lists in tcl | Part - 2

List Operations

Concat

- Concatenates the args into a single list.
  For example: set c [concat $a $b]

Linsert

- Inserts the items into list at specified index and returns the new list with inserted item
  For e.g.: set c [linsert “a b c” 1 “z”]

Lreplace

- Replaces the list item at specified index with new specified item.
  For e.g. set c [lreplace “a b c” 1 “z”]

lists_example1.tcl

```
1  #concat example
2  set emp_name1 "Jacob"
3  set emp_name2 "Tina"
4  set emp_name3 "Ryan"
5  set employee_list [concat $emp_name1 $emp_name2 $emp_name3]
6  puts $employee_list
```

these 3 variables are concatenated into single list variable

```
PS F:\tcl> tclsh .\lists_example1.tcl
Jacob Tina Ryan
PS F:\tcl>
```

lists_example1.tcl

```
1  #concat example
2  set emp_name1 "Jacob"
3  set emp_name2 "Tina"
4  set emp_name3 "Ryan"
5  set employee_list [concat $emp_name1 $emp_name2 $emp_name3]
6  puts $employee_list
7
8  #linsert example
9  set names "Jacob Ryan Marshal Rosy"
10 set new_names [linsert $names 2 "John" ]
11 puts "Old list : $names "
12 puts "New list : $new_names "
```

```
PS F:\tcl> tclsh .\lists_example1.tcl
Jacob Tina Ryan
PS F:\tcl> tclsh .\lists_example1.tcl
Jacob Tina Ryan
Old list : Jacob Ryan Marshal Rosy
New list : Jacob Ryan John Marshal Rosy
PS F:\tcl>
```

lists_example1.tcl

```
1   #concat example
2   set emp_name1 "Jacob"
3   set emp_name2 "Tina"
4   set emp_name3 "Ryan"
5   set employee_list [concat $emp_name1 $emp_name2 $emp_name3]
6   puts $employee_list
7
8   #linsert example
9   set names "Jacob Ryan Marshal Rosy"
10  set new_names [linsert $names 2 "John" ]
11  puts "Old list : $names "
12  puts "New list : $new_names "
13
14  #lreplace example
15  set new_names [lreplace $names 2 2 "John"]
16  puts "Old List : $names "
17  puts "new List : $new_names "
```

```
PS F:\tcl> tclsh .\lists_example1.tcl
Jacob Tina Ryan
PS F:\tcl> tclsh .\lists_example1.tcl
Jacob Tina Ryan
Old list : Jacob Ryan Marshal Rosy
New list : Jacob Ryan John Marshal Rosy
PS F:\tcl> tclsh .\lists_example1.tcl
Jacob Tina Ryan
Old list : Jacob Ryan Marshal Rosy
New list : Jacob Ryan John Marshal Rosy
Old List : Jacob Ryan Marshal Rosy
new List : Jacob Ryan John Rosy
PS F:\tcl>
```

TCL lecture11: TCL Data Structures | Lists in tcl | Part - 3

List Operations

Lappend	• Appends new item at the last, to the existing list
Lsort	• Sorts list and returns a new list in the sorted order. By default, sorts the list in dictionary order
Lrange	• Returns a list composed of the first through last entries in the list

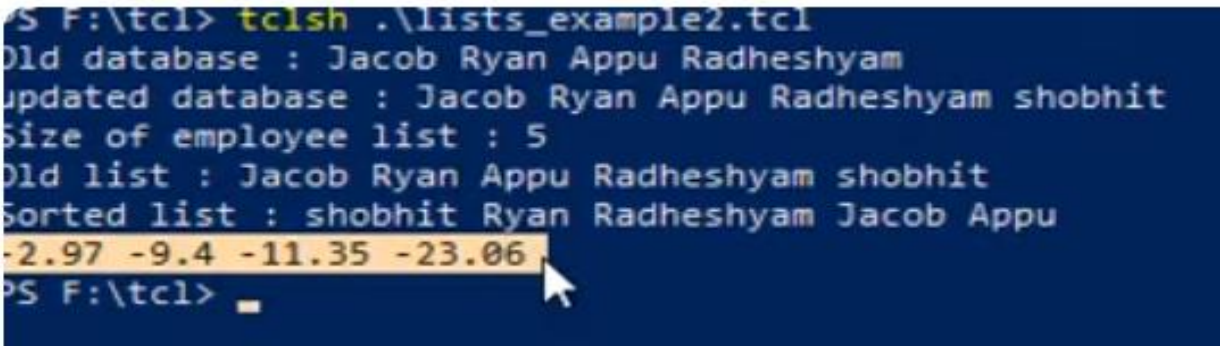
```
#lappend example
set employee_list "Jacob Ryan Appu Radheshyam"
set new_employee "shobhit"
puts "Old database : $employee_list"
lappend employee_list $new_employee
puts "updated database : $employee_list"
```

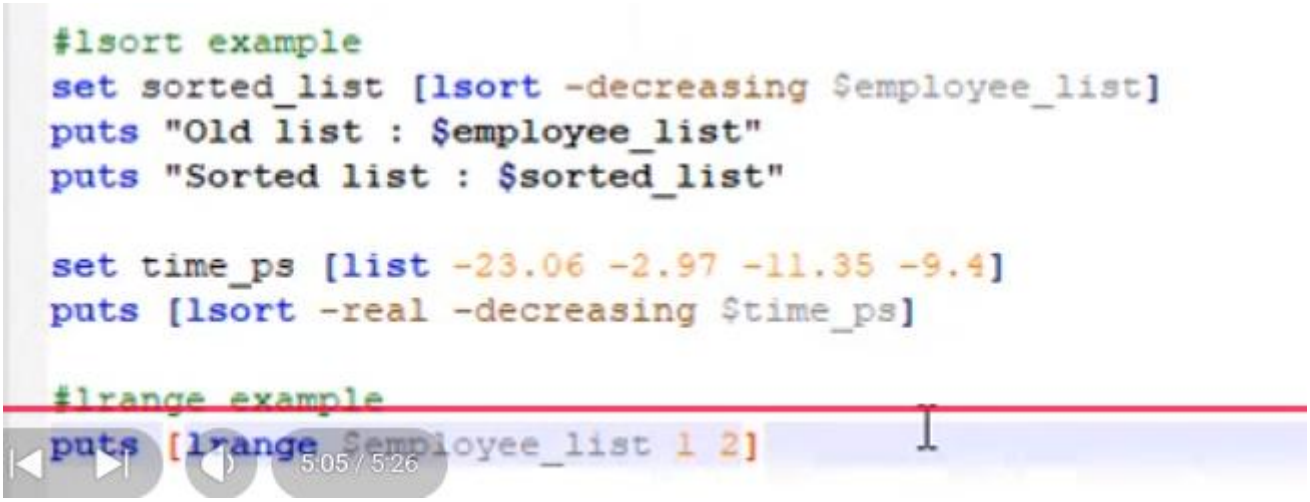
```
#lappend example
set employee_list "Jacob Ryan Appu Radheshyam"
set new_employee "shobhit"
puts "Old database : $employee_list"
lappend employee_list $new_employee
puts "updated database : $employee_list"

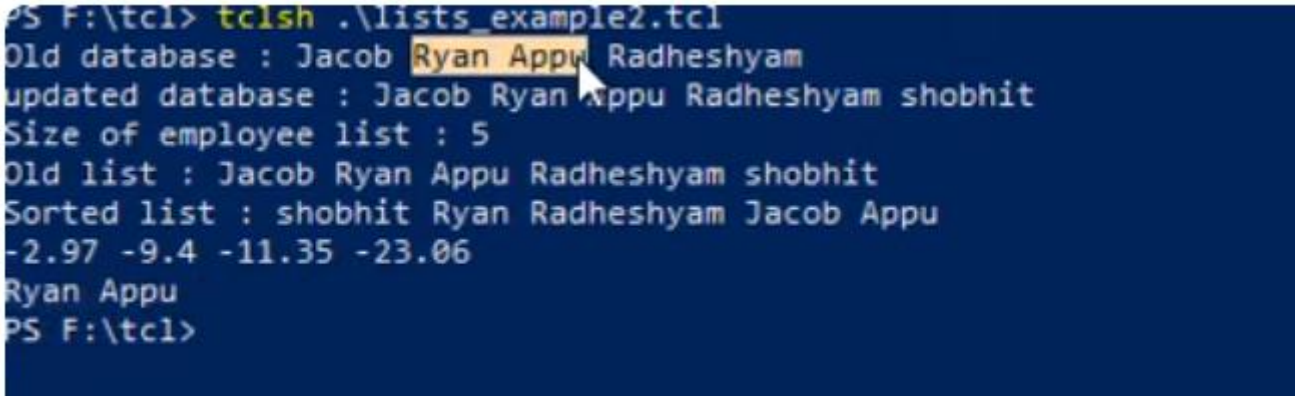
#llength example
puts "Size of employee list : [llength $employee_list]"

#lsort example
set sorted_list [lsort -decreasing $employee_list]
puts "Old list : $employee_list"
puts "Sorted list : $sorted_list"

set time_ps [list -23.06 -2.97 -11.35 -9.4]
puts [lsort -real -decreasing $time_ps]
```

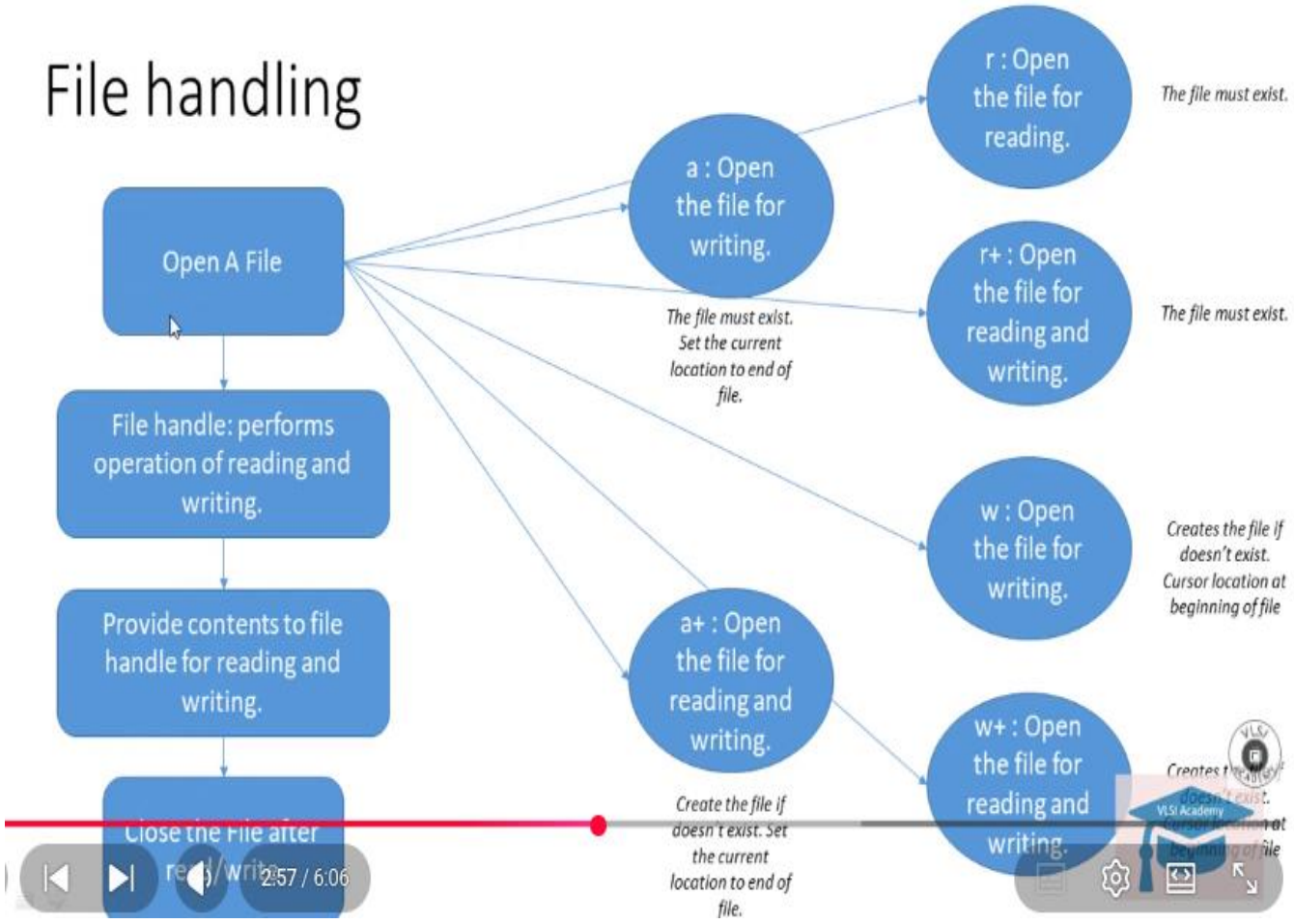






VLSI ACADEMY- Youtube

TCL lecture12 : TCL File Handling : File Read and Write in TCL - Part 1



Reading entire file at once

```
set fp [open "input.txt" r]
set file_data [read $fp]
puts $file_data
close $fp
```

Reading file line by line

```
set fp [open "input.txt" r]
while { [gets $fp data] >= 0 } {
   puts $data
}
close $fp
```

First, you have to open the file if file should exists then. r means opening for reading only. open is the command which is used for opening the file. fp is the file handle which is used to permission to read this file content. read is used for reading the content. read data will store in file_data variable and after that if you want to print then use puts $file_data then we have to close the file. in this way, we can print things.

File handling

Writing file contents

```
set fp [open "input.txt" w+]
puts $fp "test"
close $fp
```

Name of file: **results.txt**

Device_ID X Y Volt I(nA) Result

ND129P14A 0 0 0.77 20 Fail

ND127P16B 0 1 0.77 30 Pass

ND134P18D 0 2 0.77 40 Fail

ND128P24E 0 3 0.77 90 Fail

ND141P14M 1 4 0.82 60 Fail

ND159P16X 1 5 0.82 70 Pass

ND169P18V 1 6 0.82 50 Fail

ND189P24W 1 7 0.82 10 Fail

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

while {[gets $file_handle data] >=0 } {
        puts [lindex $data 5]
}
close $file_handle
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
```

Result

Fail

Pass

Fail

Fail

Fail

Pass

Fail

Fail

Printing all data at the same time

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

while {[gets $file_handle data] >=0 } {
        puts "$data"
}
close $file_handle
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ vi file_handle1.tcl
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
Device_ID    X   Y   Volt   I(nA)   Result
ND129P14A    0   0   0.77   20      Fail
ND127P16B    0   1   0.77   30      Pass
ND134P18D    0   2   0.77   40      Fail
ND128P24E    0   3   0.77   90      Fail
ND141P14M    1   4   0.82   60      Fail
```

ND159P16X 1 5 0.82 70 Pass

ND169P18V 1 6 0.82 50 Fail

ND189P24W 1 7 0.82 10 Fail

I want to print two Column line from File

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

while {[gets $file_handle data] >=0 } {
        puts "[lindex $data 0] [lindex $data 5]"
}
close $file_handle
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
Device_ID Result
ND129P14A Fail
ND127P16B Pass
ND134P18D Fail
ND128P24E Fail
ND141P14M Fail
ND159P16X Pass
ND169P18V Fail
ND189P24W Fail
```

I want to print two row line only

```
#!/usr/bin/tclsh
```

```
set filename "results.txt"
set file_handle [open $filename r]

set count 0
while {[gets $file_handle data] >=0 } {
        puts "$data"
        incr count
        if { $count ==2} {
                break
        }
}
close $file_handle
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
Device_ID    X   Y   Volt   I(nA)   Result
ND129P14A    0   0   0.77   20      Fail
```

I want to print some row of range of data only

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

set count 0
while {[gets $file_handle data] >=0 } {
        #puts "$data"
        incr count
        if { $count >= 2 && $count <=4 } {
```

```
        puts "$data"
    }

    if { $count >4} {
        break
    }
}
close $file_handle
```

Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
ND129P14A    0   0   0.77   20      Fail
ND127P16B    0   1   0.77   30      Pass
ND134P18D    0   2   0.77   40      Fail
```

I want to print some specific numbers of rows Like 2 ,4, 8

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

set count 0
while {[gets $file_handle data] >=0 } {
    #puts "$data"
    incr count
    if { $count == 2 || $count ==4 || $count ==8 } {
        puts "$data"
    }
```

```
}
close $file_handle

[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
ND129P14A    0   0   0.77   20      Fail
ND134P18D    0   2   0.77   40      Fail
ND169P18V    1   6   0.82   50      Fail
```

I want to print row number 5 and column 3 of only specific value-

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

set count 0
while {[gets $file_handle data] >=0 } {
        #puts "$data"
        incr count
        if { $count == 5 } {
                puts [lindex $data 2]
                break
       }

}
close $file_handle
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
3
```

I want to print all data to perform the addition in column wise: -

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

set line_num 0
while {[gets $file_handle line] >=0 } {
        #puts "$data"
        incr line_num
        if { $line_num == 1 } {
                continue
       }
       set colm [regexp -all -inline {\S+} $line]

       if {[llength $colm] < 5} {
                continue
}
       set X [lindex $colm 1]
       set Y [lindex $colm 2]
       set Volt [lindex $colm 3]
       set I [lindex $colm 4]

       set sum [expr {$X + $Y + $Volt + $I}]
       puts "The sum of all in column wise : $sum"
```

```
}
close $file_handle
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ vi results.txt
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
The sum of all in column wise : 20.77
The sum of all in column wise : 31.77
The sum of all in column wise : 42.77
The sum of all in column wise : 93.77
The sum of all in column wise : 65.82
The sum of all in column wise : 76.82
The sum of all in column wise : 57.82
The sum of all in column wise : 18.82
```

Printing data of Only PASS element at output terminal:

```
#!/usr/bin/tclsh
set filename "results.txt"
set file_handle [open $filename r]

while { [gets $file_handle line] >=0 } {

    if {[regexp {^Device_ID} $line]} {
        continue
    }
```

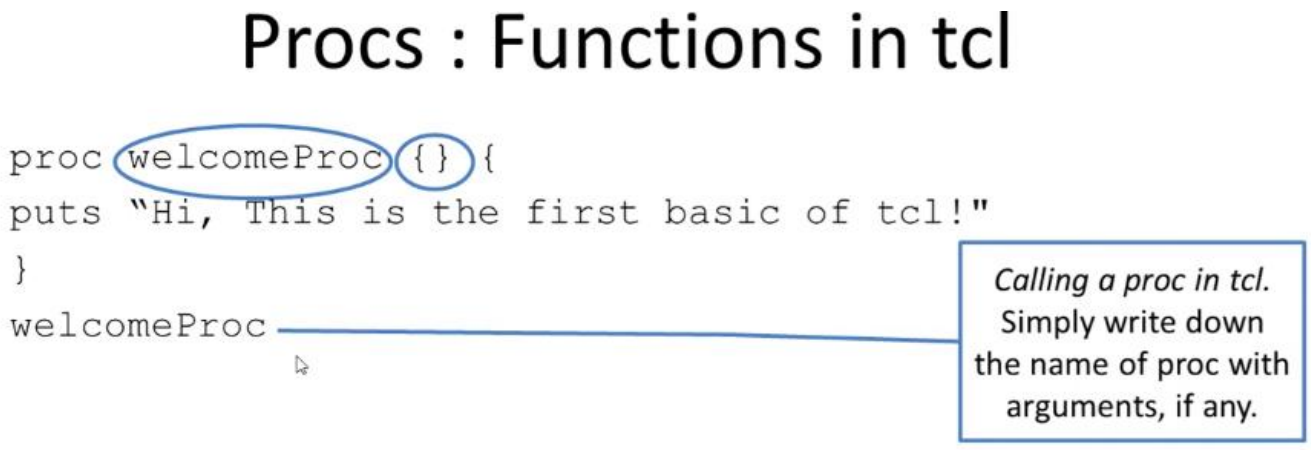
```
    set cols [regexp -all -inline {\S+} $line]

    if {[lindex $cols end] eq "Pass"} {
        puts [lindex $cols 0]
    }
}

close $file_handle
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./file_handle1.tcl
ND127P16B
ND159P16X
```



Procs with multiple arguments

```
proc sum {a b} {

    return [expr $a+$b]
}
puts [sum 10 30]
```

In this case number of arguments is known in advance and is fixed count.

Procs with multiple arguments

```
proc employee_modifier {emp_ID} {

    return [join [list "1" $emp_ID] ""]
}
puts [employee_modifier 7892]
```

Output : 17892

Arguments with default values

```
proc employee_modifier {emp_ID {code 000}} {

    return [join [list $code $emp_ID] ""]
}
puts [employee_modifier 7892]
```

```
Output : 0007892
```

Procs with variable arguments

```
set employees [list "raju" "danish" "ratnesh" "ekansh"]

proc email {employee_names} {
        set empID {}
        foreach name $employee_names {
                set ID [join [list $name "@ymail.com"] ""]
                lappend empID $ID
        }
        return $empID
}
puts [email $employees]
```

```
Output:
raju@ymail.com danish@ymail.com ratnesh@ymail.com ekansh@ymail.com
```

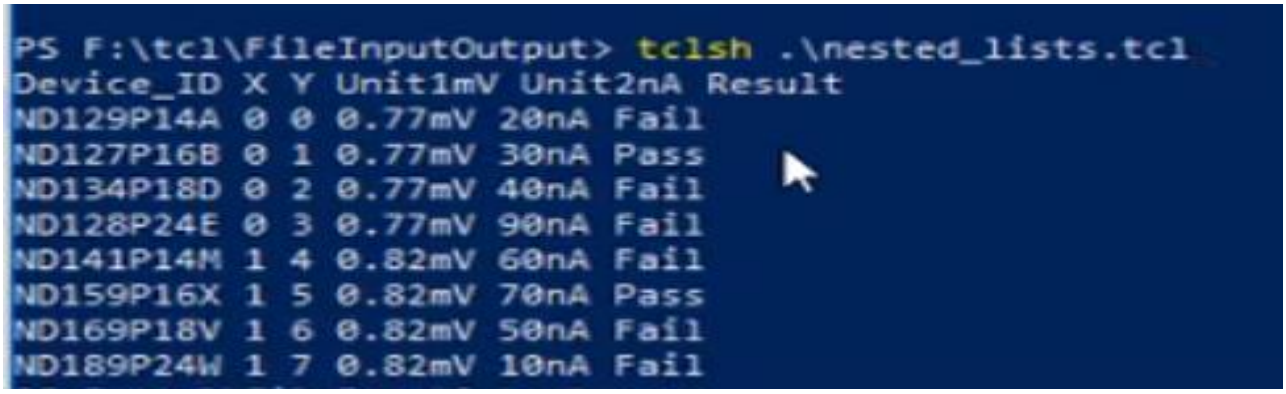
```
set filename "results.rpt"
set file_handle [open $filename r]
set data {}
while { [gets $file_handle line ] >= 0} {
    puts "[lindex $line 3] and [lindex $line 4]"


    }
```

```
PS F:\tcl\FileInputOutput> tclsh .\nested_lists.tcl
Unit1 and Unit2
0.77 and 20
0.77 and 30
0.77 and 40
0.77 and 90
0.82 and 60
0.82 and 70
0.82 and 50
0.82 and 10
```

Device_ID	X	Y	Unit1	Unit2
ND129P14A	0	0	0.77	20

```
1   set filename "results.rpt"
2   set file_handle [open $filename r]
3   set data {}
4   while { [gets $file_handle line ] >= 0} {
5       #puts "[lindex $line 3] and [lindex $line 4]"
6
7       #appending unit of millivolts (mV) to unit1  value
8       set volts [join [list [lindex $line 3] mV] {}]
9
0       #appending unit of milliamperes (mA) to unit2 value
1       set current [join [list [lindex $line 4] nA] {}]
2
3       set line [lreplace $line 3 3 $volts]
4       set line [lreplace $line 4 4 $current]
5       puts $line
6       }
        lindex
```



20 basic TCL coding problems (interview-friendly, simple, and short):-

◇ 1. Print “Hello World”

```
puts "Hello World"
```

◇ 2. Declare a variable and print it

```
set a 10
puts $a
```

◇ 3. Add two numbers

```
set a 5
set b 3
puts [expr {$a + $b}]
```

◇ 4. Check even or odd

```
set n 6
if {[expr {$n % 2}] == 0} {
```

```
    puts "Even"
} else {
    puts "Odd"
}
```

◇ 5. Find largest of two numbers

```
set a 10
set b 20
if {$a > $b} {
    puts $a
} else {
    puts $b
}
```

◇ 6. Loop from 1 to 5

```
for {set i 1} {$i <= 5} {incr i} {
    puts $i
}
```

◇ 7. Print list elements

```
set lst {a b c d}
foreach i $lst {
    puts $i
}
```

◇ 8. Count elements in a list

```
set lst {10 20 30}
```

```
puts [llength $lst]
```

◇ 9. Read a file line by line

```
set fp [open "data.txt" r]
while {[gets $fp line] != -1} {
    puts $line
}
close $fp
```

◇ 10. Write data into a file

```
set fp [open "out.txt" w]
puts $fp "Hello TCL"
close $fp
```

◇ 11. Check if a file exists

```
if {[file exists "data.txt"]} {
    puts "File exists"
}
```

◇ 12. Procedure to add two numbers

```
proc add {a b} {
    return [expr {$a + $b}]
}
puts [add 3 4]
```

◇ 13. Find string length

```
set s "TCL"
```

```
puts [string length $s]

## ◇ 14. Convert string to uppercase
set s "tcl"
puts [string toupper $s]

## ◇ 15. Match ERROR in a line
set line "ERROR found"
if {[regexp {ERROR} $line]} {
    puts "Error detected"
}

## ◇ 16. Case-insensitive ERROR match
if {[regexp -nocase {\bERROR\b} $line]} {
    puts $line
}

## ◇ 17. Use array
set arr(name) Sarif
puts $arr(name)

## ◇ 18. Dictionary example
dict set info WNS -0.3
puts [dict get $info WNS]

## ◇ 19. If-else condition
set a 5
```

```
if {$a > 0} {
    puts "Positive"
} else {
    puts "Negative"
}
```

◇ 20. Simple regression check (EDA-style)

```
set pass 1
if {$pass} {
    puts "Regression PASSED"
} else {
    puts "Regression FAILED"
}
```

20 Basic TCL Coding Problems (clean, interview-ready, no extra theory):

20 Basic TCL Coding Problems — SOLUTIONS

1. 0 Print “Hello World”

```
puts "Hello World"
```

2. 0 Store a value in a variable and print it

```
set a 10
puts $a
```

3. 0 Add two numbers

```
set a 5
set b 7
puts [expr {$a + $b}]
```

4. 0 Subtract two numbers

```
set a 10
set b 4
puts [expr {$a - $b}]
```

5. 0 Multiply two numbers

```
set a 3
set b 6
puts [expr {$a * $b}]
```

6. 0 Divide two numbers

```
set a 20
set b 5
puts [expr {$a / $b}]
```

7. 0 Check even or odd number

```
set n 9
if {[expr {$n % 2}] == 0} {
    puts "Even"
} else {
    puts "Odd"
}
```

8. 0 Find the largest of two numbers

```
set a 15
set b 10
puts [expr {$a > $b ? $a : $b}]
```

9. 0 Loop from 1 to 10

```
for {set i 1} {$i <= 10} {incr i} {
    puts $i
}
```

10 Print elements of a list

```
set mylist {A B C D}
foreach i $mylist {
    puts $i
}
```

11 Count elements in a list

```
set mylist {10 20 30 40}
puts [llength $mylist]
```

12 Access a list element

```
set mylist {red green blue}
puts [lindex $mylist 1]
```

13 Read a file line by line

```
set fp [open "input.txt" r]
while {[gets $fp line] != -1} {
    puts $line
```

```
}
close $fp
```

14 Write data to a file

```
set fp [open "output.txt" w]
puts $fp "TCL scripting"
close $fp
```

15 Check if a file exists

```
if {[file exists "input.txt"]} {
    puts "File exists"
}
```

16 Procedure to add two numbers

```
proc add {a b} {
    return [expr {$a + $b}]
}
puts [add 4 6]
```

17 Find length of a string

```
set str "VLSI"
puts [string length $str]
```

18 Convert string to uppercase

```
set str "tcl"
puts [string toupper $str]
```

19 Match ERROR in a log line

```
set line "ERROR: timing failed"
if {[regexp {ERROR} $line]} {
   puts "Error found"
}
```

20 Simple regression result check

```
set result PASS
if {$result eq "PASS"} {
   puts "Regression Passed"
} else {
   puts "Regression Failed"
}
```

regexp character in TCL

Complete list of commonly used `regexp` characters in TCL, explained simply (no heavy theory).

Basic regexp characters (TCL)

1 `.` **(dot)**

- Matches any single character

```
regexp {a.b} "acb"   ;# matches
```

2 `*`

- Matches **0 or more** of the previous character

```
regexp {ab*} "a"     ;# matches
regexp {ab*} "abbb"  ;# matches
```

3 `+`

- Matches 1 or more of the previous character

```
regexp {ab+} "abbb"  ;# matches
regexp {ab+} "a"     ;# no match
```

4 `?`

- Matches 0 or 1 of the previous character

```
regexp {colou?r} "color"   ;# matches
regexp {colou?r} "colour" ;# matches
```

5 `^`

- Matches start of line

```
regexp {^Slack} "Slack = -0.2"  ;# matches
```

6 `$`

- Matches end of line

```
regexp {ns$} "10 ns"   ;# matches
```

7 `[ ]` (character class)

- Matches any one character inside

regexp {[0-9]} "A3B"   ;# matches

8 `[^ ]`

- Matches NOT these characters

regexp {[^0-9]} "A3"   ;# matches A

9 `{n}`

- Matches exactly n times

regexp {[0-9]{3}} "123"   ;# matches

10 `{n,m}`

- Matches between n and m times

regexp {[0-9]{2,4}} "1234"   ;# matches

Common predefined patterns

1 `\d`

- Digit (0–9)

regexp {\d+} "123"

2 `\D`

- Non-digit

regexp {\D} "A1"

3 `\s`

- Whitespace (space, tab)

regexp {\s+} " A"

4 `\S`

- Non-whitespace

regexp {\S+} "Slack"

5 `\w`

- Word character (a–z, A–Z, 0–9, _)

regexp {\w+} "cell_1"

6 `\W`

- Non-word character

regexp {\W} "@#$"

🎯 Grouping & extraction

`()`

- Capture group

regexp {Slack\s*=\s*(-?\d+\.\d+)} $line -> slack

`|`

- OR condition

regexp {PASS|FAIL} "TIMING PASS"

🧠 TCL-specific useful flags

`-all`

- Find all matches

regexp -all {\d+} "1 2 3"

`-inline`

- Return matched values

set nums [regexp -all -inline {\d+} "1 2 3"]

☑ Most used in VLSI TCL scripts

✓ `\s`, `\S`

✓ `\d`, `+`, `*`

✓ `^`, `$`

✓ `()`

✓ `-all -inline`

Carrot- ^

🔑 One-line summary:

`regexp` helps you extract numbers, slack, WNS, timing info from reports even when spacing and format change.

Practice `regexp` questions with answers, VLSI-focused, short and clear.

🧪 Practice 1: Extract Slack value

Input:

set line "Slack = -0.45"

Task: Extract `-0.45`

Answer:

regexp {Slack\s*=\s*(-?\d+\.\d+)} $line -> slack

puts $slack

Output:

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

-0.45

🧪 Practice 2: Check if timing FAILED or PASSED

Input:

set line "TIMING FAILED"

Task: Detect FAILED

Answer:

```
if {[regexp {FAILED} $line]} {
    puts "Timing failed"
```

}

Output:

Timing failed

🧪 Practice 3: Extract number from report line

Input:

set line "Total cells : 12345"

Task: Extract `12345`

Answer:

regexp {(\d+)} $line -> count

puts $count

Output:

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

12345

🧪 Practice 4: Remove extra spaces (report cleanup)

Input:

set line " WNS   =   -1.23   "

Task: Clean spaces

Answer:

regsub -all {\s+} $line " " clean

puts $clean

Output:

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

WNS = -1.23

🧪 Practice 5: Extract all numbers from a line

Input:

set line "Path 3 has delay 12.45 ns"

Task: Extract `3` and `12.45`

Answer:

set nums [regexp -all -inline {\d+\.?\d*} $line]

puts $nums

Output:

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

3 12.45

🧪 Practice 6: Check if line starts with WNS

Input:

set line "WNS : -0.10"

Answer:

```
if {[regexp {^WNS} $line]} {
    puts "WNS line found"
}
```

Output:

WNS line found

🧪 Practice 7: Extract cell name

Input:

set line "Cell = U1234/NAND2_X1"

Answer:

regexp {Cell\s*=\s*(\S+)} $line -> cell

puts $cell

Output:

U1234/NAND2_X1

🧪 Practice 8: Count how many numbers in a line

Input:

set line "1 22 333 4444"

Answer:

set nums [regexp -all -inline {\d+} $line]

puts [llength $nums]

Output:

4

🧠 Interview Tip (VLSI TCL)

If interviewer asks:

> “Where do you use regexp in real flow?”

Answer:

Parsing STA reports, extracting WNS/TNS, checking violations, filtering tool outputs.

Top 10 regexp interview questions

Top 10 `regexp` Interview Questions (TCL / VLSI focused).

1 What is `regexp` in TCL?

Answer:

`regexp` is used to search and extract patterns from text (very common for report parsing).

Example:

regexp {Slack} "Slack = -0.2"

2 How do you check if a pattern exists in a line?

Answer:

Use `regexp` inside an `if`.

```
if {[regexp {FAILED} $line]} {
   puts "Timing failed"
}
```

3 How do you extract a value using `regexp`?

Answer:

Use capturing brackets `()`.

regexp {Slack\s*=\s*(-?\d+\.\d+)} $line -> slack

4 What does `\s` mean in `regexp`?

Answer:

`\s` matches whitespace (space, tab).

regexp {Slack\s*=\s*} $line

5 Difference between `regexp` and `string match`?

Answer:

- `regexp` → powerful, flexible patterns
- `string match` → simple wildcard matching

regexp {Slack} $line

string match "*Slack*" $line

6 What does `^` and `$` mean?

Answer:

- `^` → start of line
- `$` → end of line

regexp {^WNS} $line

7 How do you extract multiple numbers from a line?

Answer:

Use `-all -inline`.

regexp -all -inline {\d+\.?\d*} $line

8 What does `-?` mean in regexp?

Answer:

It means optional minus sign (for negative slack).

regexp {(-?\d+\.\d+)} $line

9 Where is `regexp` used in VLSI flow?

Answer:

- Parsing STA reports
- Extracting WNS / TNS
- Checking violations
- Filtering tool logs

regexp {WNS} $report_line

10 Common mistake with `regexp`?

Answer:

Not handling extra spaces in reports.

❌ Wrong:

regexp {Slack=-0.2} $line

☑ Correct:

regexp {Slack\s*=\s*-0.2} $line

💡 Interview Power Tip

If asked:

“Why regexp instead of scan?”

Say:

Reports are not fixed format → regexp is safer and flexible

Tricky regexp questions with traps

1 Trap: Extra spaces in reports

Input:

set line " Slack   =   -0.35 "

❌ Wrong:

regexp {Slack = -0.35} $line

☑ Correct:

regexp {Slack\s*=\s*(-?\d+\.\d+)} $line -> slack

👉 Why? Reports never have fixed spaces.

2 Trap: Negative numbers not captured

Input:

set line "WNS : -1.20"

❌ Wrong:

regexp {(\d+\.\d+)} $line

☑ Correct:

regexp {(-?\d+\.\d+)} $line -> wns

👉 **Why? Missing optional `-`.**

3 Trap: Header line matches accidentally

Input:

set line "Slack Value"

❌ Wrong:

regexp {Slack} $line

☑ Correct:

regexp {Slack\s*[:=]} $line

👉 Why? Header lines cause false matches.

4 Trap: `*` vs `+`

Input:

set line "Slack ="

❌ Wrong:

regexp {Slack\s*=\s*\d+} $line

☑ Correct:

regexp {Slack\s*=\s+\d+} $line

👉 Why?

- `*` → zero or more

- `+` → one or more

5 Trap: Using `.` without escaping

Input:

set line "Delay = 1.25"

❌ Wrong:

regexp {1.25} $line

☑ Correct:

regexp {1\.25} $line

👉 Why? `.` means any character.

6 Trap: Greedy match eats too much

Input:

set line "Path slack = -0.3 ns"

❌ Wrong:

regexp {=(.*)} $line -> value

Result: `-0.3 ns` ❌

☑ Correct:

regexp {=\s*(-?\d+\.?\d*)} $line -> value

👉 Why? `.*` is greedy.

7 Trap: Forgetting `{}` around regexp

❌ Wrong:

```
regexp "\d+\.\d+" $line
```

⚠ Tcl interprets `\d`

☑ Correct:

```
regexp {\d+\.\d+} $line
```

👉 Always use `{}` for regexp patterns.

8 Trap: Matching full line vs part of line

Input:

```
set line "Worst Slack = -0.45"
```

❌ Wrong:

```
regexp {^Slack} $line
```

☑ Correct:

```
regexp {Slack} $line
```

👉 `^` anchors start of line.

9 Trap: `string match` used instead of `regexp`

❌ **Wrong:**

string match "*-0.25*" $line

☑ Correct:

regexp {-0\.25} $line

👉 `string match` can’t handle numeric patterns well.

10 Interview Killer Question

Q: Why is `regexp` preferred for STA reports?

Perfect Answer:

STA reports are unstructured and space-variant, so `regexp` handles format changes safely.

🧠 Golden Rule (Remember This)

Reports change. Regex survives.

Mini VLSI STA Report Parser Project (TCL) — interview-ready, simple, practical.

🎯 Project Goal

Parse an STA timing report and:

- Extract WNS
- Decide PASS / FAIL

- Ignore headers & noise

📄 Sample STA Report (`timing.rpt`)

```
---------------------------------
Report Timing Summary
---------------------------------
WNS : -0.35
TNS : -12.40
Paths Violated : 5
---------------------------------
```

🧠 What we will do

1. Read file line by line
2. Find line containing `WNS`
3. Extract numeric value using `regexp`
4. Print PASS / FAIL

☑ TCL Script (Mini Project)

```
#!/usr/bin/tclsh

set fp [open "timing.rpt" r]
set wns ""

while {[gets $fp line] >= 0} {

    if {[regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns]} {
```

```
        break
    }
}
close $fp

if {$wns eq ""} {
    puts "WNS not found in report"
} elseif {$wns >= 0} {
    puts "☑ TIMING PASSED (WNS = $wns)"
} else {
    puts "❌ TIMING FAILED (WNS = $wns)"
}
```

🧩 Line-by-Line Simple Explanation

```
set fp [open "timing.rpt" r]
```

👉 **Open timing report in read mode**

```
while {[gets $fp line] >= 0} {
```

👉 Read file line by line

```
regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns
```

👉 If line contains `WNS : number`, extract the number

```
break
```

👉 Stop after finding first WNS

if {$wns >= 0}

👉 Decide PASS / FAIL

🖥️ Sample Output

❌ TIMING FAILED (WNS = -0.35)

or

☑ TIMING PASSED (WNS = 0.12)

💡 Why Interviewers Love This

✓ Uses `regexp`

✓ Handles real STA reports

✓ No hardcoding

✓ Automation-friendly

🏁 One-Line Interview Summary

“I wrote a TCL script to parse STA reports, extract WNS using regexp, and automatically flag timing PASS/FAIL.”

VLSI Report parsing analysis-

1 Extract WNS from timing report

regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns

👉 Extracts worst negative slack value.

```
#!/usr/bin/tclsh

set fp [open "timing.txt" r]
set wns_value "Not Found"

while {[gets $fp line] != -1} {
    if {[regexp {WNS\s*:\s*(-?\d+\.?\d*)} $line -> value]} {
        set wns_value $value
        break
    }
}
close $fp
puts "WNS = $wns_value"
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
WNS = -0.35
```

2 Extract TNS

regexp {TNS\s*:\s*(-?\d+\.\d+)} $line -> tns

👉 Gets total negative slack.

3 Count violated paths

regexp {Paths\s+Violated\s*:\s*(\d+)} $line -> count

👉 Reads number of timing violations.

4 Check timing PASS / FAIL

```
if {$wns >= 0} {
   puts "TIMING PASSED"
} else {
   puts "TIMING FAILED"
}
```

5 Extract slack from path-based report

```
regexp {slack\s*\(.*\)\s*(-?\d+\.\d+)} $line -> slack
```

👉 Used in PrimeTime path reports.

6 Parse cell area from area report

```
regexp {Total\s+cell\s+area\s*=\s*(\d+\.\d+)} $line -> area
```

7 Find all violated paths (loop example)

```
if {[regexp {VIOLATED} $line]} {
   puts $line
}
```

8 Extract numbers from any line (generic)

```
set nums [regexp -all -inline {\d+\.?\d*} $line]
```

👉 **Useful for messy reports.**

9 Ignore header lines safely

```
if {[regexp {^-+} $line]} {
   continue
}
```

10 Mini full example (realistic)

```
while {[gets $fp line] >= 0} {
   if {[regexp {WNS\s*:\s*(-?\d+\.\d+)} $line -> wns]} {
      puts "WNS = $wns"
      break
   }
}
```

🧠 Interview Tip (Say This)

“I use `regexp` because VLSI reports have variable spacing and format, so fixed parsing fails.”

Top 20 regexp example for VLSI domain in TCL for Design Verification interviews questions?

📄 Sample VLSI Report (Timing / QoR / Log)

```
========================================
Design Name    : top_module
```

Tool : Synopsys PrimeTime

Date : 12-Jan-2026

==========

Clock Name : clk

Clock Period : 2.50

Clock Uncertainty : 0.10

WNS : -0.35

TNS : -12.40

Violating Paths : 6

Max Delay Path

Startpoint : U_REG1/Q

Endpoint : U_REG5/D

Path Group : clk

Slack : -0.35

Cell Area : 12543.78

Total Power : 87.32 mW

Dynamic Power : 65.10 mW

Leakage Power : 22.22 mW

Number of Cells : 18450

Number of Sequential : 6240

Number of Combinational : 12210

Fanout Violation Net : net_critical_12

Fanout Count   : 18

Transition Violation : net_fast_3

Transition Value   : 0.45

Capacitance Violation : net_cap_9

Capacitance Value   : 0.62

INFO   : Optimization completed successfully

WARNING: Clock skew is high on clk

ERROR : Timing constraints not met

End of Report

- This single report contains all 20 data points (WNS, TNS, slack, power, cells, violations, warnings, errors, clock info, etc.)
- Every regexp question/example you asked earlier is extracted from THIS report**

☑ Complete TCL Script to Extract Multiple Parameters

```
set fp [open "report.txt" r]

set wns ""

set tns ""
```

```
set vio_paths ""
set total_power ""
set cell_area ""

while {[gets $fp line] != -1} {

    if {[regexp {WNS\s*:\s*(-?\d+\.?\d*)} $line -> value]} {
        set wns $value
    }

    if {[regexp {TNS\s*:\s*(-?\d+\.?\d*)} $line -> value]} {
        set tns $value
    }

    if {[regexp {Violating Paths\s*:\s*(\d+)} $line -> value]} {
        set vio_paths $value
    }

    if {[regexp {Total Power\s*:\s*(-?\d+\.?\d*)} $line -> value]} {
        set total_power $value
    }

    if {[regexp {Cell Area\s*:\s*(-?\d+\.?\d*)} $line -> value]} {
        set cell_area $value
    }
}
```

```
close $fp


puts "WNS           = $wns"
puts "TNS           = $tns"
puts "Violating Paths= $vio_paths"
puts "Total Power    = $total_power mW"
puts "Cell Area      = $cell_area"
```

🔍 What Each Regex Is Doing (Simple)

Field	Regex Logic
WNS	Capture negative/decimal number
TNS	Same numeric pattern
Violating Paths	Capture integer only
Total Power	Capture decimal number
Cell Area	Capture decimal number

☑ Expected Output (From Your Report)

```
WNS           = -0.35
TNS           = -12.40
Violating Paths= 6
Total Power    = 87.32 mW
Cell Area      = 12543.78
```

Basics of regex Example from YouTube:

```
#!/usr/bin/tclsh
puts [regexp "a" "mux"]
```

```
puts [regexp "u" "mux"]
puts [regexp "mu" "mux"]
puts [regexp "mx" "mux"]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
0
1
1
0

puts [regexp "startpoint" "startpoint: u1_a/ck"]
puts [regexp "m." "sum"]
puts [regexp "m." "sumo"]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
1
0
1

puts [regexp "\d" "1"]
puts [regexp {\d} "1"]
puts [regexp "\\d" "1"]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
0
1
1

puts [regexp {\d} "lu"]
puts [regexp {\D} "lu"]
```

```
puts [regexp {\D} "456"]
puts [regexp {\D} "pqr345"]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
0
1
0
1

puts [regexp {[a-z]} "lu"]
puts [regexp {[a-e]} "lu"]
puts [regexp "^a" "lau"]
puts [regexp "^a" "au"]
```

Simulation Output:

```
1
0
0
1

#!/usr/bin/tclsh

puts [regexp {^\d} "abc"]
puts [regexp {^\d} "1abc"]
puts [regexp {[0-9]} "1ba"]
puts [regexp {\d$} "abc"]
puts [regexp {\d$} "pw1"]
puts [regexp {[0-9]$} "pw1"]
puts [regexp "\[0-9\]$" "pw1"]
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
0
1
1
0
1
1
1

puts [regexp "^Va*S*" "VikaS"]
puts [regexp "^Va*S" "VaS"]
puts [regexp "^Va*S" "VaaaaaaaaS"]

puts [regexp "^Va*S" "VS"]
puts [regexp "^Va*S" "VikaS"]
puts [regexp "^Va+S" "VS"]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
1
1
1
1
0
0

puts [regexp "^Va?S" "VaS"]
puts [regexp "^Va?S" "VS"]
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
```

1

1

```
set testvar "abc"
if {[regexp "^a" $testvar]} {
        puts "match yes"
} else {
        puts "Not-match"
}
```

Simulation output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
match yes
```

```
#!/usr/bin/tclsh

proc average {a1 a2 a3} {
        set sum [expr {$a1 + $a2 + $a3}]
        set result [expr {$sum / 3}]
        return $result
}
puts "The average Value is [average 2 4 6]"
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
The average Value is 4
```

हिंदी मे - NS2 Tutorial#1 - How to Execute First TCL script in NS2

```
#!/usr/bin/tclsh

puts "Welcome to VLSI Engineer"
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Welcome to VLSI Engineer
```

```
#!/usr/bin/tclsh

set a 10
set b "welcome"
set c 12.23

puts "print value $a $b & $c"
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
print value 10 welcome & 12.23
```

Simulation Output:

```
#!/usr/bin/tclsh

set a 10
set b "welcome"
```

```
set c 12.23

puts [expr $a + $c]

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
22.23

#!/usr/bin/tclsh

set a 10
set b "welcome"
set c 12.23

set d [expr $a + $c]
puts $d
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
22.23

#!/usr/bin/tclsh

set a 20

if {$a > 10} {
        puts "value is greater than 10"
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
value is greater than 10
```

```
#!/usr/bin/tclsh
set a 5

if {$a > 10} {
        puts "value is greater than 10"
} else {
        puts "value is smaller than 10"
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
value is smaller than 10



#!/usr/bin/tclsh
set a 1

while { $a < 11} {

        puts "Print the value upto: $a"


        set a [expr $a + 1]
}
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
Print the value upto: 1
Print the value upto: 2
Print the value upto: 3
```

Print the value upto: 4

Print the value upto: 5

Print the value upto: 6

Print the value upto: 7

Print the value upto: 8

Print the value upto: 9

Print the value upto: 10

```
#!/usr/bin/tclsh

set a 1

for {set a 1} { $a < 11} {incr a} {
        puts "print the value of $a"
}
```

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
print the value of 1
print the value of 2
print the value of 3
print the value of 4
print the value of 5
print the value of 6
print the value of 7
print the value of 8
print the value of 9
print the value of 10
```

```
#!/usr/bin/tclsh

proc sum {a b} {
        set c [expr $a + $b]

        puts "the sum of two number is $c"
}
sum 10 20

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
the sum of two number is 30

#!/usr/bin/tclsh

proc sum {a b} {
        set c [expr $a + $b]

        return $c
}
set d [sum 10 20]
puts "print the values of $d"

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
print the values of 30

#!/usr/bin/tclsh
```

```
set a 10
puts $a

proc change { } {

    global a
    set a 20
}
change
puts $a
```

Simulation Output:

```
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
10
20
```

File will be created and store the information in file not on the terminal

```
#!/usr/bin/tclsh

set fp [open "one.txt" w]
puts $fp “welcome to VLSI ENGG. : sarif ahmad”

close $fp
```

Read operation of File

```
#!/usr/bin/tclsh
```

```
set fp [open "one.txt" r]
gets $fp data

puts $data
close $fp
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
welcome to VLSI ENGG. : sarif ahmad
```

simplest TCL code **to calculate** x^5 (x to the power 5).

```
#!/usr/bin/tclsh
set x 2
set result [expr {$x ** 5}]

puts "x power 5 = $result"
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
x power 5 = 32
```

simplest TCL code **to calculate: - x5+x3+x2+x+1**

```
#!/usr/bin/tclsh

set x 2
set result [expr {$x**5 + $x**3 + $x**2 + $x + 1}]

puts "Expression of Result = $result"
```

Simulation Output:

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

Expression of Result = 47

4×4 square using * in TCL

```
#!/usr/bin/tclsh

for {set i 0} {$i < 4} {incr i} {
        puts "****"
}
```

Simulation Output:

[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl

```
****
****
****
****
```

Method-2

```
set n 4
for {set i 0} {$i < $n} {incr i} {
    puts [string repeat "*" $n]
}
```

Triangle (Right handle triangle) using * in TCL

```
#!/usr/bin/tclsh

for {set i 1} {$i <= 4} {incr i} {
    puts [string repeat "*" $i]
```

```
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
*
**
***
****
```

Inverted right-angled triangle

```
#!/usr/bin/tclsh

for {set i 4} {$i >= 1} {incr i -1} {
    puts [string repeat "*" $i]
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
****
***
**
*
```

Centroid triangle (Pyramid)

```
#!/usr/bin/tclsh

set n 4
for {set i 1} {$i <= $n} {incr i} {
    puts "[string repeat " " [expr {$n-$i}]][string repeat "*" [expr {2*$i-1}]]"
}
[b23n_sarif_727@prac3 tck_lab]$ ./practice.tcl
   *
  ***
```

*****

****

Lecture-1

Why TCL is important?

Whatever TCL script have written in executed inside the EDA tools like cadence and synopsis and other tolls like design compiler and icc2 and prime time tool are compatible with TCL script.

[b23n_sarif_727@prac3 tck_lab]$ cd online_tcl/

[b23n_sarif_727@prac3 online_tcl]$ ls

[b23n_sarif_727@prac3 online_tcl]$ pwd

/ve/po_home/b23n_sarif_727/Documents/tck_lab/online_tcl

[b23n_sarif_727@prac3 online_tcl]$ ls -lrt

total 0

[b23n_sarif_727@prac3 online_tcl]$ echo $SHELL

/bin/bash

[b23n_sarif_727@prac3 online_tcl]$ tclsh

% set a 5

5

% puts $a

5

% puts "Hello world"

Hello world

% set b 6

6

% puts $b

6

```
% expr $a +$b
11
% set sum [expr $a+$b]
11
% puts $sum
11
% puts "The sum of two number is: $a & $b is $sum"
The sum of two number is: 5 & 6 is 11
% set c [expr $a+$b]
11
% echo $a
5
% exit
[b23n_sarif_727@prac3 online_tcl]$ echo $SHELL
/bin/bash
[b23n_sarif_727@prac3 online_tcl]$ ls -lrt
total 0
[b23n_sarif_727@prac3 online_tcl]$ vi script1.tcl
#!/usr/bin/tclsh

set a 4
set b 5

set sum [expr $a+$b]
puts "The sum of $a & $b is:$sum"
[b23n_sarif_727@prac3 online_tcl]$ vi script1.tcl
[b23n_sarif_727@prac3 online_tcl]$ ls -lrt
```

```
total 4
-rw-r--r-- 1 b23n_sarif_727 po 73 Feb  8 18:03 script1.tcl
[b23n_sarif_727@prac3 online_tcl]$ cat script1.tcl

set a 4
set b 5

set sum [expr $a+$b]
puts "The sum of $a & $b is:$sum"
[b23n_sarif_727@prac3 online_tcl]$ tclsh script1.tcl
The sum of 4 & 5 is:9
[b23n_sarif_727@prac3 online_tcl]$ which tclsh
/usr/bin/tclsh
[b23n_sarif_727@prac3 online_tcl]$ vi script1.tcl
[b23n_sarif_727@prac3 online_tcl]$ ./script1.tcl
bash: ./script1.tcl: Permission denied
[b23n_sarif_727@prac3 online_tcl]$ chmod 777 script1.tcl
[b23n_sarif_727@prac3 online_tcl]$ ls -lrt
total 4
-rwxrwxrwx 1 b23n_sarif_727 po 88 Feb  8 18:08 script1.tcl
[b23n_sarif_727@prac3 online_tcl]$ ./script1.tcl
% exit
./script1.tcl: line 7: puts: command not found
[b23n_sarif_727@prac3 online_tcl]$ ./script1.tcl
% exit
./script1.tcl: line 7: puts: command not found
[b23n_sarif_727@prac3 online_tcl]$ ls
```

script1.tcl

[b23n_sarif_727@prac3 online_tcl]$ vi script1.tcl

[b23n_sarif_727@prac3 online_tcl]$ ./script1.tcl

The sum of 4 & 5 is:9

[b23n_sarif_727@prac3 online_tcl]$ ./script2.tcl

bash: ./script2.tcl: Permission denied

[b23n_sarif_727@prac3 online_tcl]$ chmod 777 script2.tcl

```
#!/usr/bin/tclsh

set a 1
set b a
set c 1a
set d "1c.2b"
set e "1a.2b_123@123"

puts "the value of a is : $a"
puts "the value of b is : $b"
puts "the value of c is : $c"
puts "the value of d is : $d"
puts "the value of e is : $e"
```

[b23n_sarif_727@prac3 online_tcl]$ ./script2.tcl

the value of a is : 1

the value of b is : a

the value of c is : 1a

the value of d is : 1c.2b

the value of e is : 1a.2b_123@123

```
#!/usr/bin/tclsh

set a 1
set b a
set c 1a
set d "1c.2b"
set e "1a.2b_123@123"

puts "the value of a is : $a"
puts "the value of b is : $b"
puts "the value of c is : $c"
puts "the value of d is : $d"
puts "the value of e is : $e"

puts "$a $b : $c -- $d == $e"

puts "$a $b"
puts "$c"

puts "$d $e"
[b23n_sarif_727@prac3 online_tcl]$ ./script2.tcl
the value of a is : 1
the value of b is : a
the value of c is : 1a
the value of d is : 1c.2b
the value of e is : 1a.2b_123@123
1 a : 1a -- 1c.2b == 1a.2b_123@123
```

1 a

1a

1c.2b 1a.2b_123@123

puts -nonewline "$a $b"

puts "$c"

puts "$d $e"

solution:

1 a : 1a -- 1c.2b == 1a.2b_123@123

1 a1a

1c.2b 1a.2b_123@123

List

#!/usr/bin/tclsh

set f "1 2 3 4 5 6 7 8 9"

set g [list a b c d e f g h]

set h {A B C D E F G H}

set i [list 1 1.5 abcd abcd1234]

puts $f

puts $g

puts $h

puts $i

[b23n_sarif_727@prac3 online_tcl]$ ./script3.tcl

1 2 3 4 5 6 7 8 9

a b c d e f g h

A B C D E F G H

1 1.5 abcd abcd1234

#!/usr/bin/tclsh

set f "1 2 3 4 5 6 7 8 9"

set g [list a b c d e f g h]

set h {A B C D E F G H}

set i [list 1 1.5 abcd abcd1234]

puts "the valud of f is $f"

puts "the value of g is $g"

puts "the value of h is $h"

puts "the value of i is $i"

Output solution:

[b23n_sarif_727@prac3 online_tcl]$ ./script3.tcl

the valud of f is 1 2 3 4 5 6 7 8 9

the value of g is a b c d e f g h

the value of h is A B C D E F G H

the value of i is 1 1.5 abcd abcd1234

puts {the value of i is $i}

the value of i is $i

[b23n_sarif_727@prac3 online_tcl]$ vi script4.tcl

```
[b23n_sarif_727@prac3 online_tcl]$ chmod 777 *
[b23n_sarif_727@prac3 online_tcl]$ ls
script1.tcl  script2.tcl  script3.tcl  script4.tcl
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
the Names are ajay vijay sunil gaurav
the surname are kumar singh goel gupta
```

```
#!/usr/bin/tclsh

set names [list ajay vijay sunil gaurav]
set surnames [list kumar singh goel gupta]

puts "the Names are $names"
puts "the surname are $surnames"

set n1 [lindex $names 0]
set n2 [lindex $names 0]

puts $n1
puts $n2
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
the Names are ajay vijay sunil gaurav
the surname are kumar singh goel gupta
ajay
ajay
```

```
#!/usr/bin/tclsh

set names [list ajay vijay sunil gaurav]
set surnames [list kumar singh goel gupta]

puts "the Names are $names"
puts "the surname are $surnames"

# for {set i 0} { $i>=0} {incr i} {
# puts [lindex $names $i]
# }
#
#foreach variable_name list_ {
#       puts $variable_name
#}

foreach var1 $names {
        puts "$var1"
}
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
the Names are ajay vijay sunil gaurav
the surname are kumar singh goel gupta
ajay
vijay
sunil
gaurav
```

```
foreach var1 $names {
        puts "$var1"
}

foreach var2 [list kumar singh goel gupta] {
        puts "$var2"
}

foreach var2 {kumar singh goel gupta} {
        puts "$var2"
}
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
the Names are ajay vijay sunil gaurav
the surname are kumar singh goel gupta
ajay
vijay
sunil
gaurav
kumar
singh
goel
gupta
kumar
singh
goel
gupta
```

```
foreach var1 [list $names $surnames] {
        puts "$var1"
}

foreach var1 $names var2 $surnames {
        puts "$var1 $var2"
}
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
the Names are ajay vijay sunil gaurav
the surname are kumar singh goel gupta
ajay vijay sunil gaurav
kumar singh goel gupta
ajay kumar
vijay singh
sunil goel
gaurav gupta

#ajay_kumar#vlsiexpert.com
foreach var1 $names var2 $surnames {
     #  puts "$var1 $var2"
        puts "${var1}_${var2}@vlsiexpert.com"
}
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
the Names are ajay vijay sunil gaurav
the surname are kumar singh goel gupta
ajay_kumar@vlsiexpert.com
```

vijay_singh@vlsiexpert.com

sunil_goel@vlsiexpert.com

gaurav_gupta@vlsiexpert.com

```
#!/usr/bin/tclsh
puts "INFO:Enter the first name of student in variable name (seperate name)"
set names [gets stdin]
puts "INFO:Enter the second name of student in variable surname (seperate surane"
set surnames [gets stdin]

#set names [list ajay vijay sunil gaurav]
#set surnames [list kumar singh goel gupta]

puts "the Names are $names"
puts "the surname are $surnames"
foreach var1 $names var2 $surnames {
    #  puts "$var1 $var2"
      puts "${var1}_${var2}_DV_2026_@vlsiexpert.com"
}
[b23n_sarif_727@prac3 online_tcl]$ ./script4.tcl
INFO:Enter the first name of student in variable name (seperate name)
sarif azaul altaf shaya
INFO:Enter the second name of student in variable surname (seperate surane
ahmad haque ahmad1 mohd
the Names are sarif azaul altaf shaya
the surname are ahmad haque ahmad1 mohd
sarif_ahmad_DV_2026_@vlsiexpert.com
```

azaul_haque_DV_2026_@vlsiexpert.com

altaf_ahmad1_DV_2026_@vlsiexpert.com

shaya_mohd_DV_2026_@vlsiexpert.com

Fle-handling

```
#!/usr/bin/tclsh


#runtime method that input file can change

#set file_name "input_1"

puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "

set file_name [gets stdin]


set fh [open $file_name r]


set content [read $fh]

puts $content

close $fh


[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl

INFO:Enter the file name that contains the name &surname for which you want to create the email IDs:

input_1

ajay kumar

vijay singh

sunil goel

gaurav gupta

sarif ahmad
```

[b23n_sarif_727@prac3 online_tcl]$ cat input_1

ajay kumar

vijay singh

sunil goel

gaurav gupta

sarif ahmad

```
#!/usr/bin/tclsh

#set file_name "input_1"
puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "
set file_name [gets stdin]

set fh [open $file_name r]

set content [read $fh]
puts $content

set fh1 [open output_from_input_1 w]
puts $fh1 $content

close $fh
close $fh1
```

[b23n_sarif_727@prac3 online_tcl]$ ls

input_1 script1.tcl script2.tcl script3.tcl script4.tcl script5.tcl

[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl

INFO:Enter the file name that contains the name &surname for which you want to create the email IDs:

input_1

ajay kumar

vijay singh

sunil goel

gaurav gupta

sarif ahmad

[b23n_sarif_727@prac3 online_tcl]$ ls

input_1 script1.tcl script3.tcl script5.tcl

output_from_input_1 script2.tcl script4.tcl

[b23n_sarif_727@prac3 online_tcl]$ vi output_from_input_1

```
#!/usr/bin/tclsh

#set file_name "input_1"
puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "
set file_name [gets stdin]

puts "INFO:Enter the file name in which you want to store the created email IDs: "
set file_name_write [gets stdin]

set fh [open $file_name r]
set fh1 [open $file_name_write w]

set content [read $fh]
puts $content
```

```
puts $fh1 $content


close $fh
close $fh1


[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
INFO:Enter the file name that contains the name &surname for which you want to create the email IDs:
input_1
INFO:Enter the file name in which you want to store the created email IDs:
output_file_fh
ajay kumar
vijay singh
sunil goel
gaurav gupta
sarif ahmad


[b23n_sarif_727@prac3 online_tcl]$ ls
input_1         output_from_input_1  script2.tcl  script4.tcl
output_file_fh  script1.tcl          script3.tcl  script5.tcl


#!/usr/bin/tclsh


set file_name "input_1"
#puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "
#set file_name [gets stdin]
```

```
#puts "INFO:Enter the file name in which you want to store the created email IDs: "
#set file_name_write [gets stdin]

set fh [open $file_name r]
#set fh1 [open $file_name_write w]

set content [read $fh]
set split_content [split $content "\n"]
puts $split_content

close $fh
[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
{ajay kumar} {vijay singh} {sunil goel} {gaurav gupta} {sarif ahmad} {}
#!/usr/bin/tclsh

set file_name "input_1"
#puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "
#set file_name [gets stdin]

#puts "INFO:Enter the file name in which you want to store the created email IDs: "
#set file_name_write [gets stdin]

set fh [open $file_name r]
#set fh1 [open $file_name_write w]

#set content [read $fh]
#set split_content [split $content "\n"]
```

```
#puts $split_content

while {[gets $fh one_line] >= 0 } {
        puts $one_line
}

close $fh
[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
ajay kumar
vijay singh
sunil goel
gaurav gupta
sarif ahmad
```

important: Try to access one line at a time

```
set file_name "input_1"
#puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "
#set file_name [gets stdin]

#puts "INFO:Enter the file name in which you want to store the created email IDs: "
#set file_name_write [gets stdin]

set fh [open $file_name r]
#set fh1 [open $file_name_write w]

#set content [read $fh]
#set split_content [split $content "\n"]
```

```
#puts $split_content

while {[gets $fh one_line] >= 0 } {
        puts "start"
        puts $one_line
        puts "End"
}

close $fh
[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
start
ajay kumar
End
start
vijay singh
End
start
sunil goel
End
start
gaurav gupta
End
start
sarif ahmad
End

#!/usr/bin/tclsh
```

```
set file_name "input_1"

set fh [open $file_name r]

set names {}
set surnames {}

while {[gets $fh one_line] >= 0 } {
        lappend names [lindex $one_line 0]
        lappend surnames [lindex $one_line 1]

}
puts $names
puts $surnames

close $fh
[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
ajay vijay sunil gaurav sarif
kumar singh goel gupta ahmad

#!/usr/bin/tclsh

set file_name "input_1"

set fh [open $file_name r]
```

```
while {[gets $fh one_line] >= 0 } {
        lappend names [lindex $one_line 0]
        lappend surnames [lindex $one_line 1]
}

foreach var1 $names var2 $surnames {
      #  puts "$var1 $var2"
        puts "${var1}_${var2}_DV_2026_@vlsiexpert.com"
}

close $fh
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
ajay_kumar_DV_2026_@vlsiexpert.com
vijay_singh_DV_2026_@vlsiexpert.com
sunil_goel_DV_2026_@vlsiexpert.com
gaurav_gupta_DV_2026_@vlsiexpert.com
sarif_ahmad_DV_2026_@vlsiexpert.com
```

```
#set file_name "input_1"
puts "INFO:Enter the file name that contains the name &surname for which you want to create the email IDs: "
set file_name [gets stdin]

puts "INFO:Enter the file name in which you want to store the created email IDs: "
set file_name_write [gets stdin]

set fh [open $file_name r]
```

```
set fh1 [open $file_name_write w]

while {[gets $fh one_line] >= 0 } {
        lappend names [lindex $one_line 0]
        lappend surnames [lindex $one_line 1]
}

foreach var1 $names var2 $surnames {
        puts $fh1 "${var1}.${var2}_DV_2026_@vlsiexpert.com"
}

close $fh
close $fh1
[b23n_sarif_727@prac3 online_tcl]$ ls
input_1         output_from_input_1  script2.tcl  script4.tcl
output_file_fh  script1.tcl          script3.tcl  script5.tcl
[b23n_sarif_727@prac3 online_tcl]$ ./script5.tcl
INFO:Enter the file name that contains the name &surname for which you want to create the email IDs:
input_1
INFO:Enter the file name in which you want to store the created email IDs:
email_created_output_file
[b23n_sarif_727@prac3 online_tcl]$ ls
email_created_output_file  output_from_input_1  script3.tcl
input_1                    script1.tcl          script4.tcl
output_file_fh             script2.tcl          script5.tcl
[b23n_sarif_727@prac3 online_tcl]$ vi email_created_output_file
```

Lecture-2

Proc in Lecture-2

```
#!/usr/bin/tclsh

# Proc - Procedure (Function in c lang.) (this is a non-argument proc function)
proc sum {} {
        set a 3
        set b 5
        set sum [expr $a + $b]
}
puts "The sum of a & b is [sum]"

set a 5
set b 6

set sum1 [expr $a + $b]

set c 10
set sum2 [expr $sum1 + $c]
puts "$sum2"
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc.tcl
The sum of a & b is 8
21
```

```
#!/usr/bin/tclsh
```

```
# Proc - Procedure (Function in c lang.)
proc sum_arg {A B} {
        set a $A
        set b $B
        set ADD [expr $a + $b]
}
puts "The addition of 4 & 5 is [sum_arg 4 5]"
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc.tcl
The addition of 4 & 5 is 9
```

```
#!/usr/bin/tclsh

# Proc - Procedure (Function in c lang.)
proc sum_arg {A B} {
        #set a $A
        #set b $B
        set ADD [expr $A + $B]
}
puts "The addition of 4 & 5 is [sum_arg 4 5]"
puts "The addition of 2 & 7 is [sum_arg 2 7]"
puts "The addition of 3 & 8 is [sum_arg 3 8]"
puts "The addition of 4 & 9 is [sum_arg 4 9]"
puts "The addition of 5 & 3 is [sum_arg 5 3]"
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc.tcl
```

The addition of 4 & 5 is 9

The addition of 2 & 7 is 9

The addition of 3 & 8 is 11

The addition of 4 & 9 is 13

The addition of 5 & 3 is 8

```
#!/usr/bin/tclsh

# Proc - Procedure (Function in c lang.)  (Flexible coding)
proc sum_arg {A B} {
        #set a $A
        #set b $B
        set ADD [expr $A + $B]
}
puts "The addition of 4 & 5 is [sum_arg 4 5]"
puts "The addition of 2 & 7 is [sum_arg 2 7]"

#puts "The addition of 4 & 4 is [sum_arg [gets stdin] [gets stdin]]"

puts "enter the value of a1 :"
set a1 [gets stdin]
puts "enter the value of b1 :"
set b1 [gets stdin]

set sum1 [sum_arg $a1 $b1]
puts "The sum of $a1 & $b1 is $sum1"
```

[b23n_sarif_727@prac3 online_tcl]$ ./proc.tcl

```
The addition of 4 & 5 is 9
The addition of 2 & 7 is 9
4
4
The addition of 4 & 4 is 8
enter the value of a1 :
100
enter the value of b1 :
250
The sum of 100 & 250 is 350
```

Lecture-3

Procedure in file handling

```
#!/usr/bin/tclsh

proc file_handling {file_name} {
        set f1 [open $file_name r]

        while {[gets $f1 data] >= 0 } {
                lappend names [lindex $data 0]
                lappend surnames [lindex $data 1]

        }
        puts "$names"
        puts "$surnames"
}
```

```
puts "Enter the input file name:"
set file1 [gets stdin]

file_handling $file1
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
input_1
ajay vijay sunil gaurav sarif
kumar singh goel gupta ahmad
```

```
#!/usr/bin/tclsh

proc file_handling {file_name} {
        set f1 [open $file_name r]

        while {[gets $f1 data] >= 0 } {
                lappend names [lindex $data 0]
                lappend surnames [lindex $data 1]

        }
#        puts "$names"
#        puts "$surnames"
        return [list $names $surnames]

}
```

```
puts "Enter the input file name:"
set file1 [gets stdin]

set proc_data [file_handling $file1]
puts $proc_data
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
output_from_input_1
{ajay vijay sunil gaurav sarif {}} {kumar singh goel gupta ahmad {}}
```

```
#!/usr/bin/tclsh

proc file_handling {file_name} {
        set f1 [open $file_name r]

        while {[gets $f1 data] >= 0 } {
                lappend names [lindex $data 0]
                lappend surnames [lindex $data 1]

        }
#       puts "$names"
#       puts "$surnames"
        return [list $names $surnames]

}
```

```
puts "Enter the input file name:"
set file1 [gets stdin]

set proc_data [file_handling $file1]

set Name [lindex $proc_data 0]
set Surname [lindex $proc_data 1]
puts $Name
puts $Surname
[b23n_sarif_727@prac3 online_tcl]$ vi input_1
[b23n_sarif_727@prac3 online_tcl]$ vi output_from_input_1
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
output_from_input_1
ajay vijay sunil gaurav sarif
kumar singh goel gupta ahmad

#!/usr/bin/tclsh

proc file_handling {file_name} {
        set f1 [open $file_name r]

        while {[gets $f1 data] >= 0 } {
                lappend names [lindex $data 0]
                lappend surnames [lindex $data 1]

        }
```

```
#       puts "$names"
#       puts "$surnames"
        return [list $names $surnames]

}
puts "Enter the input file name:"
set file1 [gets stdin]
set proc_data [file_handling $file1]

set Name [lindex $proc_data 0]
set Surname [lindex $proc_data 1]
foreach var1 $Name var2 $Surname {
        puts "${var1}_${var2}@vlsiexpert.com"
}
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
output_from_input_1
ajay_kumar@vlsiexpert.com
vijay_singh@vlsiexpert.com
sunil_goel@vlsiexpert.com
gaurav_gupta@vlsiexpert.com
sarif_ahmad@vlsiexpert.com

#!/usr/bin/tclsh

proc file_handling_read {file_name} {
        set f1 [open $file_name r]
```

```
        while {[gets $f1 data] >= 0 } {
                lappend names [lindex $data 0]
                lappend surnames [lindex $data 1]
        }
#       puts "$names"
#       puts "$surnames"
        return [list $names $surnames]
        close $f1
}
proc file_handling_write {file_name p_name p_surname} {
        set f2 [open $file_name w]
        foreach var1 $p_name var2 $p_surname {
                puts $f2 "${var1}_${var2}@vlsiexpert.com"
        }
        close $f2
}
puts "Enter the input file name:"
set file1 [gets stdin]
set proc_data [file_handling_read $file1]

set Name [lindex $proc_data 0]
set Surname [lindex $proc_data 1]

puts "Enter the output file name"
set file2 [gets stdin]
file_handling_write $file2 $Name $Surname
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
output_from_input_1
Enter the output file name
output_1
[b23n_sarif_727@prac3 online_tcl]$ ls
email_created_output_file  output_file_fh       proc.tcl     script3.tcl
input_1                    output_from_input_1  script1.tcl  script4.tcl
output_1                   proc_fun_fh.tcl      script2.tcl  script5.tcl
[b23n_sarif_727@prac3 online_tcl]$ vi output_1
ajay_kumar@vlsiexpert.com
vijay_singh@vlsiexpert.com
sunil_goel@vlsiexpert.com
gaurav_gupta@vlsiexpert.com
sarif_ahmad@vlsiexpert.com
```

Method-2

```
#!/usr/bin/tclsh

proc file_handling_read {file_name} {
        set f1 [open $file_name r]

        while {[gets $f1 data] >= 0 } {
                lappend names [lindex $data 0]
                lappend surnames [lindex $data 1]
        }
```

```
        return [list $names $surnames]
        close $f1
}
proc file_handling_write {file_name p_name p_surname} {
        set f2 [open $file_name w]
        foreach var1 $p_name var2 $p_surname {
                puts $f2 "${var1}_${var2}@vlsiexpert.com"
        }
        close $f2
}
puts "Enter the input file name:"
set proc_data [file_handling_read [gets stdin]]

puts "Enter the output file name"
file_handling_write [gets stdin] [lindex $proc_data 0] [lindex $proc_data 1]
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
output_from_input_1
Enter the output file name
output_5
[b23n_sarif_727@prac3 online_tcl]$ ls
email_created_output_file  output_file_fh       script1.tcl  script5.tcl
input_1                    output_from_input_1  script2.tcl
output_1                   proc_fun_fh.tcl      script3.tcl
output_5                   proc.tcl             script4.tcl
```

[b23n_sarif_727@prac3 online_tcl]$ vi utils.tcl

```
# This file contains all the proc defintion
# user can source this utils.tcl directly in their main use of scripts
# to make the use of these procedures

# below proc will read in a file having names & surnames and will will return them into list
proc file_handling_read {file_name} {
	set f1 [open $file_name r]

	while {[gets $f1 data] >= 0 } {
		lappend names [lindex $data 0]
		lappend surnames [lindex $data 1]
	}
	return [list $names $surnames]
	close $f1
}
#Below proc will tale names and surname in the form of list and give out the email address
proc file_handling_write {file_name p_name p_surname} {
	set f2 [open $file_name w]
	foreach var1 $p_name var2 $p_surname {
		puts $f2 "${var1}_${var2}@vlsiexpert.com"
	}
	close $f2
}
[b23n_sarif_727@prac3 online_tcl]$ vi proc_fun_fh.tcl
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
Enter the input file name:
```

output_from_input_1

Enter the output file name

output_10

[b23n_sarif_727@prac3 online_tcl]$ ls

email_created_output_file output_5 proc.tcl script4.tcl

input_1 output_file_fh script1.tcl script5.tcl

output_1 output_from_input_1 script2.tcl utils.tcl

output_10 proc_fun_fh.tcl script3.tcl

[b23n_sarif_727@prac3 online_tcl]$ vi output_10

[b23n_sarif_727@prac3 online_tcl]$ vi proc_fun_fh.tcl

```
#!/usr/bin/tclsh

source utils.tcl

puts "Enter the input file name:"
set proc_data [file_handling_read [gets stdin]]

puts "Enter the output file name"
file_handling_write [gets stdin] [lindex $proc_data 0] [lindex $proc_data 1]
```

Output file:

ajay_kumar@vlsiexpert.com

vijay_singh@vlsiexpert.com

sunil_goel@vlsiexpert.com

gaurav_gupta@vlsiexpert.com

sarif_ahmad@vlsiexpert.com

```
#!/usr/bin/tclsh
```

```
source utils.tcl

# prefined variable inside the tcl
#argv - argument value
#argc - argument count
puts "The argument entered along with the cmd execution are: $argc"
puts "The arguments entrered are : $argv"
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl
The argument entered along with the cmd execution are: 0
The arguments entrered are :
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl sarif abcd ahmad 123 321
The argument entered along with the cmd execution are: 5
The arguments entrered are : sarif abcd ahmad 123 321
```

```
#!/usr/bin/tclsh

source utils.tcl

# prefined variable inside the tcl
#argv - argument value
#argc - argument count
puts "The argument entered along with the cmd execution are: $argc"
puts "The arguments entrered are : $argv"

puts "the first value of argument is: [lindex $argv 0]"
```

puts "the third value of argument is: [lindex $argv 2]"

[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl sarif abcd ahmad 123 321

The argument entered along with the cmd execution are: 5

The arguments entrered are : sarif abcd ahmad 123 321

the first value of argument is: sarif

the third value of argument is: ahmad

#!/usr/bin/tclsh

source utils.tcl

prefined variable inside the tcl

#argv - argument value

#argc - argument count

puts "The argument entered along with the cmd execution are: $argc"

puts "The arguments entrered are : $argv"

puts "the first value of argument is: [lindex $argv 0]"

puts "the third value of argument is: [lindex $argv 10]"

set reverse_arg [lreverse $argv]

puts "The reverse of the argument is : $reverse_arg"

[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl sarif abcd ahmad 123 321

The argument entered along with the cmd execution are: 5

The arguments entrered are : sarif abcd ahmad 123 321

the first value of argument is: sarif

the third value of argument is:

The reverse of the argument is : 321 123 ahmad abcd sarif

```
#!/usr/bin/tclsh

source utils.tcl

puts "Enter the input file name as first element value & the output file name as second value to the cmd execution.NOTE:both files name should be space seperated,space sepearted with cmd also!!!"

#puts "Enter the input file name:"
set proc_data [file_handling_read [lindex $argv 0]]

#puts "Enter the output file name"
file_handling_write [lindex $argv 1] [lindex $proc_data 0] [lindex $proc_data 1]
```

```
[b23n_sarif_727@prac3 online_tcl]$ ./proc_fun_fh.tcl output_from_input_1 argumentive_output
Enter the input file name as first element value & the output file name as second value to the cmd execution.NOTE:both files name should be space seperated,space sepearted with cmd also!!!
[b23n_sarif_727@prac3 online_tcl]$ ls
argumentive_output        output_4            proc.tcl     script5.tcl
email_created_output_file  output_5           script1.tcl  utils.tcl
input_1                   output_file_fh      script2.tcl
output_1                  output_from_input_1  script3.tcl
output_10                 proc_fun_fh.tcl     script4.tcl
[b23n_sarif_727@prac3 online_tcl]$ vi argumentive_output
ajay_kumar@vlsiexpert.com
vijay_singh@vlsiexpert.com
```

sunil_goel@vlsiexpert.com

gaurav_gupta@vlsiexpert.com

sarif_ahmad@vlsiexpert.com

[b23n_sarif_727@prac3 online_tcl]$ cat argumentive_output

ajay_kumar@vlsiexpert.com

vijay_singh@vlsiexpert.com

sunil_goel@vlsiexpert.com

gaurav_gupta@vlsiexpert.com

sarif_ahmad@vlsiexpert.com

Lecture-4

TCL Practice Task_1 (SCRIPTING LANGUAGE) – VLSI EXPERT

Input-Output File handling and rearranging data

Create a file and named it “file_input.txt”. Remember, you have create a file exactly same as given. All spaces and format should be in same manner.

Create a TCL based program which will read input file and re-arrange the data in the given format.

Print the above formatted data onto the terminal & in output file “file_output.txt”

you should put inside file_input1.txt.

Name Delay Trans Load

AND1_2X 1.50ns 0.1 1.25ff

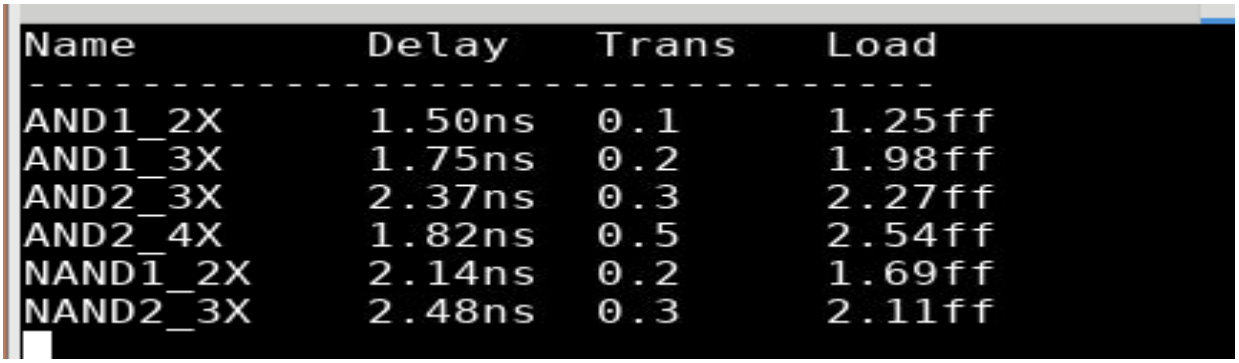
AND1_3X 1.75ns 0.2 1.98ff

AND2_3X 2.37ns 0.3 2.27ff

AND2_4X 1.82ns 0.5 2.54ff

NAND1_2X 2.14ns 0.2 1.69ff

NAND2_3X 2.48ns 0.3 2.11ff



```
Name          Delay    Trans    Load
-------------------------------------
AND1_2X       1.50ns   0.1      1.25ff
AND1_3X       1.75ns   0.2      1.98ff
AND2_3X       2.37ns   0.3      2.27ff
AND2_4X       1.82ns   0.5      2.54ff
NAND1_2X      2.14ns   0.2      1.69ff
NAND2_3X      2.48ns   0.3      2.11ff
```

reads file_input1.txt and rearranges the columns into the required format.

```
#!/usr/bin/tclsh

set fp [open "file_input1.txt" r]

puts "Name     Trans  Load   Delay"
puts "----------------------------"

gets $fp
gets $fp

while {[gets $fp line] >= 0} {
        foreach {name delay trans load} $line {}

        puts "$name  $trans  $load  $delay"
}
close $fp
```

- Opens file_input1.txt
- Skips the header lines
- Reads each data line
- Rearranges columns as:
- **Name → Trans → Load → Delay**
- Prints output in required format
- The **last {} is the loop body**, and it's empty because we only want **assignment**, not repetition.
- {} → **empty body**
- TCL does **one iteration**, assigns values, and does **nothing else**.

Simulation Output:

[b23n_sarif_727@prac3 tcl_task1]$ vi file_input1.txt

[b23n_sarif_727@prac3 tcl_task1]$ ls

file_input1.txt

[b23n_sarif_727@prac3 tcl_task1]$ vi t1.tcl

[b23n_sarif_727@prac3 tcl_task1]$ chmod 777 t1.tcl

[b23n_sarif_727@prac3 tcl_task1]$ ./t1.tcl

Name Trans Load Delay

AND1_2X 0.1 1.25ff 1.50ns

AND1_3X 0.2 1.98ff 1.75ns

AND2_3X 0.3 2.27ff 2.37ns

AND2_4X 0.5 2.54ff 1.82ns

NAND1_2X 0.2 1.69ff 2.14ns

NAND2_3X 0.3 2.11ff 2.48ns

NAND2_3X 0.3 2.11ff 2.48ns

Print the above formatted data onto the terminal & in output file `file_output.txt`

#!/usr/bin/tclsh

```
set fin [open "file_input1.txt" r]
set fout [open "file_output.txt" w]

set header "Name     Trans  Load   Delay"
set line   "-----------------------------"

puts $header
puts $line

puts $fout $header
puts $fout $line

gets $fin
gets $fin

while {[gets $fin lineData] >= 0} {

        foreach {name delay trans load} $lineData {}

        set out "$name  $trans  $load  $delay"
        puts $out

        puts $fout $out
}
close $fin
close $fout
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task1]$ ls
file_input1.txt  t1.tcl  t2.tcl
[b23n_sarif_727@prac3 tcl_task1]$ chmod 777 t2.tcl
[b23n_sarif_727@prac3 tcl_task1]$ ./t2.tcl
Name     Trans  Load   Delay
----------------------------
AND1_2X  0.1  1.25ff  1.50ns
AND1_3X  0.2  1.98ff  1.75ns
AND2_3X  0.3  2.27ff  2.37ns
AND2_4X  0.5  2.54ff  1.82ns
NAND1_2X  0.2  1.69ff  2.14ns
NAND2_3X  0.3  2.11ff  2.48ns
NAND2_3X  0.3  2.11ff  2.48ns
[b23n_sarif_727@prac3 tcl_task1]$ ls
file_input1.txt  file_output.txt  t1.tcl  t2.tcl
[b23n_sarif_727@prac3 tcl_task1]$ vi file_output.txt
[b23n_sarif_727@prac3 tcl_task1]$ vi file_input1.txt
```

☑ Step-3: TCL Program

✓ Reads `file_input1.txt`

✓ Prints formatted data on terminal

✓ Writes same data into `file_output.txt`

📜 TCL Script

```
# Open input file for reading
```

```
set fin [open "file_input1.txt" r]

# Open output file for writing
set fout [open "file_output.txt" w]

# Header
set header "Name     Trans  Load   Delay"
set line   "----------------------------"

# Print header to terminal
puts $header
puts $line

# Print header to output file
puts $fout $header
puts $fout $line

# Skip header lines in input file
gets $fin
gets $fin

# Read data lines
while {[gets $fin lineData] >= 0} {

    # Assign values
    foreach {name delay trans load} $lineData {}
```

```
    # Formatted output line
    set out "$name  $trans  $load  $delay"

    # Print to terminal
    puts $out

    # Write to output file
    puts $fout $out
}
# Close files
close $fin
close $fout
```

💻 Terminal Output:

```
Name     Trans  Load   Delay
-----------------------------
AND1_2X  0.1    1.25ff 1.50ns
AND1_3X  0.2    1.98ff 1.75ns
AND2_3X  0.3    2.27ff 2.37ns
AND2_4X  0.5    2.54ff 1.82ns
NAND1_2X 0.2    1.69ff 2.14ns
NAND2_3X 0.3    2.11ff 2.48ns
```

📁 file_output.txt

👉 Contains exact same formatted data as terminal output.

🎯 What you learned (mapped to your list)

✓ Open / close files → `open`, `close`

✓ Read / write files → `gets`, `puts`

✓ List handling → `foreach` assignment

✓ Loops → `while`, `foreach`

✓ Space handling → TCL auto-ignores extra spaces

✓ Linux-friendly script (no regexp, clean syntax)

💡 **Interview-ready line**

“I read report files using `gets`, unpack fields with `foreach`, and print results to both terminal and output files.”

What does \S+ mean?

- \S → **any character that is NOT a space**
- \+ → **one or more times**

👉 So \S+ = **a continuous word (no spaces inside)**

Method-2

```
#!/usr/bin/tclsh


set fp [open "file_input1.txt" r]


puts "Name     Trans  Load  Delay"

puts "--------------------------"


gets $fp

gets $fp
```

```
while {[gets $fp line] >=0} {

        set cols [regexp -all -inline {\S+} $line]
        set name [lindex $cols 0]
        set delay [lindex $cols 1]
        set trans [lindex $cols 2]
        set load [lindex $cols 3]

        puts "$name  $trans  $load  $delay"
}

close $fp
```

Simulation Output:

```
.txt  file_output.txt  t1.tcl  t2.tcl  t3.tcl
[b23n_sarif_727@prac3 tcl_task1]$ ./t3.tcl
Name     Trans  Load  Delay
--------------------------
AND1_2X  0.1  1.25ff  1.50ns
AND1_3X  0.2  1.98ff  1.75ns
AND2_3X  0.3  2.27ff  2.37ns
AND2_4X  0.5  2.54ff  1.82ns
NAND1_2X  0.2  1.69ff  2.14ns
NAND2_3X  0.3  2.11ff  2.48ns
```

Print **only those cells whose Delay is greater than 2ns**

```
#!/usr/bin/tclsh
```

```
set fp [open "file_input1.txt" r]

puts "Name     Trans  Load  Delay"
puts "--------------------------"

gets $fp
gets $fp

while {[gets $fp line] >=0} {

        set cols [regexp -all -inline {\S+} $line]
        set name [lindex $cols 0]
        set delay [lindex $cols 1]
        set trans [lindex $cols 2]
        set load [lindex $cols 3]

        set delay_val [string trimright $delay "ns"]

        if {$delay_val > 2.0} {
                puts "$name $trans $load $delay"
        }
}
close $fp
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task1]$ ./t3.tcl
Name     Trans  Load  Delay
```

AND2_3X 0.3 2.27ff 2.37ns

NAND1_2X 0.2 1.69ff 2.14ns

NAND2_3X 0.3 2.11ff 2.48ns

TCL Practice Task_2 (SCRIPTING LANGUAGE) – VLSI EXPERT-

Mapping two files

Create two input files (file_input1.txt and file_input2.txt).

Reuse the TCL program of Task and convert into procedure or procedures.

Write a TCL program, where you will read both input files and as per cell names present in the second file. You have to extract below data and display it in the below format.

Total_delay = Delay of cell + Delay of previous stage.

Print the above formatted data onto the terminal & in output file “file_output.txt”

File:1

file_input1.txt file_input2.txt

Name Delay Trans Load

AND1_2X 1.50ns 0.1 1.25ff

AND1_3X 1.75ns 0.2 1.98ff

AND2_3X 2.37ns 0.3 2.27ff

AND2_4X 1.82ns 0.5 2.54ff

NAND1_2X 2.14ns 0.2 1.69ff

NAND2_3X 2.48ns 0.3 2.11ff

File:2

CELL_1: AND1_2X

CELL_2: AND2_3X

CELL_3: AND2_4X

CELL_4: NAND2_3X

```
#!/usr/bin/tclsh

proc file_1 {filename} {
	set fp [open $filename r]
	set cell_dict {}

	gets $fp
	gets $fp

	while {[gets $fp line] >=0} {

		set cols [regexp -all -inline {\S+} $line]

		set name  [lindex $cols 0]
		set delay [lindex $cols 1]
		set trans [lindex $cols 2]
		set load  [lindex $cols 3]

		dict set cell_dict $name [list $trans $load $delay]
	}
	close $fp
	return $cell_dict
}
proc file_2 {filename} {
```

```
        set fp [open $filename r]
        set cell_list {}

        while {[gets $fp line] >=0} {
             if {$line eq ""} {
                  continue
             }
             set cols [regexp -all -inline {\S+} $line]
             lappend cell_list [lindex $cols 1]
        }
        close $fp
        return $cell_list
}

set cell_1 [file_1 "file_input1.txt"]
set cell_2 [file_2 "file_input2.txt"]

puts "Name      Trans   Load   Delay   Total_Delay"
puts "-----------------------------------------"

set total_delay 0

foreach cell $cell_2 {
        if {[dict exists $cell_1 $cell]} {
             set values [dict get $cell_1 $cell]
             set trans  [lindex $values 0]
```

```
            set load   [lindex $values 1]
            set delay  [lindex $values 2]

            set delay_val [string trimright $delay "ns"]
            set total_delay [expr {$total_delay + $delay_val}]

            puts "$cell    $trans   $load  ${delay_val}ns  ${total_delay}ns"
        }
}
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task2]$ ./t4.tcl
Name     Trans   Load   Delay   Total_Delay
-----------------------------------------
AND1_2X    0.1   1.25ff  1.50ns  1.5ns
AND2_3X    0.3   2.27ff  2.37ns  3.87ns
AND2_4X    0.5   2.54ff  1.82ns  5.69ns
NAND2_3X    0.3   2.11ff  2.48ns  8.17ns
```

Part-4

```
#!/usr/bin/tclsh

proc file_1 {filename} {
    set fp [open $filename r]
    set cell_dict {}

    gets $fp
    gets $fp
```

```
        while {[gets $fp line] >=0} {

                set cols [regexp -all -inline {\S+} $line]

                set name  [lindex $cols 0]
                set delay [lindex $cols 1]
                set trans [lindex $cols 2]
                set load  [lindex $cols 3]

                dict set cell_dict $name [list $trans $load $delay]
        }
        close $fp
        return $cell_dict
}

proc file_2 {filename} {

        set fp [open $filename r]
        set cell_list {}

        while {[gets $fp line] >=0} {
                if {$line eq ""} {
                        continue
                }
                set cols [regexp -all -inline {\S+} $line]
                lappend cell_list [lindex $cols 1]
```

```
        }
        close $fp
        return $cell_list
}

set cell_1 [file_1 "file_input1.txt"]
set cell_2 [file_2 "file_input2.txt"]

set fp_out [open "file_output2.txt" w]

puts "Name      Trans   Load   Delay   Total_Delay"
puts "-----------------------------------------"

puts $fp_out "Name      Trans   Load   Delay   Total_Delay"
puts $fp_out "-----------------------------------------"

set total_delay 0

foreach cell $cell_2 {
        if {[dict exists $cell_1 $cell]} {
                set values [dict get $cell_1 $cell]
                set trans  [lindex $values 0]
                set load   [lindex $values 1]
                set delay  [lindex $values 2]

                set delay_val [string trimright $delay "ns"]
                set total_delay [expr {$total_delay + $delay_val}]
```

```
            puts "$cell    $trans   $load  ${delay_val}ns  ${total_delay}ns"
            puts $fp_out "$cell    $trans   $load  ${delay_val}ns  ${total_delay}ns"

        }
}
close $fp_out
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task2]$ vi t4.tcl
[b23n_sarif_727@prac3 tcl_task2]$ ./t4.tcl
Name     Trans   Load   Delay   Total_Delay
-------------------------------------------
AND1_2X    0.1   1.25ff  1.50ns  1.5ns
AND2_3X    0.3   2.27ff  2.37ns  3.87ns
AND2_4X    0.5   2.54ff  1.82ns  5.69ns
NAND2_3X    0.3   2.11ff  2.48ns  8.17ns
[b23n_sarif_727@prac3 tcl_task2]$ ls
file_input1.txt  file_input2.txt  file_output2.txt  t4.tcl  t5.tcl
[b23n_sarif_727@prac3 tcl_task2]$ vi file_output2.txt
```

Method-2

```
#!/usr/bin/tclsh

set fp [open "file_input1.txt" r]
set tim_dict {}

gets $fp
```

```
gets $fp

while {[gets $fp line] >= 0} {
    set c [regexp -all -inline {\S+} $line]
    dict set tim_dict [lindex $c 0] \
        [list [lindex $c 2] [lindex $c 3] [lindex $c 1]]
}
close $fp

set cell_fp [open "file_input2.txt" r]
set total_delay 0

puts "Name      Trans   Load   Delay   Total_Delay"
puts "-------------------------------------------"
while {[gets $cell_fp line] >= 0} {
    if {$line eq ""} continue

    set cell [lindex [regexp -all -inline {\S+} $line] 1]

    if {[dict exists $tim_dict $cell]} {
        lassign [dict get $tim_dict $cell] trans load delay

        set d [string trimright $delay "ns"]
        set total_delay [expr {$total_delay + $d}]

        puts "$cell    $trans   $load   ${d}ns   ${total_delay}ns"
    }
```

```
}
close $cell_fp
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task2]$ ./t5.tcl
Name      Trans   Load   Delay   Total_Delay
-------------------------------------------------
AND1_2X    0.1   1.25ff   1.50ns   1.5ns
AND2_3X    0.3   2.27ff   2.37ns   3.87ns
AND2_4X    0.5   2.54ff   1.82ns   5.69ns
NAND2_3X    0.3   2.11ff   2.48ns   8.17ns
```

TCL Practice Task_3 (SCRIPTING LANGUAGE) – VLSI EXPERT

User input and Runtime decision

File:1

file_input1.txt

```
Name       Delay   Trans   Load
--------------------------------
AND1_2X     1.50ns  0.1     1.25ff
AND1_3X     1.75ns  0.2     1.98ff
AND2_3X     2.37ns  0.3     2.27ff
AND2_4X     1.82ns  0.5     2.54ff
NAND1_2X    2.14ns  0.2     1.69ff
NAND2_3X    2.48ns  0.3     2.11ff
```

TCL Code – User Input & Runtime Decision-

```
#!/usr/bin/tclsh

set fp [open "file_input1.txt" r]
set cell_dict {}

gets $fp
gets $fp

while {[gets $fp line] >=0} {

	set cols [regexp -all -inline {\S+} $line]

	set name  [lindex $cols 0]
	set delay [lindex $cols 1]
	set trans [lindex $cols 2]
	set load  [lindex $cols 3]

	dict set cell_dict $name [list $trans $load $delay]
	}
	close $fp

puts "Show the available cells in the file: "
puts "--------------------"

foreach cell [dict keys $cell_dict] {
	puts "$cell"
}
```

```
puts "\nEnter cell name: space seperated"
gets stdin user_cells

puts "\nName      Trans   Load   Delay   Total_Delay"
puts "-------------------------------------------"

set total_delay 0

foreach cell $user_cells {
        if {[dict exists $cell_dict $cell]} {
                set values [dict get $cell_dict $cell]
                set trans  [lindex $values 0]
                set load   [lindex $values 1]
                set delay  [lindex $values 2]

                set delay_val [string trimright $delay "ns"]
                set total_delay [expr {$total_delay + $delay_val}]

                puts "$cell    $trans   $load  ${delay_val}ns  ${total_delay}ns"
        } else {
                puts "Cell : $cell is not found in file"
        }
}
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task3]$ vi t4.tcl
```

```
[b23n_sarif_727@prac3 tcl_task3]$ ./t4.tcl
Show the available cells in the file:
------------------
AND1_2X
AND1_3X
AND2_3X
AND2_4X
NAND1_2X
NAND2_3X


Enter cell name: space seperated
AND1_2X

Name      Trans   Load   Delay   Total_Delay
-------------------------------------------
AND1_2X    0.1   1.25ff  1.50ns  1.5ns

[b23n_sarif_727@prac3 tcl_task3]$ ./t4.tcl
Show the available cells in the file:
------------------
AND1_2X
AND1_3X
AND2_3X
AND2_4X
NAND1_2X
NAND2_3X
```

Enter cell name: space seperated

AND1_2X AND1_3X AND2_3X AND2_4X NAND1_2X NAND2_3X

Name Trans Load Delay Total_Delay

AND1_2X 0.1 1.25ff 1.50ns 1.5ns

AND1_3X 0.2 1.98ff 1.75ns 3.25ns

AND2_3X 0.3 2.27ff 2.37ns 5.62ns

AND2_4X 0.5 2.54ff 1.82ns 7.44ns

NAND1_2X 0.2 1.69ff 2.14ns 9.58ns

NAND2_3X 0.3 2.11ff 2.48ns 12.06ns

Print the above formated data as per user input onto the terminal & in output file "file_output2.txt"

```
#!/usr/bin/tclsh

set fp [open "file_input1.txt" r]
set cell_dict {}

gets $fp
gets $fp

while {[gets $fp line] >=0} {

    set cols [regexp -all -inline {\S+} $line]
```

```
        set name  [lindex $cols 0]
        set delay [lindex $cols 1]
        set trans [lindex $cols 2]
        set load  [lindex $cols 3]

        dict set cell_dict $name [list $trans $load $delay]
        }
        close $fp

puts "Show the available cells in the file: "
puts "--------------------"

foreach cell [dict keys $cell_dict] {
        puts "$cell"
}

puts "\nEnter cell name: space seperated"
gets stdin user_cells

set out_fp [open "file_output2.txt" w]

puts "\nName      Trans   Load   Delay   Total_Delay"
puts "-----------------------------------------"
puts $out_fp "Name      Trans   Load   Delay   Total_Delay"
puts $out_fp "-----------------------------------------"
```

```
set total_delay 0

foreach cell $user_cells {
        if {[dict exists $cell_dict $cell]} {
                set values [dict get $cell_dict $cell]
                set trans  [lindex $values 0]
                set load   [lindex $values 1]
                set delay  [lindex $values 2]

                set delay_val [string trimright $delay "ns"]
                set total_delay [expr {$total_delay + $delay_val}]

                set print "$cell    $trans   $load  ${delay_val}ns  ${total_delay}ns"
                puts $print
                puts $out_fp $print
        } else {
                puts "Cell : $cell is not found in file"
        }
}
close $out_fp
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task3]$ ./t5.tcl
Show the available cells in the file:
------------------
AND1_2X
AND1_3X
AND2_3X
```

AND2_4X

NAND1_2X

NAND2_3X

Enter cell name: space seperated

AND1_2X AND1_3X AND2_3X AND2_4X NAND1_2X NAND1_3X

Name Trans Load Delay Total_Delay

AND1_2X 0.1 1.25ff 1.50ns 1.5ns

AND1_3X 0.2 1.98ff 1.75ns 3.25ns

AND2_3X 0.3 2.27ff 2.37ns 5.62ns

AND2_4X 0.5 2.54ff 1.82ns 7.44ns

NAND1_2X 0.2 1.69ff 2.14ns 9.58ns

Cell : NAND1_3X is not found in file

[b23n_sarif_727@prac3 tcl_task3]$ ls

file_input1.txt file_output2.txt t4.tcl t5.tcl

TCL Practice Task_4 (SCRIPTING LANGUAGE) – VLSI EXPERT-

Command Line Development- Using switch statement

File:1

file_input1.txt

Name Delay Trans Load

AND1_2X 1.50ns 0.1 1.25ff

```
AND1_3X     1.75ns  0.2     1.98ff
AND2_3X     2.37ns  0.3     2.27ff
AND2_4X     1.82ns  0.5     2.54ff
NAND1_2X    2.14ns  0.2     1.69ff
NAND2_3X    2.48ns  0.3     2.11ff
```

```
#!/usr/bin/tclsh

set args $argv
set show_trans 0
set show_load 0
set show_delay 0
set show_total 0
set cell_names ""

foreach arg $args {
   switch -- $arg {
      "-trans"        { set show_trans 1 }
      "-load"         { set show_load 1 }
      "-delay"        { set show_delay 1 }
      "-total_delay"  { set show_total 1 }
      default {
         if {[string match "-cell_name*" $arg]} {
            set cell_names [lindex [split $arg "="] 1]
         }
      }
   }
```

```
}

if {$cell_names eq ""} {
    puts "ERROR: -cell_name is required"
    puts "Usage: ./task4.tcl -cell_name AND1_2X,AND2_3X [-trans] [-load] [-delay] [-total_delay]"
    exit
}

if {!$show_trans && !$show_load && !$show_delay && !$show_total} {
    puts "ERROR: At least one of -trans, -load, -delay, -total_delay is required"
    exit
}

set fp [open "file_input1.txt" r]
set cell_dict {}

gets $fp
gets $fp

while {[gets $fp line] >= 0} {
    set cols [regexp -all -inline {\S+} $line]
    set name  [lindex $cols 0]
    set delay [lindex $cols 1]
    set trans [lindex $cols 2]
    set load  [lindex $cols 3]

    dict set cell_dict $name [list $trans $load $delay]
```

```
}
close $fp

puts -nonewline "Name    "

if {$show_trans}  { puts -nonewline " Trans" }
if {$show_load}   { puts -nonewline " Load" }
if {$show_delay}  { puts -nonewline " Delay" }
if {$show_total}  { puts -nonewline " Total_Delay" }
puts ""
puts "-------------------------------------------"

set total_delay 0
set cells [split $cell_names ","]

foreach cell $cells {
    if {![dict exists $cell_dict $cell]} {
        puts "ERROR: Cell $cell not found"
        continue
    }

    set values [dict get $cell_dict $cell]
    set trans  [lindex $values 0]
    set load   [lindex $values 1]
    set delay  [lindex $values 2]

    set delay_val [string trimright $delay "ns"]
```

```
    set total_delay [expr {$total_delay + $delay_val}]

    puts -nonewline "$cell"

    if {$show_trans}  { puts -nonewline " $trans" }
    if {$show_load}   { puts -nonewline " $load" }
    if {$show_delay}  { puts -nonewline " $delay" }
    if {$show_total}  { puts -nonewline " ${total_delay}ns" }

    puts ""
}
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task4]$ ./t1.tcl -cell_name=AND1_2X -trans -load -delay -total_delay
Name    Trans Load Delay Total_Delay
-----------------------------------------
AND1_2X 0.1 1.25ff 1.50ns 1.5ns

[b23n_sarif_727@prac3 tcl_task4]$ ./t1.tcl -cell_name=AND1_2X,AND2_3X -trans -load -delay -total_delay
Name    Trans Load Delay Total_Delay
-----------------------------------------
AND1_2X 0.1 1.25ff 1.50ns 1.5ns
AND2_3X 0.3 2.27ff 2.37ns 3.87ns
[b23n_sarif_727@prac3 tcl_task4]$ ./t1.tcl -cell_name=AND1_2X -trans -load -delay -total_delay
Name    Trans Load Delay Total_Delay
```

AND1_2X 0.1 1.25ff 1.50ns 1.5ns

[b23n_sarif_727@prac3 tcl_task4]$ ./t1.tcl -cell_name=AND1_2X,AND1_3X,AND2_3X,AND2_4X,NAND1_2X,NAND2_3X -trans -load -delay -total_delay

Name Trans Load Delay Total_Delay

AND1_2X 0.1 1.25ff 1.50ns 1.5ns

AND1_3X 0.2 1.98ff 1.75ns 3.25ns

AND2_3X 0.3 2.27ff 2.37ns 5.62ns

AND2_4X 0.5 2.54ff 1.82ns 7.44ns

NAND1_2X 0.2 1.69ff 2.14ns 9.58ns

NAND2_3X 0.3 2.11ff 2.48ns 12.06ns

[b23n_sarif_727@prac3 tcl_task4]$ ./t1.tcl -cell_name=AND1_2X,AND1_3X,AND2_3X,AND2_4X,NAND1_2X,NAND2_3X -trans

Name Trans

AND1_2X 0.1

AND1_3X 0.2

AND2_3X 0.3

AND2_4X 0.5

NAND1_2X 0.2

NAND2_3X 0.3

in this code only modify: add- Print the above formated data as per user input onto the terminal & in output file "file_output2.txt"

```
#!/usr/bin/tclsh

set args $argv
set show_trans 0
set show_load 0
set show_delay 0
set show_total 0
set cell_names ""

foreach arg $args {
   switch -- $arg {
      "-trans"       { set show_trans 1 }
      "-load"        { set show_load 1 }
      "-delay"       { set show_delay 1 }
      "-total_delay"  { set show_total 1 }
      default {
         if {[string match "-cell_name*" $arg]} {
            set cell_names [lindex [split $arg "="] 1]
         }
      }
   }
}

if {$cell_names eq ""} {
   puts "ERROR: -cell_name is required"
   puts "Usage: ./task4.tcl -cell_name AND1_2X,AND2_3X [-trans] [-load] [-delay] [-total_delay]"
   exit
```

```
}

if {!$show_trans && !$show_load && !$show_delay && !$show_total} {
    puts "ERROR: At least one of -trans, -load, -delay, -total_delay is required"
    exit
}

set fp [open "file_input1.txt" r]
set cell_dict {}

gets $fp
gets $fp

while {[gets $fp line] >= 0} {
    set cols [regexp -all -inline {\S+} $line]
    set name  [lindex $cols 0]
    set delay [lindex $cols 1]
    set trans [lindex $cols 2]
    set load  [lindex $cols 3]

    dict set cell_dict $name [list $trans $load $delay]
}
close $fp

set fp_out [open "file_output2.txt" w]

set header  "Name    "
```

```
if {$show_trans}  { append header " Trans" }
if {$show_load}   { append header " Load" }
if {$show_delay}  { append header " Delay" }
if {$show_total}  { append header " Total_Delay" }

puts $header
puts $fp_out $header

#puts ""
puts "-----------------------------------------"
puts $fp_out "-----------------------------------------"

set total_delay 0
set cells [split $cell_names ","]

foreach cell $cells {
   if {![dict exists $cell_dict $cell]} {
      set print "ERROR: Cell $cell not found"
      puts $print
      puts $fp_out $print
      continue
   }

   set values [dict get $cell_dict $cell]
   set trans  [lindex $values 0]
   set load   [lindex $values 1]
   set delay  [lindex $values 2]
```

```
    set delay_val [string trimright $delay "ns"]
    set total_delay [expr {$total_delay + $delay_val}]

    set line  "$cell"

    if {$show_trans}  { append line " $trans" }
    if {$show_load}   { append line " $load" }
    if {$show_delay}  { append line " $delay" }
    if {$show_total}  { append line " ${total_delay}ns" }
    puts $line
    puts $fp_out $line

}
close $fp_out
```

Simulation Output:

```
[b23n_sarif_727@prac3 tcl_task4]$ ./t2.tcl -cell_name=AND1_2X,AND1_3X,AND2_3X,AND2_4X,NAND1_2X,NAND2_3X -trans -load -delay -total_delay
Name    Trans Load Delay Total_Delay
-----------------------------------------
AND1_2X 0.1 1.25ff 1.50ns 1.5ns
AND1_3X 0.2 1.98ff 1.75ns 3.25ns
AND2_3X 0.3 2.27ff 2.37ns 5.62ns
AND2_4X 0.5 2.54ff 1.82ns 7.44ns
NAND1_2X 0.2 1.69ff 2.14ns 9.58ns
NAND2_3X 0.3 2.11ff 2.48ns 12.06ns
```

[b23n_sarif_727@prac3 tcl_task4]$ ls

file_input1.txt file_output2.txt t1.tcl t2.tcl

Solution-2 After enter one wrong name of the cell

[b23n_sarif_727@prac3 tcl_task4]$ ./t2.tcl -cell_name=AND1_2X,AND1_3X,AND2_3X,AND2_4X,NAND1_8X,NAND2_3X -trans -load -delay -total_delay

Name Trans Load Delay Total_Delay

AND1_2X 0.1 1.25ff 1.50ns 1.5ns

AND1_3X 0.2 1.98ff 1.75ns 3.25ns

AND2_3X 0.3 2.27ff 2.37ns 5.62ns

AND2_4X 0.5 2.54ff 1.82ns 7.44ns

ERROR: Cell NAND1_8X not found

NAND2_3X 0.3 2.11ff 2.48ns 9.92ns